AALTO UNIVERSITY COURSE BOOK

Elements of Scalable Web Development

Architecture, measurement, scaling pressure, and agentic workflows

Juho Vepsäläinen

AALTO UNIVERSITY COURSE BOOK

Elements of Scalable Web Development

Architecture, measurement, scaling pressure, and agentic workflows

Juho Vepsäläinen

Travel edition - 28 August 2026

scalableweb.dev

READER'S MAP

Contents

This edition includes the core book, appendices, case studies, and the course project guide. Interactive activities link to the web edition.

Front matter

Introduction

Part I - Establish

Establish - Build and measure a baseline

1. What does scalability mean for web development?
2. How can coding agents support scalable development?
3. How can we measure web applications and development workflows?

Part II - Challenge

Challenge - Expose architectural assumptions

4. How does web architecture shape scalability?
5. How does development friction limit web projects?
6. Which web application assumptions fail under pressure?

Part III - Compare

Compare - Evaluate workflows and scaling responses

7. When do agent fleets improve web development?
8. How should web applications respond to scaling pressure?
9. What do three agent-assisted implementations teach us?

Appendices

Appendices

A. Evaluating agentic development workflows
B. Reproducible development environments
C. Advanced control boundaries in web applications
D. Agent-system security, control, and traceability
E. Formal scalability and reliability models
F. Durable workflows and retry-safe effects

Case studies

Slideotter - Learning faster than the product converged
Thesis Journey Tracker - Shipping by constraining the problem
Kirjolab - Exceptional speed, incomplete architectural understanding
Webernetes - Compatibility through source mapping and shared tests
Modern Web Guidance - Keeping web-platform advice current

Course project guide

Common project specification
Establish project
Challenge project
Compare project
Course development environments
Course assessment rubric

INTRODUCTION

Elements of Scalable Web Development

Architecture, measurement, scaling pressure, and agentic workflows

Scalability describes the relationship between changing pressure and the behavior people expect a system to maintain. For a web application, that pressure may come from users, data, traffic, failures, geography, or new requirements. The response may involve architecture and infrastructure, but it can also change cost, complexity, ownership, and the speed at which the application can evolve.

This book therefore studies two connected systems. The first is the web application that receives requests, moves data, preserves authority, and produces browser-visible behavior. The second is the development system of people, agents, tools, instructions, tests, and review that changes the application. Scaling one while ignoring the limits of the other does not create a scalable project.

Coding agents make this connection especially important. Writing every line of a web application by hand is no longer a sensible default for most projects: agents can now perform substantial parts of the implementation loop from a specification and feedback from their environment. That added production capacity does not remove the need for software engineering. It moves more human attention toward intent, architecture, constraints, validation, integration, and ownership while making weak feedback loops more consequential.

Agents are consequently treated here in the context of web development rather than as a separate subject. The central question is how application behavior and the development process can continue to meet their goals as pressure grows.

> **Sustainability angle**
>
> Scalability also has an environmental dimension. More traffic, more data, more retries, more logs, and more infrastructure all consume resources. Throughout the book, you can ask whether a design scales efficiently, not only whether it scales at all.

0.1 Why this book now

This book is informed by twenty years of work in web development, my research on the topic, a doctorate completed in 2025, and the conversations around agentic development that led me to organize Future Frontend. The rapid change in coding agents forced me to rethink what a book about scalable web development should teach. The result is a 2026 snapshot rather than a claim to settled best practice: use its models and evidence to form your own judgments.

You can read the chapters independently and follow their references without completing the course project. The full progression is most useful when you want to connect runtime scalability, architecture, measurement, and agent-assisted development through one sustained example.

0.2 One application, three development systems

The course project implements the same Work Activity Tracker specification three times. The required browser-visible behavior remains substantially stable while product interpretation, stack, rendering, storage, interfaces, and architecture remain open to justified choices.

Each phase begins from a fresh implementation and changes the development setup deliberately:

1. **Establish:** a locally hosted model in a single-agent workflow produces implementation A and its evidence baseline.
2. **Challenge:** a foundation model in a single-agent workflow produces implementation B while the course exposes architectural assumptions and development friction.
3. **Compare:** a declared agent fleet produces implementation C before all three applications and development workflows are compared.

The comparison is bounded rather than perfectly controlled. Students learn the domain between phases, implementations may make different design choices, and model or tool configurations can change. The shared specification, acceptance material, pressures, and reporting categories make those differences discussable without pretending that the course establishes a universal ranking.

Each main chapter ends with a **Project artifact** section. These sections are staged checkpoints for building the current phase's evidence; they do not silently create separate submissions. The common project specification and current phase specification provide the authoritative deliverable checklist, and one clearly structured artifact may satisfy several chapter checkpoints.

> **Protect learning, not only output**
>
> Immediate task performance is not the only outcome worth protecting. Across three randomized experiments involving short arithmetic and reading-comprehension tasks, brief AI assistance improved work while it was available but reduced later unaided performance and persistence; the decline was concentrated among participants who used AI for direct solutions, while hint-oriented users did not differ significantly from the control group (Liu et al., 2026). These tasks do not establish the long-term effect on professional programming, but they support a practical course rule: use agents for hints, critique, and verification when the underlying skill must remain yours.

0.3 How the argument develops

0.3.1 Establish — Build and measure a baseline

1. What does scalability mean for web development? defines scalability through maintained goals, pressures, constraints, and evidence.
2. How can coding agents support scalable development? examines how intent, authority, validation, correction, and ownership shape agent-assisted work.
3. How can we measure web applications and development workflows? creates comparable runtime and development evidence for implementation A.

0.3.2 Challenge — Expose architectural assumptions

4. How does web architecture shape scalability? traces how responsibility moves across servers, cloud, edge, rendering, data, and caches.
5. How does development friction limit web projects? follows the cost of changing one web behavior across those boundaries.
6. Which web application assumptions fail under pressure? uses documented incidents and the course project to connect failed assumptions, signals, and recovery.

0.3.3 Compare — Evaluate workflows and scaling responses

7. When do agent fleets improve web development? grows one agent into a bounded fleet and makes correction part of the workflow.
8. How should web applications respond to scaling pressure? selects architectural responses from measured limits rather than from a technique catalog.
9. What do three agent-assisted implementations teach us about web scalability? synthesizes a justified target architecture and development workflow from the three builds.

The main chapters provide the shortest path through the argument. The appendices preserve deeper evaluation methods, implementation details, security controls, and formal models for readers who need them.

0.4 References

1. Liu, G., Christian, B., Dumbalska, T., Bakker, M. A., & Dubey, R. (2026). AI Assistance Reduces Persistence and Hurts Independent Performance. arXiv Preprint arXiv:2604.04721. https://doi.org/10.48550/arXiv.2604.04721

PART I

Establish — Build and measure a baseline

A baseline is a bounded point of comparison, not an implementation assumed to be correct or an architecture that later work must imitate. It gives the next two builds stable behavior, questions, and evidence against which meaningful differences can be discussed. In this part, you define what should remain true under pressure, establish a controlled agent-assisted development process, and measure the first implementation of the shared web application.

I.1 From scalability claims to comparable evidence

The three chapters form one progression:

1. What does scalability mean for web development? identifies the maintained goal, relevant pressures, constraints, and evidence a scalability claim would need.
2. How can coding agents support scalable development? treats the application and its development system as connected systems, with explicit constraints, validation, correction, and ownership.
3. How can we measure web applications and development workflows? turns runtime behavior and the development process into a repeatable evidence baseline.

By the end of Establish, implementation A should be modest, complete enough to exercise the common behavior, and bound to a declared revision and evidence package. Its value is not that it predicts the final design. Its value is that later comparisons do not have to rely on memory or impressions.

I.2 Implementation A — Work Activity Tracker

The Establish project specification shows what you should implement during this part of the book. You will implement the shared project through a single-agent workflow backed by the approved locally hosted model. The purpose is to experience what that particular model, agent harness, project harness, and environment make easy or difficult while producing a baseline that can be inspected and measured.

The specification leaves many decisions open deliberately, and you can customize the domain to your liking while choosing your own technical stack, rendering approach, storage, architecture, deployment path, and one specific implementation question worth investigating. There is no single required architecture, and the first choice does not need to be final. Record assumptions and trade-offs so the later builds can reveal which differences matter under the shared requirements and pressures.

The following parts begin from fresh implementations. You may carry forward the common specification, permitted acceptance material, technology-neutral lessons, and the reporting method, but not application code or implementation-specific tests. This preserves learning while keeping the three development setups meaningfully comparable.

I.3 Implementation discovery lab

Use this lab to turn the shared specification into a direction to explore. It produces a first project-direction document that you can refine as implementation creates evidence.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/parts/establish/

CHAPTER 1

What does scalability mean for web development?

A web application may cope with more requests while becoming harder to modify, accept more events while making old events too slow to query, or let more agents produce code while overwhelming review and integration. All of these are examples of how scalability can be a challenging topic. In this book, **scalability** means a system's ability to continue meeting a stated goal as a named pressure grows or changes. The goal, pressure, system boundary, and evidence are all part of the claim. Without them, scalability becomes a vague synonym for “large” or “well engineered.”

This framing is especially useful for the web because one user outcome can cross a browser, a network, an application boundary, persistent data, caches, external services, and operational tooling. The system also has to keep changing as browsers, consumers, requirements, dependencies, developers, and development agents evolve independently. Runtime scale and development scale are therefore related without being interchangeable.

The central claim of this chapter is that scalability is a relationship between **pressure** and **maintained behavior**. Architecture and infrastructure are possible responses to that relationship; they are not themselves the things that grow.

1.1 Begin with the goal that must continue to hold

Consider the event tracker at the center of the course project. At its smallest useful scale, a consumer records an event, the application validates and authorizes the request, accepted state becomes durable, and a later activity view exposes the result. A baseline should state which parts of that behavior matter rather than merely report that the application runs.

Different goals produce different scalability claims:

- A successful submission should still create one accepted event when many consumers submit concurrently.
- Recent activity should remain useful as retained history grows.
- A bounded feature change should remain understandable and reviewable as more people or agents contribute.
- An HTTP or data contract should remain evolvable as clients and requirements change independently.

Each claim can be expressed through the same frame:

> As **pressure P** changes within **range or scenario R**, the system within **boundary B** continues to meet **goal G**, as shown by **evidence E**, under **constraints C**.

The frame prevents one successful observation from becoming a universal promise. A browser measurement says little about database recovery; one load level does not prove behavior at another; and a fast generated change does not prove that another contributor can understand or reverse it. The Benchmarking chapter develops how to produce bounded evidence for these claims.

1.2 Four pressures organize the course

Many taxonomies of scalability are possible. This book uses four pressures that can be applied consistently to the event tracker and to its three independent implementations: demand, data, contributors, and change.

Table 1.1. Four pressures used to form bounded scalability claims in this book.

Pressure	What grows or changes	Goal that may need to hold	Early signals in a web project
Demand	Requests, concurrent actions, connected clients, traffic bursts, or geographic distance	Useful responses, accepted writes, latency, fairness, and correctness	Tail latency, error rate, saturation, queue depth, browser responsiveness
Data	Stored events, tenants, relationships, event variety, or retention time	Durable writes, useful queries, integrity, recovery, and bounded cost	Storage growth, write latency, query plans, replica lag, recovery time
Contributors	Developers, teams, agents, parallel tasks, or handoffs	Understandable ownership, reviewable changes, and reliable integration	Lead time, review queues, merge conflicts, rework, failed integrations
Change	Requirements, client versions, schemas, dependencies, policies, or deployment frequency	Compatible evolution, maintained behavior, and safe recovery	Change fan-out, regression rate, migration effort, failed rollbacks, recovery time

The dimensions interact without collapsing into one score. More demand usually creates more data. More contributors can increase the rate of change. Frequent schema changes can increase data migration work even when traffic stays constant. A design can therefore scale well along one axis while approaching a limit on another.

> **Sustainability angle**
>
> Meeting greater demand usually consumes more computation, storage, transfer, or human attention. A technically successful response may still be inefficient. Record resource use per useful outcome and the work displaced elsewhere instead of treating capacity as the only goal.

1.2.1 Demand changes the distribution of web behavior

Demand is not only a higher request count. More consumers create more concurrency, more varied devices and networks, more retries after ambiguous responses, and more opportunities for rare paths to occur. A browser may remain responsive while the origin saturates, or the server may respond quickly while client rendering stalls on a growing activity view.

People are not the only consumers. Integrations, crawlers, and authorized agents can call an interface repeatedly and faster than a person operating a screen. The goal should therefore describe the authoritative outcome and acceptable user experience rather than assume one kind of client behavior.

1.2.2 Data changes paths as well as volume

Data pressure includes more than disk space. As event history grows, the useful write, read, analytical, deletion, archival, and recovery paths can diverge. A query that works for a few hundred events may scan too much history later; an index that accelerates one filter can increase write cost; and keeping every derived view immediately current can make ingestion harder.

The important questions are which state is authoritative, which views can be rebuilt, which results may be stale, and how long information should remain available. Those choices determine which data goals must be maintained as volume and variety change.

1.2.3 Contributors increase coordination pressure

A codebase that one person understands informally may become difficult to change when several people, teams, or agents work in parallel. More code output does not automatically increase accepted product output. Review capacity, integration, ownership, project memory, and the ability to diagnose failed work can become binding limits first.

Agents make this pressure especially visible because they can generate several plausible changes before a person could have implemented one. Stable contracts, focused tasks, executable checks, and explicit ownership can let contributors work more independently. They also require maintenance and can become harmful if they preserve rules that no longer serve the product.

1.2.4 Change makes old commitments visible

A web application changes even when its traffic does not. Browsers, APIs, integrations, event schemas, authentication rules, libraries, hosting platforms, and user expectations evolve on different schedules. Once a consumer relies on a response or production data adopts a representation, a change may require compatibility and migration rather than a simple code replacement.

Change scalability is the ability to keep making justified changes without turning every release into a coordinated rewrite. It depends on visible boundaries, useful tests, staged evolution, and recovery paths, but no design eliminates the need to understand what has already become a commitment.

1.3 One pressure can cross the whole web system

Suppose a domain-specific event causes many consumers to submit events during the same short interval. That one demand pressure can affect several boundaries:

1. The browser must keep the interaction responsive and communicate pending, accepted, or failed state.
2. The HTTP boundary must handle concurrency, timeouts, retries, authentication, and request limits.
3. Application execution may exhaust workers, connections, memory, or dependency quotas.
4. The authoritative store must preserve event and workflow invariants under concurrent writes.
5. Queries and cached activity views must expose an acceptable level of freshness.

6. Operators need signals that distinguish user-visible failure from waiting, overload, duplication, or stale derived state.

Adding application servers helps only if application execution is the binding limit and the rest of the path can absorb the new work. Optimizing the browser helps only the work that occurs there. The response has to follow the pressure to the first limit that threatens the stated goal.

The same reasoning applies to development pressure. Adding agents can increase candidate changes while review or integration remains fixed. The result may be more waiting and rework rather than more accepted behavior. The scalable unit is therefore the complete path to a useful outcome, not the amount of activity at one stage.

1.4 Separate pressures, responses, and constraints

The earlier taxonomy deliberately names what varies. Several other concerns shape the response:

- **Architecture** assigns responsibilities, authority, contracts, and allowed dependencies across the system.
- **Infrastructure** supplies execution, storage, networking, deployment, and operational capacity.
- **Product and business priorities** determine which user outcomes matter, which risks are tolerable, and when additional capacity or coordination is worth its cost.
- **Constraints** such as budget, energy use, privacy, regulation, geography, hardware, provider quotas, time, and team expertise narrow the acceptable design space.

These are not secondary concerns. They explain why two applications facing similar demand may choose different responses. The distinction simply prevents a mechanism such as cloud infrastructure from being mistaken for the pressure it is meant to address.

A **binding limit** is the condition currently preventing the system from maintaining its goal as pressure increases. It may be a browser's main thread, network bandwidth, CPU, memory, a connection pool, database coordination, an external-service quota, a review queue, or missing expertise. Increasing capacity somewhere else can add cost and complexity without moving that limit.

1.5 Distinguish scalability from neighboring properties

Several useful properties are easy to conflate with scalability. The difference lies in the condition being studied.

Table 1.2. Scalability and related system properties answer different questions.

Property	Question	Example evidence from the event tracker
Performance	How does the system behave under one stated set of conditions?	Latency and resource use for one activity-view workload
Scalability	How does maintained behavior change as a named pressure changes?	Latency, correctness, and resource curves across increasing concurrent writes
Elasticity	Can supplied capacity expand and contract as demand changes over time?	Provisioning delay, backlog, and cost during and after a traffic burst

Property	Question	Example evidence from the event tracker
Resilience	Can the system preserve or recover useful service when something fails?	Accepted events and recovery time when storage or a dependency is unavailable
Efficiency	How much resource or coordination produces one useful outcome?	Compute, transfer, storage, or review time per accepted event or change

A system can perform well at its current load but degrade sharply as demand grows. It can add capacity automatically yet reproduce a correctness error on every instance. It can survive a dependency failure through expensive overprovisioning. No single metric establishes all five properties.

1.6 Reason from pressure to evidence

Use one causal chain before choosing a scaling technique:

```
pressure → binding limit → response → trade-off → evidence
```

- **Pressure:** name what changes and the plausible range or scenario.
- **Binding limit:** identify what first prevents the maintained goal from holding.
- **Response:** change the application, architecture, infrastructure, workflow, or scope at that limit.
- **Trade-off:** state which cost, latency, complexity, consistency, autonomy, or recovery obligation moved elsewhere.
- **Evidence:** repeat a representative observation that could challenge the claim.

For a runtime example, concurrent event submissions may exhaust the database connection pool before CPU becomes scarce. Admission control or buffering might protect accepted writes, but it can add waiting and require an explicit pending state. A useful evaluation would inspect success, duplication, tail latency, queue age, and recovery as the workload changes rather than only count server instances.

For a development example, parallel agent tasks may produce changes faster than people can review them. Smaller task boundaries, executable contracts, and automated checks might move repeatable findings earlier, but maintaining those controls also consumes time. Evidence should cover accepted outcomes, review delay, rework, failed integrations, and human intervention rather than generated lines of code.

This chain does not guarantee the correct response. It makes the hypothesis explicit enough to test and revise.

1.7 Constraints prevent premature scaling

Scalability is not a requirement to prepare every component for unlimited growth. An implementation that serves its likely demand, preserves its data, and remains understandable to its owners can be more appropriate than a distributed design built for a hypothetical future. Every additional cache, queue, replica, service, region, contract, and automation layer creates a new operating and change obligation.

Premature scaling begins when a solution is selected before the pressure, goal, or limit is known. Preparation is still valuable when it preserves a cheap path to learn: keep authority clear, avoid irreversible coupling without a reason, collect a baseline, and identify the signal that would justify the

next move. The aim is not to predict the final architecture. It is to make the next decision from evidence rather than from a generic idea of scale.

For the course project, time, available hardware, privacy, model access, team expertise, and implementation scope are legitimate design inputs. A bounded local implementation can be a strong baseline precisely because its constraints are visible. Later implementations can test different hypotheses without pretending that newer or more distributed means better.

1.8 Scalability dimensions lab

The demonstration below applies four pressures to the event tracker. Select the scenarios that matter and observe which dimensions receive the greatest relative strain. The weights are deterministic teaching inputs, not measurements of a real application, so use the result to form a claim rather than to select an architecture.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/what-is-scalability/

Choose one leading dimension and write its pressure, maintained goal, likely first limit, and first useful signal. Then select a second pressure and inspect how the conclusion changes. If several dimensions rise together, preserve them as separate claims instead of averaging them into a single scalability score.

1.9 Project artifact

Students should produce two bounded scalability claims for implementation A: one about runtime demand or data, and one about contributors or change. For each claim, record:

- the goal that must continue to hold,
- the pressure and plausible range or scenario,
- the system boundary and relevant constraints,
- the likely first binding limit and signal,
- one candidate response and its expected trade-off,
- the evidence that would support or challenge the claim,
- and one dimension intentionally left outside the claim.

These claims become inputs to the shared measurement protocol in the Benchmarking chapter. They should be revised when evidence reveals a different limit, not protected as predictions that must appear correct.

1.10 Summary

Scalability describes whether a system can continue meeting a stated goal as a named pressure grows or changes. Demand, data, contributors, and change provide four reusable pressures for examining the event tracker and its development system. Architecture and infrastructure are responses; product priorities and constraints determine which responses are acceptable.

Performance describes one set of conditions, scalability describes change across conditions, elasticity describes capacity adaptation, resilience describes behavior under failure, and efficiency relates useful outcomes to consumed resources or coordination. None substitutes for the others.

A useful scalability argument follows pressure to a binding limit, proposes a response, exposes the resulting trade-off, and names evidence that could disprove it. This turns “Can it scale?” from a slogan into a bounded engineering question.

CHAPTER 2

How can coding agents support scalable development?

As systems evolve, existing requirements and code constrain later change. Agentic coding can amplify that pressure by increasing the rate at which Large Language Models (LLMs), tools, and environmental feedback produce candidate implementations. The abstraction level has shifted, but the central engineering problem remains: requirements still have to become one of many possible concrete implementations that people can validate and maintain.

2.1 What is an agent?

The agent concept predates LLMs. In classical AI, an agent is situated in an environment: sensors turn stimuli into percepts, a controller chooses what to do, and actuators turn its commands into actions. A purposive agent chooses actions in relation to goals or preferences. This literature explicitly uses robots as a motivating example while treating computer programs as agents in information environments under the same model (Poole & Mackworth, 2023).

In this book, a **coding agent** is a goal-directed software system that repeatedly observes a development environment, chooses a next action, performs it through available tools, and uses the result to continue, change direction, stop, or escalate. An LLM by itself is not the whole agent. The model operates inside an **agent harness** that assembles observations, exposes tools, carries state between steps, invokes the model, and returns the effects of actions to the next step. A survey focused on software-engineering agents uses the closely related modules of perception, memory, and action, with the LLM as their cognitive core (Wang et al., 2025).

Within this system, the controller repeats an observe–decide–act loop as actions change the environment and create new observations. The table below maps the parts of that loop from a physical robot to a coding agent and shows which development choices tune each part.

Table 2.1. Agent anatomy mapped from a physical robot to a coding agent.

Agent concept	Robot	Coding agent	What to tune
Goal or preference	Reach a destination while avoiding hazards	Requested outcome, acceptance criteria, constraints, budget, and stopping conditions	Task specification, examples, graders, priorities, and escalation rules
Environment	A room, road, or factory floor	Repository, filesystem, terminal, browser, remote services, and collaborating people	Sandbox, workspace setup, available services, and environmental determinism
Sensors / perception	Camera, microphone, or proximity sensor	Prompt and instructions, file reads, search results, tool output, test failures, and images	Context selection, retrieval, observation format, provenance, and feedback quality
Controller / brain	Control software that selects motor commands	LLM plus the agent-harness loop that constructs context and interprets model outputs	Model and runtime configuration, instructions, planning policy, and orchestration
State / memory	A map, state estimate, and recent readings	Context window, session history, task state, retrieved documents, and durable project memory	Selection, summarization, retention, freshness, and source-of-truth rules
Actuators / actions	Wheels, arms, speaker, or display	File edits, shell commands, browser interaction, API calls, and messages	Tool design, permissions, approvals, validation, rollback, and rate limits

The analogy is a design model, not a claim that a coding agent has a body, desires, or a human-like mind. Its observations can be incomplete, its internal model can be wrong, its tool calls can fail, and its stated goal can be an inadequate proxy for the outcome people actually want. Developing an agent therefore means more than changing the prompt or swapping the LLM. You can improve what it can observe, how it decides and remembers, what actions it may take, how action effects return as feedback, and how its goal is translated into checks and stopping conditions. The constrain–generate–validate–revise loop later in this chapter is one practical way to tune these parts together.

2.2 How did development control change before and after agents?

Coding agents did not introduce specifications, executable tests, model-driven generation, or iterative feedback. The distinctive change is that a probabilistic, tool-using system can now perform much more of the implementation loop and produce candidate changes at a rate that may exceed human comprehension, review, and integration. Earlier engineering artifacts consequently reappear as machine-readable context, constraints, and feedback.

The table below uses representative milestones rather than claiming that a broad practice was invented in one year. The approaches overlap and can be combined; the chronology is not a progression from worse to better.

Table 2.2. Representative milestones in the movement from human implementation and deterministic generation toward agent-executed development workflows.

Representative milestone	Approach	Primary artifact	Implementation loop	Main control and trade-off
1970	Staged, plan- and design-led development	Requirements, architecture, interface, and test plans	Humans implement through planned stages with feedback between them	Documentation and staged verification create traceability; discoveries arriving late can invalidate expensive commitments
2001	Iterative and agile development	Working increments, a backlog, and stakeholder feedback	Humans build and review small slices repeatedly	Short feedback supports learning; architecture and cross-cutting quality require continuous stewardship
2001	Model-Driven Architecture	Platform-independent and platform-specific models	Deterministic generators transform models into implementation artifacts	A model can create repeatable leverage across platforms; escape paths and generator constraints can become expensive
Early 2000s	Test-Driven and Behavior-Driven Development	Executable examples and tests	Humans implement through short behavior and test feedback loops	Evidence moves close to implementation; checks cover only the properties and examples they encode
February 2025	Vibe coding	Conversational intent and the running result	An agent generates while a person observes and redirects visible behavior	Exploration becomes fast; traceability and control of hidden contracts, data, security, and maintenance remain weak
August–September 2025	Spec-Driven Development	Specification, plan, tasks, and implementation artifacts	Agents work through durable phases with review between them	Intent survives one session or agent; artifact maintenance and drift from runtime behavior become new work
February 2026	Harness engineering	Project harness: repository structure, instructions, tools, and tests	Agents repeatedly act inside encoded boundaries	Mechanized feedback supports repeated high-throughput work; building and maintaining the project-specific environment is costly

Representative milestone	Approach	Primary artifact	Implementation loop	Main control and trade-off
April 2026	Shift-Up	Executable BDD requirements, C4 models, and ADRs	Agents implement while people focus on requirements, architecture, and acceptance	Established design knowledge becomes a persistent guardrail; the current evaluation is exploratory and adds upfront work
April 2026	Structured Prompt-Driven Development	Version-controlled business, domain, design, task, and safeguard context	Agents generate within a reviewed prompt boundary	Prompts become governable and reusable team artifacts; keeping them aligned becomes development work

For the event tracker, every approach could produce the same visible event form and activity view. The difference is where intent and control live: a staged design can describe the complete request and data path, an iterative team can deliver it as a vertical slice, BDD can express its accepted behavior, a model can generate schemas or clients, and an agentic workflow can interpret repository context and act across all of those artifacts. A working browser view alone still does not establish authorization, HTTP compatibility, durable persistence, retry behavior, or safe evolution.

Shift-Up makes the historical reuse explicit by treating BDD requirements, C4 models, and ADRs as agent guardrails. Its initial evaluation compared unstructured vibe coding, structured prompting, and a partial Shift-Up workflow while building a full-stack snack-bar web application (Lipsanen et al., 2026). Treat that comparison as an exploratory example rather than a universal ranking of the approaches.

In this book, the same bounded project is first implemented with a local model, then a foundation model, and finally a fleet of agents. The phases do not prescribe one row of the table. They hold required web behavior stable while students compare how different models, constraints, validation, and orchestration affect the path from a candidate change to accepted work.

2.2.1 When generation outruns comprehension

Greater code-production capacity does not make software development scale by itself. The evidence below describes different parts of the resulting capacity mismatch rather than one universal productivity law.

Table 2.3. Evidence behind the distinction between candidate production and accepted change.

Evidence	What it supports	Important limit
Three field experiments covering 4,867 developers found about 26% more completed tasks with an AI coding assistant (Cui et al., 2026)	Candidate production can increase in some settings	Individual experiments were noisy, and the result does not apply equally to every person or project
A study of 78 developers across seven projects classified about 58% of observed time as program comprehension (Xia et al., 2018)	Understanding already consumed substantial development capacity before generative AI	Two companies using Java and C# do not establish a universal reading-to-writing ratio
A professional code-review study found that understanding supports knowledge transfer and alternative solutions as well as defect detection (Bacchelli & Bird, 2013)	Review is broader than checking whether generated code passes	The study predates current coding agents
A live LLM-review study at Mozilla and Ubisoft found that generated comments sometimes informed revisions, but reviewers accepted only a minority (Olewicki et al., 2026)	AI-assisted review can contribute evidence while still consuming human judgment	One deployment does not establish the effectiveness of every review agent

An agent can increase the arrival rate of candidate changes more readily than a team can increase its comprehension, review, and integration capacity. A generated diff is therefore inventory, not completed work. Useful throughput is **accepted change**: work that someone can explain, validate against the

intended behavior and architecture, integrate, and remain willing to own. When candidate arrival exceeds that capacity, the backlog appears as large diffs, delayed review, shallow acceptance, rework, or technical debt.

Types, tests, contracts, static analysis, small modules, architectural notes, and assisted search can direct attention and compress review. They help only when they reduce uncertainty rather than create more assertions to inspect. Effort and transcript length remain poor correctness proxies: a quick change can be sound, a laborious change can be wrong, and neither proves that the application meets its requirements.

Brooks (1987) distinguishes difficulties tools can reduce from essential difficulties rooted in the software problem, while Lehman (1980) observes that evolving programs become more complex unless deliberate work maintains or simplifies them. The case studies show how faster generation interacts with product understanding, validation, and maintained project guidance in concrete projects.

2.3 Sustainable throughput requires ownership

Prompting can produce visible results quickly, especially when a task is easy to specify. Sustaining change requires enough context, explicit variation boundaries, relevant acceptance evidence, and an owner who can correct the result. Guardrails and quality gates help, but they consume time and can be optimized mechanically if their score becomes the goal; the Benchmarking chapter develops that problem.

Agentic development therefore includes the supporting development system as well as the application. Project instructions, architecture notes, tests, static analysis, reproducible commands, and reviewable examples can make future changes cheaper and safer. Their value depends on whether they reduce uncertainty around the work that matters, not on how many checks or documents the repository contains.

The ownership standard should follow the work's consequences and expected lifetime rather than whether a human or model typed it. A disposable experiment may need little maintenance, while shared code makes other people inherit the cost of understanding, operating, and correcting it—what Cory Doctorow calls accretive work. Publishing an issue or patch also transfers cost to an external maintainer, so the Agent Fleets chapter treats publication as an externally consequential action that needs a human owner.

> **Sustainability angle**
>
> Sustainable agentic development has two meanings here. It means keeping the project maintainable for humans, but it also means avoiding wasteful tool use. Focused prompts, smaller context, reusable instructions, and strong validation loops reduce repeated generation and unnecessary compute.

2.4 Human-agent relationship is a control design

The phrase *Human Agent Relationship* (HAR) can suggest an interpersonal question: should the agent be treated as a tool, assistant, colleague, or team member? Those metaphors may shape expectations, but they do not define how work is controlled. In this book, HAR is a practical name for the allocation of **intent, authority, execution, evidence, intervention, and accountability** across a human-agent development system.

An agent does not need one global autonomy level. For a particular decision, it may **propose** an option, **prepare** an action for approval, or **execute** within explicit limits. Broader authority is easier to justify when the work is bounded, observable, independently checkable, reversible, and low in blast radius. Ambiguous intent, weak evidence, irreversible commitments, and material external effects should narrow that authority.

Invoking a tool does not transfer accountability to it. The accepting person or group still needs enough understanding, evidence, and control to stand behind the result and correct it. The Agent Fleets chapter develops the research background, decision path, and authority-mapping exercise.

Perceived effort can influence quality judgments when quality is difficult to assess (Kruger et al., 2004). It is a heuristic, not correctness evidence. The table below separates four questions that should not be collapsed into one quality judgment.

Table 2.4. Questions that separate an artifact's origin, endorsement, accountability, and evidence.

Concept	Question it answers	Record to preserve	Does not prove
Provenance	Which exact revision or artifact is this, and which inputs, people, and tools produced or transformed it?	Artifact identity, revision, material inputs, actors, and tools	That the artifact is correct
Endorsement	Who stands behind which claim or decision on the final revision, including any reservations?	Named endorser, endorsed claim, scope, revision, and reservations	That every property was checked
Accountability	Who owns the consequences and must correct, maintain, or retire the result if it becomes wrong or outdated?	Accountable owner and correction or retirement obligation	That the owner produced the artifact
Evidence	Which properties were checked on that revision, under what conditions, and what remains unverified?	Check, environment, result, revision, limits, and omissions	Universal quality outside those checks

Sloptimists, Sloperators and Slopportunism makes the distinction concrete: Atharva Raykar describes the essay body as unedited model output, then says in his commentary that he does not fully endorse it. Disclosed origin and human acceptance are different, and neither establishes quality. Survey respondents have reported tracking and monitoring for later review or debugging as reasons for declaring AI-generated code, but the available study did not test whether declaration improves software quality (Kashif et al., 2026).

Use the AI-use log together with the submitted project evidence: record material assistance and corrections in the log, and bind checks and known limits to the submitted revision. Provenance can make a process traceable; it does not make the artifact correct or transfer responsibility to the tool. Record effort for comparison and reflection, not as a quality score.

The loop below is the local form of this relationship. The human and agent may occupy different positions in each phase, but the workflow still needs a visible owner, admissible actions, acceptance evidence, and a route for correction.

2.5 Constrain, generate, validate, revise

Agentic development follows a repeated loop: constrain what may happen, generate a candidate, validate it against the intended outcome, and revise both the result and the project's durable knowledge. The loop's useful throughput depends on the evidence required to advance each stage, not on generation speed alone.

Constraints at the start of a task are hypotheses, not necessarily complete knowledge. Brooks (1987) argues that rapid prototyping supports the iterative specification of requirements because important questions become visible only when people interact with a concrete system. Naur (1985) similarly describes programming as theory building rather than program-text production. Implementation can therefore be a way to learn what the specification should say, not only a way to execute it.

The table below summarizes the question, work, and exit evidence at each stage before the sections that follow examine them in detail.

Table 2.5. The constrain–generate–validate–revise loop and the evidence that advances each stage.

Stage	Governing question	Main action	Exit evidence
Constrain	What outcome is wanted, and what may vary?	State the goal, context, boundaries, budget, and stopping or escalation rule	A bounded task and explicit acceptance criteria
Generate	Which candidate might satisfy those limits?	Let a chosen model and agent harness act with relevant context and tools	A candidate artifact plus its trace, cost, and attempts
Validate	Does the candidate satisfy the real need?	Apply independent checks and inspect the artifact a user will consume	Acceptance evidence and known unverified properties
Revise	What should change before the next attempt?	Correct the result and codify learning in durable project material	An improved artifact and better starting constraints

> **Delegate work without surrendering judgment**
>
> Cognitive offloading—using external action or aids to reduce a task's cognitive demand—is one reason tools can help people scale their work (Risko & Gilbert, 2016). Shaw & Nave (2026) use *cognitive surrender* for a narrower proposed failure: adopting AI output with little scrutiny so its judgment replaces rather than supports the person's own. Their experiments used short reasoning tasks, so treat the term as a workflow diagnostic rather than settled cognitive architecture or evidence of professional programming skill loss. Generation can be delegated, but constraints, validation, and revision should not all collapse into the same model's unsupported judgment. Before accepting a consequential change, the team should be able to explain the intended behavior, name a plausible failure, connect acceptance to independent evidence, and correct or reject the result.

Constraining

Constrain both the task and the environment in which it runs. State the outcome, what may vary, relevant architecture and technology boundaries, budget, forbidden actions, acceptance criteria, and escalation rule. Durable project instructions such as `AGENTS.md` can preserve constraints that should apply beyond one prompt.

Generation

Model choice should follow the work and local evidence. A more capable model may avoid retries on uncertain or coupled tasks, while a smaller model may reliably satisfy strong checks on bounded work. Treat labels such as "planning" and "implementation" as routing hypotheses rather than rules. A model name is not a capability result: runtime configuration, context, tools, hardware fit, routing, and fallback can all change the outcome and its cost or data boundary. Appendix A: Evaluating agentic development workflows shows how to identify and compare the configuration that actually served the work.

Context selection is another generation decision. Focused retrieval and compact tool output can reduce irrelevant material, but any lossy view can also hide the fact needed to complete or review the task. Keep important raw evidence retrievable and test context-reduction policies instead of treating fewer tokens as an outcome by itself.

Validation

Validation decides whether a candidate advances or returns for correction. Agents can generate and run checks, but those checks are candidate artifacts too: acceptance depends on their demonstrated relationship to the required behavior.

Validation is stronger when it compares a candidate with an independent reference or inspects the artifact a user actually consumes. The Webernetes case study develops the first pattern through tests shared with a real k3s cluster; the Slideotter case study develops the second through geometry, browser, and rendered-output checks.

> **A generated test is still generated output**
>
> In a contrasting workshop report, Talwar (2026) describes an agent-generated application whose screenshot tests did not exercise the application, whose claimed passkey authentication accepted a username without authentication, and whose data disappeared after restart. A useful conclusion is that specifications and generated tests do not constrain a system merely because they exist. A screenshot check should launch the real application, authentication should be tested against observable security behavior, and persistence should survive a restart. Tests produced by the same agent remain candidate artifacts until their relationship to the required behavior has been established.

A reversible mutation can strengthen this relationship by checking that a test distinguishes the accepted implementation from one named fault. That contrast does not prove the mutant is realistic or the broader requirement is satisfied, but it is stronger than coverage alone. The concern-directed mutation lab lets you apply the pattern to authorization, retry, or archived-state behavior; study its source or download the standalone lab.

Revision

Revision corrects the candidate and improves the next starting point. Codify durable learning in requirements, tests, architectural decisions, or project instructions; otherwise the implementation and its supporting material drift apart.

2.6 Agents learn patterns from your codebase

The codebase itself is also part of the context an agent learns from during a task. Robinson (2026) describes allowing several duplicated authorization checks because an agent could maintain them later. Instead, those accepted shortcuts became examples for the agent to copy into the next change. A shortcut therefore has two costs: humans may have to repair it later, and agents may treat it as evidence of the project's preferred style. Validation should ask not only whether the current change works, but also whether it leaves behind a pattern worth repeating.

2.6.1 Constraint feedback loop

The demonstration below starts with an underspecified request to add CSV export to an event list. Generate a candidate, validate it against the constraints that were stated, revise the project memory from failed checks, and then begin the next round. Try completing the loop once with the default constraints and once after selecting every constraint before generation.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/scalable-development/

The generated candidate is deterministic teaching data rather than model output. The useful part is the shape of the loop: validation produces evidence, revision makes the learning explicit, and the revised memory changes the next generation boundary instead of becoming a vague instruction to "try again."

2.7 Bound probabilistic generation with deterministic controls

Agentic development does not make the generator deterministic. The practical goal is to keep variation inside explicit boundaries so behavior remains predictable where correctness depends on it. Two runs may interpret the same task differently, but they do not need unrestricted authority to invent business rules, state transitions, permissions, or completion criteria.

Developers can decide what may vary and encode what must remain stable. In his talk, David Khourshid (Beyond the Prompt: "Goodbye slop; welcome determinism") argues for explicit models, state machines, and deterministic core logic around generative components. This gives validation a clearer target: let the model handle work that benefits from interpretation, while expressing rules and invariants in forms that tools can check repeatedly. The resulting controls can still encode the wrong rule or omit an important condition; determinism makes a boundary repeatable, not automatically correct or complete.

Khourshid describes one way to combine these ideas as letting an LLM navigate a deterministic state machine: given a goal and the current state, the model chooses from the events the machine currently permits. The demonstration below applies that pattern to an agent delivering a code change. At each state, choose among only the events that the machine currently enables. The statechart expresses hierarchy, parallel verification, recovery, and completion while keeping the model away from events that are not currently legal.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/scalable-development/

Notice what the statechart provides beyond a linear checklist. One cancellation rule applies throughout the compound active state, tests and human review can progress independently, and either kind of failure can return the workflow to implementation. The model may choose a route, but the statechart defines the available routes. It does not prove that the generated code satisfies the web application's requirements: the tests, review criteria, and observed behavior still have to provide that evidence.

Deterministic boundaries also depend on how the application represents valid data and commits state. The Architecture chapter develops that connection through constructive data modeling, runtime parsing, authorization, and atomic persistence.

2.8 The project harness

When using agents, the application code is only one part of the project. The other part is the **project harness** around the code: the versioned files, commands, rules, and examples that help humans and agents understand what is expected. This is related to, but distinct from, the agent harness introduced earlier.

Table 2.6. Three related control layers in agentic development.

Term	Scope	What it includes	Governing question
Agent harness	One model-and-tool execution loop	Context assembly, tools, state, permissions, model invocation, and action feedback	How can the model observe, decide, and act?
Project harness	One repository or maintained codebase	Instructions, specifications, architecture notes, environments, commands, tests, rules, and examples	How can a contributor change this project without hidden context?
Development system	The process that changes an application	People, agents, agent harnesses, project harnesses, review, integration, ownership, and organizational policy	How does candidate work become accepted and maintainable change?

An agent harness can consume a project's instructions and invoke its commands, but those artifacts remain part of the project harness rather than the runtime itself. A development system may use several agent harnesses against the same project harness and still rely on people for review, integration, and ownership. A good project harness reduces the hidden context needed to make a useful change and returns failures in forms that people and agents can act on.

An analysis of more than 3,800 reported bugs across three coding tools found that API, integration, or configuration errors accounted for 36.9% of root causes, while tool invocation and command execution were the most affected workflow stages (Zhang et al., 2026). The sample is limited to public issue reports, but the result is a useful warning: model reasoning is only one possible failure source. Strict tool contracts, useful errors, reproducible commands, and observable execution belong to the project harness too.

vibe-template provides one concrete example: it separates a starter application from agent instructions, architecture rules, feature specifications, decision records, setup notes, and validation commands. The reusable lesson is to push repeatable project knowledge and feedback into maintained artifacts rather

than leave them in one conversation.

The common project specification owns the exact required artifacts. The practical baseline is simpler than the list: can another student or agent clone the project, understand its intended shape, run it and its checks, and make a small change without guessing? An environment definition is part of that contract rather than neutral background; the Friction chapter develops how to test it from a fresh checkout. That chapter also explains why validation cost should follow change risk instead of running every check after every edit.

2.8.1 Reuse practices without cloning the whole project harness

Copying a complete starter repository into an existing project is usually too coarse. Package a reusable capability with its fit criteria, files, dependencies, integration instructions, checks, provenance, and update path so a project can adopt the practice without inheriting every source-template decision.

Modern Web Guidance shows how a maintained external capability can package fast-changing domain knowledge with targeted retrieval, compatibility constraints, an update path, and task-level evaluation. Imported capabilities also create a trust boundary because instructions or bundled code may influence tools operating with the agent's authority. The Agent Fleets chapter develops the security implications, while the skill-adoption lab provides a bounded review exercise; study its source or download the standalone lab.

2.9 Project artifact

For implementation A, create the first version of the project harness alongside the web application. Keep it small enough to maintain, but concrete enough that another group can run the project, inspect the main event workflow, and understand how agent-produced work becomes accepted work. Include:

- a README with the supported setup and run path,
- project instructions for the coding agent,
- one architecture note and one feature specification for the core event workflow,
- one decision record for a consequential stack or storage choice,
- one fast validation command,
- and a short AI-use record that binds a representative task to its relevant configuration, checks, human correction, acceptance decision, and known limits.

The complete requirements for the project harness and evidence package remain defined by the project specification. This chapter's contribution is to make intent, constraints, feedback, and ownership explicit before generated changes accumulate.

2.10 Summary

Coding agents have changed web development by increasing the rate at which a specification can become candidate code. A generated change is not completed work: useful throughput is accepted behavior that people can explain, validate, integrate, correct, and remain willing to own. When

generation outruns those capacities, speed reappears as review backlog, shallow acceptance, rework, or technical debt.

Specifications, architecture notes, project instructions, examples, tests, and validation commands therefore become control surfaces for both people and agents. They constrain what may vary, return evidence to the next attempt, and help future work recover the intent behind the codebase. Deterministic checks can bound probabilistic generation, but they establish only the properties and conditions they actually examine.

Human–agent control should be allocated per decision rather than expressed as one global autonomy level. An agent may propose, prepare, or execute within explicit limits, while provenance, endorsement, accountability, and evidence remain separate questions. For a web application, acceptance must reach beyond a plausible browser view to the relevant HTTP contracts, authorization rules, persistence behavior, retry semantics, and maintenance obligations. Whatever produced the change, maintained work still needs an accountable human owner.

2.11 References

1. Lipsanen, P., Rannikko, L., Christophe, F., Kalliokoski, K., Stirbu, V., & Mikkonen, T. (2026). Shift-Up: A Framework for Software Engineering Guardrails in AI-native Software Development – Initial Findings. arXiv Preprint arXiv:2604.20436. https://doi.org/10.48550/arXiv.2604.20436
2. Zhang, R., Dai, W., Pham, H. V., Uddin, G., Yang, J., & Wang, S. (2026). Engineering Pitfalls in AI Coding Tools: An Empirical Study of Bugs in Claude Code, Codex, and Gemini CLI. arXiv Preprint arXiv:2603.20847. https://doi.org/10.48550/arXiv.2603.20847
3. Cui, K. Z., Demirer, M., Jaffe, S., Musolff, L., Peng, S., & Salz, T. (2026). The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers. Management Science. https://doi.org/10.1287/mnsc.2025.00535
4. Xia, X., Bao, L., Lo, D., Xing, Z., Hassan, A. E., & Li, S. (2018). Measuring Program Comprehension: A Large-Scale Field Study with Professionals. IEEE Transactions on Software Engineering, 44(10), 951–976. https://doi.org/10.1109/TSE.2017.2734091
5. Bacchelli, A., & Bird, C. (2013). Expectations, Outcomes, and Challenges of Modern Code Review. Proceedings of the 35th International Conference on Software Engineering, 712–721. https://doi.org/10.1109/ICSE.2013.6606617
6. Olewicki, D., Da Silva, L. M. P., Ben Sghaier, O., Mujahid, S., Amini, A., Mah, B., Castelluccio, M., Habchi, S., Khomh, F., & Adams, B. (2026). Impact of an LLM-Based Review Assistant in Practice: A Mixed Open-/Closed-Source Case Study. IEEE Transactions on Software Engineering, 1–12. https://doi.org/10.1109/TSE.2026.3663093
7. Shaw, S. D., & Nave, G. (2026). Thinking—Fast, Slow, and Artificial: How AI Is Reshaping Human Reasoning and the Rise of Cognitive Surrender. PsyArXiv, 1–58. https://doi.org/10.31234/osf.io/yk25n_v1
8. Risko, E. F., & Gilbert, S. J. (2016). Cognitive Offloading. Trends in Cognitive Sciences, 20(9), 676–688. https://doi.org/10.1016/j.tics.2016.07.002

9. Talwar, S. (2026, July). LLMs Can't Program. functional.computer. https://functional.computer/blog/llms-cant-program
10. Brooks, Jr., Frederick P. (1987). No Silver Bullet—Essence and Accidents of Software Engineering. Computer, 20(4), 10–19. https://doi.org/10.1109/MC.1987.1663532
11. Lehman, M. M. (1980). Programs, Life Cycles, and Laws of Software Evolution. Proceedings of the IEEE, 68(9), 1060–1076. https://doi.org/10.1109/PROC.1980.11805
12. Naur, P. (1985). Programming as Theory Building. Microprocessing and Microprogramming, 15(5), 253–261. https://doi.org/10.1016/0165-6074(85)90032-8
13. Robinson, S. (2026, July). Write Code Like a Human Will Maintain It. Unstack. https://unstack.io/write-code-like-a-human-will-maintain-it
14. Kruger, J., Wirtz, D., Van Boven, L., & Altermatt, T. W. (2004). The Effort Heuristic. Journal of Experimental Social Psychology, 40(1), 91–98. https://doi.org/10.1016/S0022-1031(03)00065-9
15. Kashif, S. M., Liang, P., & Tahir, A. (2026). On Developers' Self-Declaration of AI-Generated Code: An Analysis of Practices. ACM Transactions on Software Engineering and Methodology, 35(7), 1–37. https://doi.org/10.1145/3771937
16. Poole, D. L., & Mackworth, A. K. (2023). Artificial Intelligence: Foundations of Computational Agents (3rd ed.). Cambridge University Press. https://doi.org/10.1017/9781009258227
17. Wang, Y., Zhong, W., Huang, Y., Shi, E., Yang, M., Chen, J., Li, H., Ma, Y., Wang, Q., & Zheng, Z. (2025). Agents in Software Engineering: Survey, Landscape, and Vision. Automated Software Engineering, 32(2), 70. https://doi.org/10.1007/s10515-025-00544-2

CHAPTER 3

How can we measure web applications and development workflows?

You cannot evaluate a property of a web application without evidence about it. A page that feels fast on one developer's machine does not establish how it behaves for other devices, workloads, data volumes, or network conditions. Benchmarking turns a question about that behavior into a controlled comparison.

Measurement is necessary, but it is not self-explanatory. A metric can represent the wrong outcome, conceal variation, or become misleading when people and agents optimize for it. Even a trustworthy result can show that behavior changed without explaining why. This chapter therefore moves from benchmarking, through the limits of measurement, to observability and finally to evidence about the agentic workflow that produced the application.

3.1 Benchmark to test a claim

A **measurement** assigns evidence to an observed property. A **benchmark** uses declared conditions to compare that property across workloads, systems, or revisions. Begin with the decision the comparison should support: for example, whether the event tracker still records events within an agreed latency objective as concurrent demand increases.

Define the protocol before selecting a runner:

- **Decision and claim:** What comparison or threshold would change an engineering decision?
- **Workload:** Which event-recording, querying, filtering, or browser actions are performed, and in what mix?
- **Load:** How many users, requests, events, or concurrent sessions apply the workload, and for how long?
- **Boundary and conditions:** Which application revision, environment, data volume, cache state, dependencies, and client conditions are included?
- **Metrics and distribution:** Which latency, throughput, error, resource, or queue measurements answer the question, and which percentiles or ranges preserve important variation?
- **Acceptance and limits:** What result would support or challenge the claim, and what does the setup leave untested?

Tools such as Artillery and k6 can generate load, schedule scenarios, and collect observations from that protocol. They cannot decide whether the workload represents real use, whether the boundary matches the question, or whether the comparison supports a statistically defensible conclusion. An agent can help draft a script, but the generated workload, metrics, thresholds, and interpretation still have to be reviewed against the intended decision.

3.1.1 Simulated benchmark lab

The demonstration below models 60 requests to an event-recording endpoint. Change the number of concurrent clients, the share of requests that take a slow storage path, and the p95 latency objective. The values are deterministic and run only in the browser, so this is a teaching model rather than a capacity test.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/benchmarking/

Predict the default result before running the simulated workload, then compare the average with p95. Treat the first result as a baseline, change one condition, and predict again before recording the comparison. The useful lesson is not the simulated number itself: it is that the workload, load, metric, and acceptance threshold have to be stated together.

Synthetic benchmarks deliberately simplify reality. They can omit workload mixes, client behavior, cache histories, data shapes, provider quotas, dependency contention, and failure combinations that appear in production. A result therefore supports a claim only for its declared conditions. Compare synthetic evidence with field telemetry where available, and revise the protocol when real behavior exposes a missing condition.

3.1.2 Benchmark browser journeys with Playwright and Lighthouse

Server latency does not describe the whole user experience. A browser still has to download resources, parse and execute JavaScript, calculate styles, lay out the page, paint it, and respond to input. Three related tools expose different parts of that work:

- The browser's Performance APIs can measure marks, durations, resources, and selected rendering events in the page itself.
- Playwright drives a real browser through a repeatable user journey and checks whether the expected behavior occurred.
- Lighthouse audits a page in a controlled lab setup and reports performance metrics and diagnostics.

These tools are complementary rather than interchangeable. A Playwright test can prove that filtering the event list produces the expected rows, but its elapsed test time is not a Web Vital. A Lighthouse navigation can diagnose initial page load, but it does not prove that the application's business flow works. Lighthouse user flows can audit interactions, although they require browser automation outside the page; a normal web page cannot grant itself the required browser-control privileges.

Browser performance workbench

The workbench below runs completely in the current page. It builds and filters an event list either as one uninterrupted task or in batches that yield to the browser. It measures from the start of the work until the next painted frame, repeats the same flow five times, and reports both total time and the longest JavaScript slice observed by its own instrumentation.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/benchmarking/

Before each measurement, predict which strategy will finish sooner and which will leave the main thread available sooner. Run both strategies with the same number of events. Then increase the workload and check whether the conclusion remains the same.

This is a real measurement, but it is deliberately narrow. The timer includes work performed by this demo on this browser and device; it is not a Lighthouse score, INP measurement, or claim about other users. The chunked strategy may take longer overall because it waits between batches while still improving responsiveness by avoiding one long uninterrupted task. This distinction between throughput and responsiveness is often more useful than asking which version is simply "faster."

For client-side collection in a real application, the small web-vitals library implements LCP, INP, CLS, FCP, and TTFB using browser APIs and follows the same measurement rules used by Google's web-performance tools. Those measurements describe actual page visits. Lighthouse instead provides synthetic lab evidence under declared conditions.

From page metrics to product metrics

Standard browser metrics deliberately use definitions that work across many kinds of pages. That makes them comparable, but the element selected for a metric such as LCP is not necessarily the part of the interface that a user came to see. An event tracker might care when its event list becomes visible; another application might care about a search-results region, product summary, or account overview. The measurement boundary should follow the question rather than whichever element happens to be largest.

Element Timing can report when one annotated image or text element rendered. The experimental Container Timing API extends that idea to an author-defined DOM region and reports a sequence as new contentful areas inside it are painted. That sequence can reveal whether a component appeared together or accumulated through several delayed stages, moving the measurement boundary from a browser-selected element toward a user-recognizable product component.

Container Timing does not declare that a component is complete, interactive, accurate, or ready to use. Those are product claims that may require application instrumentation or interaction measurements in addition to paint evidence. Because the API is experimental, its immediate value here is the design principle rather than the exact interface: choose a meaningful region, give the signal a stable name, state how an entry is selected from the sequence, sample collection deliberately, and record which browsers and users the dataset represents.

For the course project, choose one user-recognizable region and define what useful evidence would mean for it. Then decide whether a page-level Web Vital, one Element Timing entry, a sequence of container paints, or an application milestone best supports that claim, and explain what the chosen signal cannot prove.

Run a page audit on your own machine

Use PageSpeed Insights for the lowest-setup version of the exercise. It runs Lighthouse remotely, so the target page must be deployed at a publicly reachable URL; a development server at `localhost` is not reachable by the service.

1. Choose one representative public route and state why it matters to a user.
2. Select the mobile profile and run the same URL five times without changing the application.
3. Record LCP, TBT, CLS, transferred bytes, and the overall performance score for every run.
4. Report the median and the fastest-to-slowest range for each time-based metric.
5. Select one Lighthouse diagnostic, make one change, deploy it, and repeat the five runs under the same conditions.
6. State the narrow conclusion supported by the comparison and at least one factor it does not control.

The PageSpeed Insights result can include Lighthouse lab data and, for sufficiently popular public pages, field data derived from real Chrome visits. Keep those sources separate: lab and field measurements use different populations and conditions.

If the application is not public, open it in Chrome, select **Developer Tools → Lighthouse**, choose the mobile profile, and generate a report locally. Close unrelated applications where practical, do not run audits concurrently, and keep the browser version, profile, URL, application version, and machine consistent.

Performance values vary even when the application has not changed because the browser, client hardware, resource contention, network, and server introduce noise. Heričko et al. (2021) found that five consecutive Lighthouse runs aggregated with the median greatly reduced variability in their experiment.

Study a repeatable Playwright journey

PageSpeed Insights is convenient for navigation audits, but it cannot exercise a private or multi-step project flow. The standalone Playwright lab separates a small event-filter fixture from the browser journey that exercises it, so you can inspect locators, interaction, assertions, and trace collection without first adapting the example to a framework. Study the source or download the standalone lab.

The example establishes that filtering produces the expected visible row. It does not measure application loading, network behavior, or Web Vitals because it opens a local fixture. After it passes, replace the fixture URL with the project's real route and adapt the accessible names while preserving the same user-visible contract. Inspect the trace to verify what was acted on and rendered, then use Lighthouse or another suitable performance tool separately; the flow can be correct but slow, or appear fast because it failed to perform the intended work.

Teams that need a repeatable CI gate can use the official `lighthouse` Node module or Lighthouse CI to collect several runs and assert a budget. Start with reporting rather than a strict pass/fail threshold, learn the normal variation of the runner, and only then set a budget with enough tolerance to detect a meaningful regression.

3.1.3 Check measurements against a model

Measurements become more useful when they can be checked against a model. Little's Law states that, for a stable system over a suitable observation period, average work in progress (`L`) equals the average arrival rate (`λ`) multiplied by average time in the system (`W`) (Little, 1961):

`L = λW`

For an event pipeline, `L` can be the average number of events waiting or being processed, `λ` the average arrival rate, and `W` the average time from arrival to completion. If 20 events arrive per second and each spends half a second in the system, the expected average work in progress is 10 events.

Queue balance lab

Use the lab below to check whether three observed averages are mutually consistent. Change one value at a time and explain whether the mismatch suggests overload, incompatible measurement windows, or a missing boundary.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/benchmarking/

The law does not identify the cause of latency, prove that a queue is healthy, or promise that the system can sustain any chosen arrival rate. Before applying it, state the system boundary and check that arrivals and completions were approximately balanced during the observation window. A large mismatch is a reason to inspect the measurement setup or system stability, not permission to force the numbers to fit.

3.2 Measurements can lead you astray

Goodhart's law states: "Any observed statistical regularity will tend to collapse once pressure is placed upon it for control purposes" (Goodhart, 1975). In plain language, the relationship that made a metric informative can break down when that metric is used as a target. The familiar shorthand is that a measure tends to stop being a good measure when it becomes the target. This is especially relevant to software because dashboards and quality gates make measurable proxies easy to optimize.

Consider a few examples:

- Test coverage can reveal unexercised code, but a coverage target can reward assertions that execute lines without checking meaningful behavior.
- A Lighthouse score can direct attention to performance problems, but optimizing only the score can remove valuable functionality or tune a page for the audit rather than its users.
- Mean latency can improve while tail latency or error rate becomes worse.
- Agent task pass rate can rise because tasks, judges, or outputs become tailored to the evaluation set rather than because the workflow handles new work better.
- Commit count, generated lines, and closed tickets can all increase while review and maintenance costs grow elsewhere.

Goodhart's law does not imply that targets or metrics are useless. It means that the measurement system changes once people or agents can adapt to it. A quality gate should therefore be treated as a partial model of the desired outcome, not as the outcome itself.

For an important target, record four things:

1. **Outcome:** What user or operational result is the metric intended to represent?
2. **Proxy:** Why should the chosen measure move with that outcome under the current conditions?
3. **Gaming path:** How could the number improve without the outcome improving?
4. **Counterevidence:** Which second signal, failure review, or qualitative check would expose that divergence?

Keep raw distributions and failed examples available instead of retaining only an aggregate score. Review the proxy when the workload, system, incentives, or development process changes. An effective metric is not permanently valid; it is a hypothesis that must survive contact with the behavior it influences.

3.2.1 Goodhart pressure test

The challenge below turns that four-part review into two decisions. First choose a proxy and predict how an optimizer will respond when the number becomes a target. Then inspect the displaced cost and revise the gate so that improving the proxy cannot compensate for losing the intended outcome. The scenarios use deterministic illustrative values to make the feedback loop visible; they are not empirical comparisons between tools or teams.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/benchmarking/

Try at least two targets. Notice that monitoring the proxy is still useful after the repair, but it no longer decides success alone.

3.2.2 Resource proxies are not the outcome

Energy use is harder to measure than latency or transferred bytes, and neither of those proxies is an automatic substitute for it (Kalliola & Vepsäläinen, 2025).

Barrocas et al. (2026) provide one bounded example of a before-and-after energy study. On a single M4 MacBook and Chrome testbed, the authors report a mean 13.4% reduction in local CPU and GPU energy, recorded with Powermetrics after EcoAssist optimized 500 deliberately resource-heavy standalone webpages. The pages were selected for a size above 600 KB and at least ten DOM-affecting operations, so the sample does not represent arbitrary production applications. The benchmark compared each optimized page with its own unoptimized baseline, not with another optimizer.

In a separate single-condition lab study, twenty early-career developers generated and optimized portfolio pages and reported low workload. The study did not compare productivity against a control, and five participants observed removed content or layout changes after optimization. The reported before-and-after page measurements do not include the optimizing model's full training and inference

footprint; the paper estimates that overhead separately, and neither result is a whole-system carbon estimate. Treat the optimization as a candidate change that must preserve the intended product behavior, not as evidence that a lower energy number is automatically better.

Reuse one representative Playwright journey and the repeated-measurement discipline above. Preserve the baseline, make one targeted AI-assisted frontend optimization, and run the same journey under the same declared conditions. Repeat it at least five times before and after when the chosen signal varies from run to run. Record direct energy if the platform exposes a defensible measure; otherwise label transferred bytes, JavaScript or asset size, CPU time, or long-task time as **resource proxies**, not energy or carbon. For deterministic measures such as built asset size, record one before-and-after value; for run-varying measures, report the median and range. Re-run the functional checks, inspect for visual or accessibility regressions, note any rejected suggestion or regression, and state what the comparison cannot establish.

3.3 Measurement alone cannot explain behavior

A benchmark can reveal that the request distribution changed, but it does not necessarily explain why one request was slow. Telemetry provides evidence about a running system through signals such as traces, metrics, and logs. Observability is the broader capability to use that evidence to investigate system behavior, including failures you did not predict in advance.

OpenTelemetry is a vendor-neutral framework for generating, processing, and exporting telemetry. It supplies APIs, SDKs, semantic conventions, and a Collector, but it is not itself a storage or visualization backend. That distinction matters: instrumenting an application creates evidence, while operating the pipeline and asking useful questions turns that evidence into an observability practice.

3.3.1 Simulated OpenTelemetry trace lab

The trace explorer below reuses the event-recording request from the benchmark demonstration. Switch between a healthy write, a slow write, and a successful request that required a retry. Select individual spans to inspect their duration, status, source, and attributes. A span represents one timed unit of work and carries its own span ID, while every span in the same trace shares a trace ID. The spans are not necessarily sent as separate network messages: an SDK can batch completed span records into one OTLP export request before a Collector processes and routes them.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/benchmarking/

The slow trace localizes most of the request time to connection waiting and event storage. That is a stronger lead than the end-to-end latency alone, but it still does not establish root cause. The next investigation might inspect database saturation, pool configuration, query behavior, or competing workloads. Also notice that useful domain spans and attributes require conscious instrumentation, while generic HTTP spans can often be produced automatically.

3.4 Observe the agentic workflow too

Agentic development introduces a second system to observe: the development workflow that produces and changes the web application. The parallel is useful as long as outcome evidence remains separate from diagnostic traces. An HTTP trace can explain how a request traveled through the application but does not prove that the user received the correct result; an agent transcript can explain how a change was produced but does not prove that the accepted implementation is correct.

Table 3.1. Parallel evidence roles in a running web application and its agentic development workflow.

Evidence role	Web application	Agentic development workflow
Outcome to protect	Useful user and operational behavior	Accepted implementation and maintained project goal
Controlled comparison	Benchmark run under a declared workload	Bounded task or evaluation trial
Diagnostic evidence	Logs, metrics, request traces, and profiles	Prompts, tool calls, diffs, checks, and retries
Configuration	Revision, deployment, dependencies, and resource limits	Model, agent harness, tools, starting revision, and limits
Intervention	Operator diagnosis, mitigation, and recovery	Human review, correction, approval, and escalation

For a bounded task, record whether the resulting artifact passed its acceptance checks, how long the work took, what it cost, how many retries occurred, and how much human review or correction it required. Generated lines of code, transcript length, and nominal hours saved are weak outcome measures because they can move work into review, rework, or maintenance. Organizations report broad AI adoption and perceived productivity gains, while objective measurement of productivity and quality remains limited (Giray et al., 2025).

Keep the **task**, one attempted **trial**, its actual **outcome**, and the **trace** that led there distinct. Grade the outcome when several valid paths can solve the task; use the trace to diagnose failures, account for intervention and cost, and check process requirements such as permission compliance. A plausible transcript does not prove that the claimed change occurred, while one repository test is usually only one check on the result.

Bind each result to the configuration that produced it: the model and runtime where exposed, the agent harness and tools, the starting revision, relevant resource limits, and any routing or fallback. Then preserve the same reporting categories across implementations A, B, and C: accepted outcome, checks, elapsed time, retries, cost or resource use, human correction, and known limits. Treat the comparison as evidence about those tasks and conditions rather than a universal model or workflow ranking.

For the Establish project, choose one representative AI-assisted task and preserve its task brief, relevant context, configuration, outcome, checks, trace, corrections, and acceptance decision. The later implementations can reuse the reporting frame without pretending that one task is a statistically stable benchmark.

For the complete evaluation vocabulary, grader design, configuration qualification, repeated-trial methods, uncertainty, and routing experiments, continue with Appendix A: Evaluating agentic development workflows.

3.5 Project artifact

Complete the measurement and observation baseline required by the Establish project specification. Preserve it with implementation A so the same reporting categories can be applied to implementations B and C. Include:

- one representative runtime workload, load level, system boundary, and predicted result,
- the runtime and browser-visible metrics collected, their acceptance thresholds, and the observed distribution where the values vary,
- one telemetry signal or trace that helps explain the result,
- one bounded AI-assisted task with its configuration, outcome, checks, trace, retries, human correction, and acceptance decision,
- the application revision and relevant environment conditions,
- and one inspected failure, the limits of the evidence, and the first signal that would warn the group that behavior is moving outside the measured conditions.

This is a local baseline for a declared application and development setup, not a universal ranking of models, tools, or architectures.

3.6 Summary

Benchmarking turns a question about web behavior into a disciplined comparison rather than a collection of isolated numbers. A useful benchmark begins with a decision or prediction, defines a representative workload and system boundary, controls relevant conditions, repeats measurements that vary, and preserves the baseline. Formal relationships such as Little's Law can check whether observations are mutually consistent, but they do not replace that declared boundary.

The resulting numbers remain partial evidence. Goodhart's law explains why a proxy can lose its relationship to the intended outcome once it becomes a target, while distributions, counterevidence, and regression checks make that divergence easier to see. Telemetry and observability address a different limitation: they help investigate where and why measured behavior occurred, including conditions the benchmark did not anticipate.

The same separation applies to agentic development. The accepted implementation is the outcome to evaluate; the agent trace, configuration, retries, and human interventions are evidence for explaining how that outcome arose. Neither an application trace nor an agent transcript proves correctness by itself.

3.7 References

1. Kalliola, J., & Vepsäläinen, J. (2025). Challenges Related to Approximating the Energy Consumption of a Website. IEEE Access, 13, 139001–139017. https://doi.org/10.1109/ACCESS.2025.3596459

2. Barrocas, A., Jardim Nunes, N., Nisi, V., & Martelaro, N. (2026). EcoAssist: Embedding Sustainability into AI-Assisted Frontend Development. Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems, 1–16. https://doi.org/10.1145/3772318.3791330
3. Little, J. D. C. (1961). A Proof for the Queuing Formula: L = λW. Operations Research, 9(3), 383–387. https://doi.org/10.1287/opre.9.3.383
4. Goodhart, C. A. E. (1975). Problems of Monetary Management: The U.K. Experience. In Papers in Monetary Economics (Vol. 1). Reserve Bank of Australia. https://www.econbiz.de/Record/problems-of-monetary-management-the-u-k-experience-goodhart-charles/10002525062
5. Giray, G., Demirörs, O., Kalinowski, M., & Mendez, D. (2025). An Empirical Study of Generative AI Adoption in Software Engineering. arXiv Preprint arXiv:2512.23327. https://doi.org/10.48550/arXiv.2512.23327
6. Heričko, T., Šumak, B., & Brdnik, S. (2021). Towards Representative Web Performance Measurements with Google Lighthouse. Proceedings of the 2021 7th Student Computer Science Research Conference, 39–42. https://doi.org/10.18690/978-961-286-516-0.9

PART II

Challenge — Expose architectural assumptions

Implementation A established a bounded baseline, not an architecture assumed to be correct. Challenge asks which hidden boundaries made that application work, what it costs to change them, and which assumptions fail when the application leaves its controlled starting conditions. The result is a second implementation and evidence package that can be compared with the baseline rather than a revision of the same codebase.

II.1 From hidden assumptions to comparable evidence

The three chapters form one progression:

1. How does web architecture shape scalability? separates responsibilities from locations, traces how web placement choices accumulated, and turns one implementation A decision into an architecture hypothesis.
2. How does development friction limit web projects? follows one browser-visible change across setup, code, contracts, validation, review, and deployment to expose where the project resists change.
3. Which web application assumptions fail under pressure? connects documented production failures, application signals, degradation, and recovery to assumptions that local correctness left untested.

By the end of Challenge, implementation B should preserve the common required behavior while expressing a fresh architecture and a declared foundation-model single-agent workflow. Its measurements and development evidence should remain comparable with implementation A, while its differences become hypotheses to examine rather than improvements attributed automatically to the model.

II.2 Implementation B — Work Activity Tracker

The Challenge project specification defines the second fresh implementation. Requirements, permitted acceptance material, shared pressures, and reporting categories remain stable. Application code and implementation-specific tests do not carry forward, and neither the earlier architecture nor its development history becomes the default. The exact transfer policy remains defined by the project brief.

Students now know the domain better than they did during Establish. That practice effect is part of the comparison: record what changed, distinguish model and workflow differences from team learning where the evidence permits it, and leave the explanation unresolved when it does not. Use

implementation A and its public discussion as evidence about earlier choices, not as a repository to improve incrementally.

II.3 Technical debt reduces optionality

Technical debt names earlier choices that make later change harder. A shortcut can be rational while a team is learning or time is limited; the problem begins when its future cost stays invisible, unpriced, or allowed to compound. A system with high debt may still run, pass tests, and satisfy users today while giving the team fewer safe choices tomorrow.

Architecture makes those commitments visible, friction reveals their daily cost, and production pressure shows which ones restrict recovery or growth. Challenge therefore treats debt as one supporting lens for comparing the two implementations rather than as a synonym for defects or for every simple design.

> **Sustainability angle**
>
> Inefficient design can become sustainability debt. Unbounded logs, poor retention policies, N+1 queries, heavy dependencies, and wasteful background jobs may not break the system immediately, but they compound infrastructure cost and resource use over time.

CHAPTER 4

How does web architecture shape scalability?

Students often experience web development as a collection of frameworks, libraries, and deployment platforms. Those tools change quickly, but web applications still have to connect consumers, representations, computation, and authoritative data across networks that introduce latency, partial failure, independent change, and untrusted input.

Web architecture has not progressed by replacing one correct design with another. Instead, it has expanded the places where responsibility can live. Physical servers made the machine visible; cloud platforms made capacity programmable; content delivery networks placed copies near consumers; programmable edge platforms placed bounded computation beside those copies; and modern rendering approaches divide work across build systems, servers, edge locations, and browsers. As computation became more commoditized, architects gained placement options without escaping the constraints of distance, state, trust, cost, and failure.

The central claim of this chapter is that every movement of responsibility addresses a particular pressure and creates a new coordination obligation. Newer placement is not inherently better. The useful question is what should move, why it should move, and which contract must remain stable when it does.

4.1 A conceptual model: separate roles from locations

A framework-shaped label such as “serverless application” or “single-page application” combines several decisions that can be examined separately. Begin with four architectural roles:

Table 4.1. Four roles that remain useful as web deployment locations change.

Role	Question it answers	Typical examples
Consumer	Who or what expresses intent and observes the result?	A person using a browser, an integration, a crawler, or an agent
Representation	What crosses a boundary to make state or action visible?	HTML, JSON, an image, a stream, a form, or a hypermedia control
Execution	Where is behavior evaluated?	A build process, browser, physical server, cloud service, or edge node
Authority	Which component decides and preserves the current truth?	An application service, database, event log, or external system

Roles are not machines. One machine may perform several roles, and one role may be distributed across many machines. An **origin** is the authoritative source for a representation in an HTTP exchange, not necessarily one physical server. Likewise, an edge function may execute application logic without becoming the authority for the data it reads.

Use the event tracker to keep the model concrete. A `GET /events` request asks for a representation and may be answered from a browser cache, an edge cache, or the origin. A `POST /events` request expresses an action whose validation may begin near the consumer, but whose accepted result must eventually

reach the component that owns the event history. The two paths can share an interface while placing execution, copies, and authority differently.

> **Changed consumer assumption: agents**
>
> An agent may follow links in rendered HTML, operate a browser, call an API, or invoke a capability-oriented interface. It still reaches the application through an existing consumer boundary, but it can act repeatedly, retry after ambiguous failure, traverse interfaces faster than a person, and initiate consequential actions without a person examining every intermediate screen. Contracts, authorization, idempotency, rate limits, action confirmation, and evidence therefore become more important; the agent does not become a new architectural tier. The same operation should have one authoritative meaning whether a person clicked a form, an integration sent a request, or an authorized agent invoked it.

4.1.1 Request boundary lab

The demonstration below traces one `POST /events` request through the browser, network, API, storage, and response path. Each boundary has an owner, a failure mode, and a contract that future clients, humans, and agents may depend on.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/architecture/

Use this trace before drawing the architecture diagram. If the request path is unclear, the diagram is likely to hide the most important architectural decisions.

4.1.2 Automated consumers change demand and exposure

A crawler, integration, monitoring probe, authorized agent, and abusive bot can send technically similar HTTP requests while creating very different value, cost, and risk. Architecture should therefore distinguish which representations are intentionally public, which actions require identity and authorization, which expensive paths need a quota or admission boundary, and which automated uses the product is willing to support.

The Robots Exclusion Protocol lets a cooperating crawler discover access preferences in `/robots.txt`. It is not authorization or a content-security boundary, and listing a path there makes the path discoverable. Information that must remain private still needs an application-layer access control; public information should be designed with the assumption that a permitted consumer can copy it.

Rate limiting controls resource use rather than deciding whether every automated consumer is malicious. An HTTP `429 Too Many Requests` response can explain the limit and carry `Retry-After`, but RFC 6585 deliberately leaves the identity and counting policy to the application. Limits based only on an IP address can group unrelated users or be distributed across many addresses, while limits tied to an account, tenant, capability, or costly operation require their own fairness and privacy decisions.

The OWASP automated-threat catalog treats scraping as one form of automated use among several, and its practical implication here is to begin from the endpoint and intended outcome rather than from a generic "block bots" goal. For one exposed path, record the legitimate consumers, information returned, authoritative action or cost triggered, likely abuse pattern, first useful signal, control point, acceptable false-positive cost, and recovery or override path. Cache safe public reads, minimize exposed and retained data, protect expensive or consequential actions, and observe origin work per useful outcome before adding a challenge to every request.

> **Sustainability angle**
>
> Placement determines resource flow. Chatty APIs, unnecessary round trips, duplicated computation, oversized payloads, heavy client bundles, aggressive polling, and poorly targeted edge execution all turn architectural choices into recurring resource use. Moving work is valuable when it removes a justified constraint rather than merely multiplying infrastructure.

4.2 The origin-centered web made machines visible

In an origin-centered design, a browser sends a request to a known server, the server applies application logic, reads or changes data, and returns a representation. The earliest useful deployment for a dynamic site may be one physical machine running the web server and database. Larger installations can place a reverse proxy or load balancer in front of several application servers and move the database to separate hardware. This is often described as a three-tier shape—presentation, application logic, and data—even though the tiers are responsibilities and do not have to occupy three machines.

Physical ownership makes several constraints difficult to ignore. Capacity is purchased and installed before it is used, a failed machine has to be repaired or replaced, and vertical scaling has a hardware ceiling. Horizontal scaling adds machines, but then request routing, deployment, shared state, and data coordination become system responsibilities. The architecture can still be excellent: a stable workload, predictable cost, data locality, regulatory control, or specialized hardware may justify owned infrastructure.

The important distinction is not "old server" versus "modern platform." It is whether the application assumes one long-lived machine, several interchangeable processes, or services whose lifecycle is controlled elsewhere.

4.2.1 HTTP provides a stable seam

HTTP semantics define a stateless request-response protocol in which clients act on resources through messages and receive status information and representations in return. The protocol intentionally hides how a service is implemented behind a uniform interface. It also allows intermediaries—proxies, gateways, and tunnels—to participate between the user agent and origin. Those properties let deployment topology change without requiring every consumer to understand the new arrangement.

A request method communicates intent, the target identifies what the request concerns, headers carry metadata and control information, and the response status describes the result. Correct use of safe and idempotent methods, validators, cache metadata, authentication, and content negotiation affects what browsers and intermediaries are allowed to do. HTTP is therefore architectural material rather than plumbing supplied by a framework.

HTTP being stateless does not mean the application has no state. It means each request has understandable semantics in isolation from the connection that carried it. Identity, authorization, workflow continuity, and durable state still need explicit homes. The data section returns to that distinction.

4.2.2 Application contracts outlive deployment choices

HTTP can carry several application interaction styles, including resource-oriented messages, hypermedia controls, GraphQL operations, and RPC calls. Each style distributes knowledge and change pressure differently between consumers and providers, but none removes the need to define representations, errors, retries, authorization, and compatibility.

For the project, choose one minimal event-creation and event-query contract, then ask how it changes when requests are retried, devices go offline, workflow states evolve, or consumers need additional data. Appendix C compares the interaction styles and contains the API contract lens and executable HTTP example.

4.2.3 Organizational boundaries shape application contracts

An application contract is also a communication contract between its owners. In his original 1968 paper, Melvin Conway observed that organizations tend to produce system structures shaped by their communication structures. For web architecture, this means that a shared API or data model can become the place where several teams must coordinate, while a nominally independent service can remain tightly coupled when every change still requires the same people to agree.

Do not create a service boundary merely to imitate an organizational chart. Instead, identify a capability with coherent authority and change reasons, give its owners enough control to evolve its internals, and keep the consumer contract small enough that other teams can depend on it without joining every implementation decision. When one workflow crosses several owners, record who may change the contract, how compatibility is checked, how consumers learn about deprecation or migration, and which decision requires cross-team agreement. The ownership card later in this chapter should therefore name both the code boundary and the communication boundary that maintains it.

4.3 Cloud made infrastructure programmable

Cloud computing changed how capacity is acquired and operated more than it changed the physical laws beneath an application. The NIST definition of cloud computing emphasizes on-demand access to a shared pool of configurable resources, rapid provisioning and release, elasticity, and measured service. These properties made compute, storage, and networking available through APIs instead of only through hardware procurement and manual installation.

4.3.1 From named machines to replaceable capacity

A cloud-hosted application may still run the same web server and database that ran on owned hardware. The architectural shift appears when instances can be created from declarations, replaced after failure, scaled with demand, and distributed across failure zones. Teams can move from a pet-like server with accumulated local state toward replaceable instances whose configuration and dependencies are explicit.

This shift favors stateless request handlers because interchangeable instances are easier to add, remove, and recover. It also makes hidden machine-local assumptions more dangerous. A session, uploaded file, background job, or generated artifact stored only on one instance can disappear when the platform replaces it or can become unreachable when the next request reaches another instance.

Elasticity is not infinite capacity. New instances take time to start, quotas and downstream services impose limits, and a database or external API may saturate before the application tier does. Measured service also turns inefficient design into a recurring bill. Cloud architecture therefore requires capacity signals, cost signals, and explicit saturation boundaries rather than faith in automatic scaling.

4.3.2 Managed services move operational work

Object storage, managed databases, identity services, queues, functions, and observability platforms let a team delegate installation, patching, replication, or availability work to a provider. That can reduce undifferentiated operations and make sophisticated capabilities accessible to smaller teams. It also creates service contracts: supported regions, quotas, consistency behavior, failure modes, pricing dimensions, export paths, and recovery procedures become architectural dependencies.

Cloud did not make infrastructure disappear. It changed who controls each layer and which parts the application can observe. An architecture diagram should therefore show managed dependencies and provider boundaries instead of collapsing them into an anonymous cloud shape.

4.4 CDNs grew into a programmable edge

Cloud platforms made centralized pools of computation easier to allocate. Content delivery networks addressed a different physical constraint: a distant origin cannot make light travel faster, and repeated delivery from one location wastes both time and origin capacity.

4.4.1 CDNs began by moving representations

The web had used proxies, replicas, and caches long before “edge” became a general application-platform label. RFC 3040, published in 2001, documents an already distributed web of origins, surrogates, caching proxies, replica selection, and cache coordination. RFC 6392 describes a CDN as content-delivery, request-routing, distribution, and accounting infrastructure in which edge servers deliver copies close to users.

This content-oriented model moves a reusable representation while the origin remains authoritative. For the event tracker, public scripts, styles, images, and a non-personalized summary can be delivered near consumers without moving the event-writing authority. The benefit depends on reuse: a

personalized or rapidly changing representation with a fragmented cache key may miss often enough that the edge only adds another hop.

4.4.2 Programmability moved request handling beside the cache

Once providers had globally distributed request-routing and delivery infrastructure, they could expose some processing at the same locations. For example, the 2017 Lambda@Edge release connected functions to CloudFront viewer and origin request or response events, allowing code to inspect, rewrite, authorize, personalize, or generate responses near users. This illustrates the expansion from selecting and serving content to running bounded application logic in the delivery path.

Edge is relative rather than singular. An execution location can be close to a user but far from the authoritative database, close to a device but outside a cloud region, or close to an origin service while still crossing organizational boundaries. Moving request validation, routing, personalization, or rendering outward can reduce latency and origin work. Moving an authoritative write outward can instead create a coordination problem across locations.

The edge therefore extends rather than replaces origin and cloud architecture. Use it when locality, repeated delivery, early rejection, or distributed ingress addresses a measured pressure. Keep work centralized when it depends on strongly coordinated data, specialized runtime capabilities, or operational simplicity.

4.5 Rendering moves work across time and place

Cloud and edge platforms expanded where code can execute, while build tooling and capable browsers expanded when a representation can be created. Rendering labels often collapse these decisions into one framework-shaped category.

Server-side rendering (SSR), static site generation (SSG), incremental static regeneration (ISR), and client-side rendering (CSR) primarily describe **where and when a view is produced or refreshed**. Multi-page applications (MPAs), hypermedia-driven applications (HDAs), and single-page applications (SPAs) primarily describe **how the browser reaches later application states**.

CSR is common in single-page applications because the browser can fetch data, render views, and handle later navigation without replacing the document. The terms are not synonyms: CSR is a rendering strategy, while SPA describes a broader navigation and state-management shape. An HDA can receive its initial document through SSR, while an MPA can serve pages from a statically generated or incrementally regenerated cache.

The useful comparison is not which acronym wins. Ask what each decision does to freshness, origin or edge work, navigation, client responsibility, caching, accessibility, and failure.

4.5.1 Web architecture lens

Use the same event-recording interaction across nine combinations. First choose how the initial document is produced, then choose how the application presents the state after `POST /events`. Run the trace and watch which artifact crosses the network.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/architecture/

The ISR behavior in the lab is a simplified stale-while-revalidate model. Exact regeneration, invalidation, and consistency behavior depend on the framework and deployment platform. That platform-specific contract should be documented as part of the architecture.

4.5.2 Activation is a separate architectural decision

Rendering and navigation do not explain how delivered HTML becomes interactive. Hydration, progressive hydration, islands, and resumability move different amounts of code, state, and startup work across the server-browser boundary. Record which regions need behavior, when that behavior becomes available, and which state or communication constraints the approach introduces. Appendix C compares the activation approaches and provides a focused lab.

4.5.3 Runtime generation adds a control boundary

A component written by a coding agent during development becomes ordinary application code once it has been reviewed and committed. Runtime-generative UI is different: model output can select data, composition, or executable behavior while the application is running.

The more freedom the output receives, the more the host must control. A declarative descriptor needs an approved component and action registry, schema validation, authorization when actions execute, bounded structure, and a safe fallback. Generated executable code additionally needs isolation without ambient credentials, a narrow bridge to host data and tools, explicit permissions, resource limits, and observability. Appendix C develops the full control-boundary spectrum and a descriptor-boundary audit.

4.6 Data limits how freely responsibility can move

Compute and representations can often be duplicated or replaced more easily than authoritative data. A read may be served from a replica or cache, but concurrent writes still need a rule for which result becomes true. Moving execution closer to users without deciding where authority lives can exchange network latency for inconsistent state, complex conflict resolution, or a longer path to the real owner.

For the event tracker, raw events, current work-item state, and derived summaries have different roles. Raw events may support replay or audit, current state must preserve workflow invariants, and summaries can often be rebuilt or temporarily stale. The architecture should state which data is authoritative, which is derived, where each is placed, and what happens when a location cannot reach its authority.

4.6.1 Data placement follows authority and access

An origin-centered application can keep its primary database close to the application server and pay one predictable network cost for most operations. Cloud services make storage independently provisioned and managed, and they make replicas, object stores, queues, and analytical systems easier to combine.

Edge execution makes the remaining distance more visible: code may run near a consumer while the authoritative write still has to cross a region or continent.

Choose a data shape from the behavior that must remain correct and the access path that must remain efficient. A relational model is useful when constraints and relationships must be enforced together; a document can keep an aggregate together; a key-value store can serve known-key access; object storage can hold large immutable representations; and an append-oriented log can preserve history for replay or derivation. These are composable roles rather than stages of progress. One application may use a relational source of truth, object storage for attachments, and a derived search index without giving all three equal authority.

Distribution makes four distinctions especially important: authoritative versus derived data, reads versus writes, transactional versus analytical work, and current state versus retained history. Replicating a read path may reduce latency without changing write authority. Partitioning can increase capacity or isolation, but then the partition key determines which queries and transactions remain local.

4.6.2 Stateless requests still depend on state

Stateless request handling means that an interchangeable server does not need an earlier request's hidden in-process session to interpret the next one. It does not mean that the application has no state. A later request may carry an explicit workspace or workflow handle, while the service still has to authenticate the caller, authorize access, resolve authoritative state, and define that state's lifecycle.

A handle is an address, not authorization. If its state exists only on the instance that minted it, affinity has merely moved behind a new identifier. Likewise, a protocol request identifier does not automatically make a domain mutation safe to retry. When a mutation can commit before its response is lost, use a stable application command or idempotency key and store the mutation and reusable result atomically at the authoritative boundary.

Appendix C develops state placement, authorization, request continuity, and a two-instance retry audit.

4.6.3 Transactions preserve invariants across failure and concurrency

A transaction groups operations that must commit together and defines how concurrent work may interact. For the event tracker, a successful lifecycle transition must update current state and append its activity event without allowing either write to escape alone. The deployed database's constraints, isolation level, acknowledgment point, and failure model determine the actual guarantee.

Appendix C develops ACID, concurrent transitions, and the transaction lab.

4.6.4 Model valid data combinations

A transaction constrains how a valid state change becomes durable. Constructive data modeling reduces the contradictory combinations that ordinary code can create before that boundary. Use a tagged union, sealed hierarchy, guarded constructor, or another stack-native representation when each outcome requires different data.

A precise in-memory representation does not establish authorization, persistence, or concurrency safety. Untrusted or deserialized input still needs runtime parsing, and the durable write must still check authoritative state and preserve the relevant invariant atomically.

Appendix C contains a complete transition-result example and a selected-invariant exercise.

4.7 Caching coordinates controlled copies

Caching began as a central content-delivery technique, but its architectural reach is wider than the CDN. A browser can keep a private response, a shared intermediary can reuse an HTTP representation, an application can retain a computed result, and a data system can expose a replica or materialized view. Each case creates a controlled copy whose identity, lifetime, and relationship to authority must be defined.

Table 4.2. Caching layers as controlled copies of authoritative or derived state.

Cache location	Copy being reused	Main benefit	Main correctness question
Browser	A response for one user agent	Avoids transfer and repeated work	May this user reuse it, and when must it validate?
CDN or reverse proxy	A response shared across suitable requests	Reduces latency and origin work	Which request properties belong in the cache key?
Application	A query, computation, or assembled view	Avoids repeated service or data work	What invalidates it, and can callers accept bounded staleness?
Data layer	A read replica, index, or materialized result	Moves or accelerates read work	How far may it lag behind the authoritative write path?

4.7.1 HTTP cache control is application policy

An HTTP cache can avoid an origin request, reduce transferred bytes, and hide network latency only when reuse is correct for the response and its consumers. The response must communicate who may store it, how long it remains fresh, how it can be validated, and whether stale reuse is acceptable.

The HTTP caching specification defines the core directives and freshness model:

- `public` permits storage by shared caches, while `private` limits a response to private caches.
- `max-age` defines freshness relative to response generation; `s-maxage` can define a different freshness lifetime for shared caches.
- `no-cache` still permits storage, but the stored response has to be validated before reuse.
- `no-store` asks caches not to store the response at all.
- `must-revalidate` prevents reuse of a stale response without successful validation.
- `Vary` makes selected request headers part of response selection. This can protect correctness, but every varying dimension can fragment the cache.

Validators avoid retransmitting an unchanged body. An origin can attach an `ETag`; once the stored response needs validation, a cache sends that value through `If-None-Match`. A matching validator allows the origin to return `304 Not Modified` without the representation body, while a changed validator produces a new `200` response.

The `stale-while-revalidate` extension moves latency again: a cache may immediately serve a stale response for a bounded period while validating it in the background. That can be appropriate for a public activity summary, but not for every response. Personalization, authorization, invalidation, and the cache key are correctness and security decisions rather than deployment details.

4.7.2 Cache decision lab

The lab below follows a second request through browser cache, shared cache, and origin. Change the response profile, move time forward, and decide whether the origin representation changed. Then inspect whether the request is served fresh, served stale while validation happens, conditionally revalidated, or fetched in full.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/architecture/

Cache behavior should be tested at the deployed boundary because application code is only one participant. A framework, reverse proxy, CDN, browser, or hosting platform may add, remove, or reinterpret cache metadata. Record the final response headers, cache status, `Age`, and origin traffic rather than assuming the source-code declaration is the complete policy.

4.8 Architecture is maintained as responsibility moves

A system does not avoid architectural decay merely because it began with a diagram or a recognizable framework. Foote & Yoder (1997) identify throwaway code, piecemeal growth, and pressure to keep the system working as forces that gradually produce a Big Ball of Mud. The growing set of placement options can accelerate that decay when a project adopts cloud services, edge functions, browser state, caches, and generated components without recording which role each one owns.

Conscious architecture does not mean designing the entire system in advance. It means repeatedly deciding which boundaries must remain visible, which shortcuts are temporary, and when working code needs to be reorganized before further growth makes change unsafe. Christopher Alexander's pattern language offers a middle path between a complete upfront blueprint and unregulated growth: shared patterns allow local, piecemeal changes while preserving relationships that contribute to a coherent whole (Alexander et al., 1977).

In an agentic project, architecture notes, specifications, examples, tests, and decision records give humans and agents a project-specific language for making changes without rediscovering—or accidentally redefining—the system on every task. The cathedral and bazaar metaphor describes how development and feedback are organized, whereas a Big Ball of Mud describes the resulting structure (Raymond, 2001). Either development model can preserve architecture through stable boundaries, review, and continuous integration, and either can produce mud when local changes accumulate without stewardship.

4.8.1 Make the implementation architecture reviewable

Students have already made architecture choices in implementation A, even if they did not yet have a complete vocabulary for them. The public project discussion should make those choices visible: different groups received the same behavioral specification but chose different stacks, boundaries, storage models, and request paths.

Each implementation should treat a small set of project memory files as architectural artifacts:

- `ARCHITECTURE.md` records global rules and cross-cutting constraints, such as application shape, storage model, validation boundaries, deployment assumptions, and rules future changes should preserve.
- `specs/{feature}/spec.md` records feature-level contracts, workflows, regression guardrails, and verification steps.
- `docs/adrs/` or a simple decision log records durable decisions and trade-offs that future groups would otherwise have to rediscover.
- `AGENTS.md` records how AI tools should operate in the repository, including required commands, review expectations, protected files, and escalation points.

If the code, diagram, specification, and decision log disagree, the project has an architectural problem even if the application still runs.

A component diagram can show that two boxes exist without explaining which one owns a decision. For each independently evolvable capability, record its source root, composition root, state authority, public contracts, allowed dependency direction, and intentionally excluded responsibilities. The resulting ownership card is a review aid rather than a demand for one directory per feature.

File size, fan-in, churn concentration, or a crowded directory can act as structural smoke alarms, but none proves that a split is correct. When such a signal fires, inspect whether several authorities or change reasons have accumulated together. Consolidate ownership, split responsibilities, or document a justified exception; do not divide a file merely to satisfy a number.

4.9 Architectural dimensions for a web application

The history of the web expanded the available choices along several dimensions. Use the table to compare architectures without treating any option as a universal destination.

Table 4.3. Architectural dimensions to examine when designing or comparing web applications.

Dimension	Choices that expanded over time	Pressure the choices can address	Obligation introduced by the decision
Infrastructure ownership	Owned server, hosted machine, VM, container, function, managed service	Provisioning speed, utilization, operational capacity	Provider contracts, quotas, cost visibility, and recovery
Execution placement	Build system, browser, origin, cloud region, edge location	Latency, elasticity, device capability, or data locality	More boundaries, deployment targets, and partial-failure paths
Representation rendering	Build time, request time, incremental regeneration, client time	Freshness, origin work, delivery speed, personalization	Invalidation, duplicated logic, activation, and fallback behavior
Browser interaction	Document navigation, hypermedia updates, client-managed navigation	Interaction richness, continuity, client responsiveness	Client state, compatibility, accessibility, and recovery

Dimension	Choices that expanded over time	Pressure the choices can address	Obligation introduced by the decision
Client activation	Hydration, progressive hydration, islands, resumability	Initial payload and time to useful interaction	Serialization, loading policy, tooling, and cross-boundary state
State authority	Client-local state, application service, database, event log, external owner	Correctness, offline work, audit, and integration	Invariants, authorization, retries, and lifecycle
Data distribution	Central store, replicas, partitions, derived views	Volume, locality, isolation, and read or write pressure	Consistency rules, routing, migration, and reconciliation
Caching and freshness	Browser, intermediary, application, and data-layer copies	Repeated work, transfer, latency, and origin capacity	Cache identity, freshness, invalidation, privacy, and observability
Consumer contract	HTML and forms, resource HTTP, hypermedia, GraphQL, RPC, events	Independent evolution and different consumer needs	Compatibility, idempotency, rate limits, and bounded authority
Operational ownership	One team and machine through multiple teams, providers, and regions	Organizational scale and specialized capability	Explicit ownership, signals, incident paths, and decision records

The rows interact. Moving rendering to the edge may require a cache-freshness policy; moving interaction to the browser may require a new API contract; moving compute into replaceable cloud instances may require state to leave process memory; and moving reads near users may lengthen the path to authoritative writes. A design is coherent when these consequences agree rather than when it selects the newest option in each row.

4.10 Project artifact

Students should review the architecture and data-flow diagrams submitted with implementation A, identify one consequential architectural choice, and write an architecture hypothesis for implementation B. The hypothesis should use the conceptual model and dimensions table to state:

- which consumer, representation, execution location, and authority the selected workflow uses,
- which pressure justifies preserving or changing that placement,
- what new contract, consistency rule, or operational obligation follows,
- which automated consumers may reach the selected path and where cooperation, access control, quotas, or rate limits belong,
- which team or organizational boundary owns the contract and which changes require coordination across it,
- and what evidence would show whether the decision works.

Students should also complete an ownership card for the event workflow and identify one structural signal that would cause them to review that ownership again. Implementation B should still include its own diagrams and decision record.

4.11 Summary

Web architecture has accumulated placement choices rather than moving through a ladder of universally better designs. Origin-centered systems make physical machines and request paths visible. Cloud platforms make pooled capacity programmable and move operational work into provider contracts. CDNs move reusable representations closer to consumers, while programmable edge

platforms can also move bounded request handling into that delivery path. Rendering divides work across build time, request time, edge execution, and the browser; durable data authority constrains how freely that work can move; and caching coordinates the copies created along the way.

HTTP and application contracts let consumers survive many of these topology changes, including the addition of agents as more autonomous consumers. The architectural task is to place each responsibility in response to a real pressure, preserve the authority and contracts that make the system coherent, and remain able to explain and revise the decision as the application changes.

4.12 References

1. Foote, B., & Yoder, J. W. (1997). Big Ball of Mud. Proceedings of the Fourth Conference on Pattern Languages of Programs. https://www.laputan.org/mud/
2. Raymond, E. S. (2001). The Cathedral and the Bazaar: Musings on Linux and Open Source by an Accidental Revolutionary (Revised). O'Reilly Media. http://www.catb.org/~esr/writings/cathedral-bazaar/
3. Alexander, C., Ishikawa, S., Silverstein, M., Jacobson, M., Fiksdahl-King, I., & Angel, S. (1977). A Pattern Language: Towns, Buildings, Construction. Oxford University Press.

CHAPTER 5

How does development friction limit web projects?

A web application does not have to serve more users before it encounters a scaling problem. It can first become difficult to change. A feature that sounds local may cross a browser form, an HTTP contract, server-side validation, persistent data, a query, a cached representation, and a deployment path. Each boundary adds knowledge, evidence, and coordination that the change must preserve.

Development friction is the resistance a team feels when trying to set up, understand, change, test, review, deploy, or operate that system. It is the everyday experience of technical debt, hidden coupling, and weak feedback. A system with high friction may still be useful, but each change requires more effort than its behavior alone would suggest.

The central claim of this chapter is that friction accumulates where a web change crosses boundaries and commitments. Some of that friction protects users, data, and collaborators; some merely repeats work. The goal is not to remove every constraint, but to move useful evidence earlier and remove resistance that provides no corresponding control.

5.1 Trace one web change from the browser to production

Consider one change to the event tracker: add an optional event category and let users filter the activity view by category. The visible request is small, but its implementation can follow this path:

1. **Browser form:** add a control, preserve accessible labels and error states, and decide what an omitted category means.
2. **HTTP contract:** extend the event representation without breaking an older client or independently implemented consumer.
3. **Validation:** apply the same allowed values and defaulting rules at the untrusted-input boundary.
4. **Storage:** persist the category, account for existing events, and decide whether a migration or backfill is required.
5. **Query and view:** filter without changing authorization, pagination, ordering, or the empty state.
6. **Cache:** include the filter in the cache identity and invalidate or tolerate previously derived representations.
7. **Deployment:** run compatible versions together, observe the new path, and preserve a safe way to stop or reverse the change.

This path is not a prescription for seven separate components. It is a way to find where one behavior is represented and who owns it. In a small application, several stages may live in one module and deploy together. They remain distinct responsibilities because a mistake at each boundary produces different evidence and a different user-visible failure.

Friction becomes visible when the path is hard to trace, the same rule has several unsynchronized definitions, a local check cannot exercise the changed boundary, or a deployment makes earlier assumptions costly to recover. The path also gives the chapter a stable unit of analysis: ask what the category change costs in implementation A, then repeat the same audit for implementation B.

> **Sustainability angle**
>
> Friction creates repeated work. Slow setup, flaky tests, confusing documentation, and unclear code make humans and agents rerun commands, regenerate changes, and spend more compute to reach the same result. Developer experience therefore has an indirect resource cost.

5.2 Recognize where friction appears

The category change exposes several related forms of resistance:

- **Setup friction** appears when a fresh checkout cannot reach a running application and baseline checks without undocumented local state.
- **Comprehension friction** appears when a developer cannot find where the category contract, query, or cache policy is owned.
- **Change friction** appears when one rule must be edited in unrelated places or when hidden consumers fail after the contract changes.
- **Testing friction** appears when checks are slow, flaky, or unable to distinguish a browser problem from an API, storage, or cache problem.
- **Review friction** appears when generated or human-written diffs mix the category change with unrelated rewrites and configuration churn.
- **Deployment and operational friction** appears when compatibility, rollout, signals, or recovery depend on knowledge that is not represented in the repository or platform.
- **Coordination friction** appears when ownership is unclear and teams or agents duplicate work, wait on one another, or make incompatible assumptions.

These categories interact. For example, unclear ownership increases comprehension time, produces wider changes, and sends more uncertainty into review. A slow or untrusted test suite encourages larger batches, making a failure harder to localize. An undocumented environment turns every collaborator into a separate integration experiment.

Not all of this resistance is waste. Authentication, validation, compatibility checks, migration stages, review, and release gates can be **protective friction** when they control a named risk and return useful evidence. A flaky test, repeated manual setup, or approval that merely duplicates an automated check is **accidental friction** when it delays work without increasing confidence or control. Before removing a step, ask which failure it is meant to prevent and what evidence would be lost.

5.3 What documented web-development cases reveal

The accounts below were published by the organizations that performed the work. They are useful evidence about particular systems, not controlled comparisons or transferable performance promises. Follow the links for the architecture and migration details, then use the transfer question to examine the event tracker.

Table 5.1. Documented cases interpreted as forms of web-development friction.

Documented case	Friction exposed	Intervention and reported result	Transfer question for the event tracker
Spotify Golden Paths, Backstage, and Tingle	Creating a web project required manual coordination across a repository, cloud project, CI/CD, documentation, and other engineers	Spotify automated a common path for websites and backend services and reports reducing setup to a first build and deployment from about 14 days to under five minutes	Which repeated setup and deployment decisions can become a maintained project path?
GitHub.com development in Codespaces	Local environments and branch changes became brittle as dependencies, services, and schemas changed	GitHub used declared environments, cached dependencies, and prebuilds, reporting that GitHub.com environment creation fell from 45 minutes to 10 seconds	Which environment assumptions should be versioned, rebuilt, or replaced instead of repaired by hand?
Shopify's modular Rails monolith	Hundreds of developers needed understandable ownership and dependency boundaries without stopping feature work	Shopify replaced call-graph analysis taking over an hour with Packwerk static analysis taking a few minutes, allowing dependency violations to become pull-request feedback	Which dependency or ownership rule would make the category change easier to locate and review?
Stripe's online subscription migration	A data-model change had to preserve availability and accuracy across hundreds of millions of live objects	Stripe staged the change through dual writes, moving read paths, moving write paths, and removing the old model while the service remained operational	At which step does the data change become a coordinated migration rather than an ordinary code revert?
Slack's CI/CD circuit breakers	Test traffic, queues, retries, and dependency overload caused misleading failures and cascading disruption	Slack deferred and shed test work under pressure, reporting no cascading internal-tooling incidents during the following two years while its time-to-test-results metric held steady	When would delaying or selecting a check return more trustworthy feedback than running everything at once?

The cases cover the same route from a different angle. Spotify reduces the decisions required to start and deploy a web project; GitHub treats the development environment as replaceable infrastructure; Shopify brings architectural feedback into review; Stripe accepts deliberate migration work to protect live data; and Slack adds control to keep the validation system useful under load. Together they show why a shorter workflow is not automatically a better workflow. The useful outcome is a change path with less repeated work and enough evidence at each consequential boundary.

5.4 Find the commitment point

Fast generation can make a change cheap to introduce without making it cheap to reverse. In a repository study of ten open-source database applications, schema revisions often co-evolved with dependent application code, illustrating how a local data-model choice can acquire a wider change footprint (Qiu et al., 2013).

For the category change, the **commitment point** is the first moment when an ordinary code revert no longer restores the prior state. This may occur when persistent production data adopts the new representation, an independently deployed consumer relies on the extended contract, a queued job

carries the new value, or a cached artifact can outlive the deployment. Beyond that point, reversal becomes a coordinated change involving compatibility, migration, invalidation, reconciliation, or communication.

A commitment is not automatically technical debt. The audit asks whether its future coordination cost was visible and justified. Before the point, deleting a candidate implementation may be enough. Near the point, expand-and-contract changes, mixed-version tests, staged rollout, and observation add protective friction because they preserve options. After the point, “roll back” must name what happens to accepted data and dependent consumers rather than only which code artifact is deployed.

The Reality chapter returns to compatibility, migration, rollback, and fix-forward under production pressure. Here, make the asymmetry between introduction and safe reversal visible. Florian Herrengt's practitioner essay dramatizes this asymmetry with database changes; its broader labor-market prediction is outside this claim.

5.5 Reduce friction without hiding the boundary

5.5.1 Treat the environment as a versioned input

A dependency lockfile does not describe the whole development environment. Runtime and package-manager versions, system packages, local services, configuration names, process readiness, supported platforms, and relevant resource limits can remain hidden on one workstation. For an agent, this state is part of the task rather than neutral background.

Declare the supported environment and one path from a fresh checkout to a running application and its baseline checks. Record required configuration names without secret values, inject secrets at runtime, and name the host, hardware, network, mutable data, and external-service boundaries that remain outside the declaration. A container or environment manager can implement this contract, but its presence does not by itself prove isolation or reproducibility.

Use the cross-team audit as the test. Record setup time and outcome, undocumented intervention or retry, the failure stage, and relevant host or resource conditions. After a repair, replay the path from fresh state. The result supports a bounded claim about that setup—not that production is identical or that a stochastic agent will produce the same patch.

Appendix B: Reproducible development environments distinguishes setup automation, declared environments, hermetic execution, and reproducible builds and provides the complete fresh-state audit.

5.5.2 Make ownership and contracts traceable

The category path should have one understandable owner for each rule even if several layers enforce it. The browser may give immediate feedback, but the server-side input boundary owns acceptance. The database may constrain stored values, but the application contract owns what consumers are promised. The cache may copy a representation, but it does not own the authoritative event.

Record those responsibilities in the architecture, specification, schema, types, and tests that future developers and agents are likely to inspect. Keep duplicate enforcement mechanically connected where possible—for example, generate types from a schema or run the same contract examples against independent implementations. The aim is not to place every rule in one file; it is to prevent several files from silently becoming competing authorities.

5.5.3 Agents can move friction rather than remove it

Agent assistance can change which technology choices appear feasible. In a practitioner essay, Ronacher (2026) argues that agents reduce language-learning and rewrite friction enough to make performance-oriented languages and low-level systems work accessible to more developers, while also noting that prior knowledge improves the result. Treat this as a hypothesis about the capability of a human-agent pair, not evidence that the human has retained the same skill or that an unfamiliar stack is cheaper to own.

Lower entry friction can move cost into comprehension, debugging, security review, operations, and succession. Before adopting an unfamiliar stack, use a bounded spike to compare it with a familiar baseline under the same behavioral tests and explicit performance and resource targets; require a team member to explain and modify the result; and record the toolchain and rollback path. Choose the stack only when the measured benefit survives those ownership costs.

This cost can be visible even when final behavior is unchanged. Trivedi & Schmitt (2026) compared matched clean and deliberately degraded repositories across 660 agent trials. Cleanliness did not change the task pass rate, but agents working in cleaner repositories used 7–8% fewer tokens and revisited files 34% less often. The result does not show that every clean-code rule improves every task, but it supports a narrower claim: maintainability can affect the operational footprint of agentic development, not only human preference.

Compressed diffs and summaries can route attention, but a lossy view is not acceptance evidence. Keep the complete artifact and independent checks available; Appendix A develops a bounded audit for compressed review evidence.

5.6 Match feedback to the changed boundary

Validation can itself become a source of friction. A ten-minute clean-install or browser suite may provide valuable release evidence while being a poor response to every saved file. Conversely, a fast test that never crosses the changed boundary can create confidence without useful coverage.

For the category change, a focused validation-rule test can return the first failure quickly. An HTTP contract check can then cover serialization and compatibility; a database migration test can cover existing and new rows; a browser check can cover the filter interaction; and a deployed-boundary check can cover the actual cache and rollout behavior. Running the most expensive stage first wastes feedback time, while stopping at the focused stage leaves the web change unverified.

Agent review can keep producing candidate findings after the useful ones are exhausted. Count independently confirmed findings and the human time needed to adjudicate them, and stop against an explicit risk threshold; Appendix A develops the full review-evidence exercise.

For one representative change, map the project's feedback ladder. The table below separates the evidence needed during implementation from the more expensive evidence needed at a changed boundary or release decision.

Table 5.2. The project feedback ladder from focused checks to boundary-specific evidence.

Stage	Evidence returned	Typical cost	When it is required
Focused	One behavior or local invariant	Seconds	During implementation
Affected	Likely consequences of changed files	Seconds to minutes	Before sharing the change
Baseline	Ordinary project readiness	Minutes	Before accepting non-trivial work
Boundary-specific	Browser, clean install, workflow, performance, security, or mutation evidence	Potentially expensive	When that boundary changed or release confidence is required

The exact commands depend on the project. The important property is that each stage has a named purpose. Measure both time-to-first-useful-failure and total readiness time. If a slow check repeatedly catches problems that no earlier stage sees, consider whether a cheaper focused signal can move that feedback forward without weakening the final gate.

5.7 Audit the event-tracker workflow

5.7.1 Workflow loop lab

The demonstration below puts the work into a visible loop: human framing, implementation, tool feedback, review, and correction. It separates agent speed from workflow quality, because an agent can produce more output while still pushing uncertainty into human review.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/friction/

Use this first to make the human-in-the-loop process explicit. Then use the friction audit to ask which part of that process is creating repeated work.

5.7.2 Friction audit lab

The demonstration below treats friction as repeated work in a small change path. The goal is not to calculate exact minutes. The goal is to make students name where the resistance appears and choose one feedback loop to improve before adding more features.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/friction/

Apply the same audit to implementation A using the category change or another equivalently bounded request. Score setup, comprehension, change, testing, review, deployment, operations, and coordination; name the largest source of resistance; and choose one concrete improvement. For one implemented choice, compare introduction with safe reversal: mark its commitment point or record that none was reached, name what now depends on it, and identify the evidence that would show reversal succeeded.

Repeat the unchanged audit for implementation B. The comparison is useful only if the change request and categories remain stable. A different score may come from architecture, tooling, documentation, generated defaults, or team practice; the audit should identify the mechanism rather than attributing every difference to the implementation approach.

5.8 Project artifact

Students should produce a friction audit for implementations A and B. It should:

- trace one bounded web change from the browser-facing interaction through its contract, validation, data, derived view, cache, and deployment boundaries,
- separate protective friction from repeated work that provides no useful evidence or control,
- identify the first commitment point and explain what safe reversal requires after it,
- name the largest source of setup, comprehension, change, testing, review, deployment, operational, or coordination friction,
- map the project's validation ladder and identify one check that should move earlier, later, or become conditional,
- and improve one feedback loop that can be measured again without changing the audit request.

5.9 Summary

Development friction is the resistance encountered while setting up, understanding, changing, testing, reviewing, deploying, operating, and coordinating a web system. Following one category change from browser to production shows why apparently local work becomes cross-boundary work.

The documented Spotify, GitHub, Shopify, Stripe, and Slack cases show different responses: automate repeated setup, make environments replaceable, bring dependency evidence into review, stage committed data changes, and protect the feedback system under load. Their reported results belong to their own contexts, but the mechanisms provide questions that can be transferred to a smaller project.

Some friction protects important boundaries, while accidental friction delays useful work without adding evidence or control. Agents can reduce visible implementation effort while moving cost into comprehension, debugging, review, operations, or long-term ownership. A versioned environment, traceable responsibilities, visible commitment points, and a risk-based feedback ladder make a web application easier to change without weakening its safeguards.

5.10 References

1. Trivedi, P., & Schmitt, O. (2026). Does Code Cleanliness Affect Coding Agents? A Controlled Minimal-Pair Study. arXiv Preprint arXiv:2605.20049. https://doi.org/10.48550/arXiv.2605.20049
2. Ronacher, A. (2026, August). Fast and Hard Code. Personal technical essay. https://lucumr.pocoo.org/2026/8/22/fast-hard-code/
3. Qiu, D., Li, B., & Su, Z. (2013). An Empirical Analysis of the Co-Evolution of Schema and Code in Database Applications. Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering, 125–135. https://doi.org/10.1145/2491411.2491431

CHAPTER 6

Which web application assumptions fail under pressure?

A web application that works locally has shown that one implementation can satisfy one workload in one controlled environment. Production changes the environment: browser applications and integrations retry, networks delay or lose responses, live data accumulates, shared dependencies slow down, deployments change running systems, and operators have to recover without discarding authoritative state.

The central claim is that production failure exposes a mismatch between what an architecture assumed and what its operating environment actually did. The first task is therefore not to choose a scaling technique. It is to reconstruct the failure chain, identify the assumption that broke, find the earliest useful signal, and preserve a path back to useful service.

6.1 Local correctness leaves production assumptions untested

Production is not simply a larger version of a developer's laptop. It contains independently changing consumers, intermediaries, services, data stores, providers, and operational tools. A failure can cross several of these boundaries before a user sees an error.

Consider five assumptions that local development can easily hide:

- **Consumer behavior:** a browser, integration, or agent may retry after an ambiguous timeout or reconnect in a synchronized burst.
- **Network behavior:** a request can reach the authority even when its response never reaches the consumer.
- **Dependency behavior:** a slow identity, storage, payment, or messaging service can hold resources and spread waiting through otherwise healthy components.
- **Data behavior:** replicas can lag, writes can conflict, queues can age, and a restored endpoint can still return stale or incomplete state.
- **Operational behavior:** the dashboard, deployment system, credentials, status page, or rollback mechanism may depend on the service being repaired.

These are architectural assumptions because they determine where authority, waiting, retries, and recovery live. Testing the happy path proves none of them.

> **Sustainability angle**
>
> Production pressure often exposes waste that local development hides: retry storms, polling, failed jobs, duplicated events, abandoned data, and oversized logs. Backoff, deduplication, retention, and load shedding can reduce both failure amplification and unnecessary resource use.

6.2 What documented web incidents reveal

The four accounts below were published by the organizations that experienced the incidents. They provide concrete timelines, observations, and recovery choices, but they are not controlled experiments or independent audits. Treat each reported number as evidence about that system and use the transfer question to decide what, if anything, applies to the event tracker.

Table 6.1. Documented web incidents interpreted as failed assumptions and recovery questions.

Incident	Failure chain	Assumption exposed	First useful signal and recovery	Transfer question for the tracker
Discord authentication outage	Zonal maintenance, a degraded disk, cold row caches, limited capacity, timeouts, and retries compounded	Zonal redundancy and a successful short test provided enough margin to continue	HTTP success collapsed; downstream-service time and saturation led to load shedding while capacity returned	Which optional actions can stop before retries overwhelm a shared dependency?
GitHub database partition	A network partition triggered failover, leaving writes in two data-center histories	Automated database failover would produce a topology the application could safely use	Replication topology and faults exposed divergence; GitHub degraded features, failed forward, rebuilt, and drained queues	Which state must remain authoritative even if availability has to be reduced?
Fastly global outage	A valid customer configuration activated a latent software bug across the edge network	A valid configuration change could not trigger a platform-wide failure	Monitoring detected global disruption within one minute; disabling the triggering configuration restored most service	How small is the blast radius of one valid configuration or deployment change?
Cloudflare Workers KV incident	Deployment automation selected the wrong build; KV and tools depending on KV failed together	Tested rollback, authentication, and deployment tooling would remain usable during loss	Immediate alerting identified failure, but recovery required a break-glass route to the previous known-good build	Can the application be recovered when its ordinary control path is unavailable?

Read together, the cases expose four different failure mechanisms. Discord shows how retries can amplify reduced capacity; GitHub shows why protecting authoritative data may require prolonged degraded availability; Fastly shows how one valid configuration can acquire a global blast radius; and Cloudflare shows why recovery tooling cannot depend entirely on the service it must repair. The value lies in reconstructing those mechanisms from the linked sources rather than copying the companies' architectures or treating their remediations as universal patterns.

6.3 Trace one failure across the web boundary

The Architecture chapter separated consumer, representation, execution, and authority. A production failure becomes understandable when the same roles are traced over time rather than inspected as a static diagram.

Consider a deliberately small event-tracker scenario:

1. A browser sends `POST /events` without a stable idempotency key.
2. The origin validates the request and commits the event to the authoritative store.
3. The response is lost after the commit, so the browser sees a timeout.
4. Client application logic retries because it cannot tell whether the action completed.
5. Another application instance accepts the retry and records a second event.

6. A cached activity view briefly shows neither event and later shows both.
7. Request-success metrics look healthy even though one user intent produced duplicate durable state.

No individual observation explains the whole failure. The timeout is a consumer symptom, the lost response is a network event, the retry is a policy, the duplicate is an authority failure, and the stale view is a derived-state delay. Useful evidence has to connect them through a stable command identifier, request trace, commit result, cache status, and user-visible outcome.

The corrective design is similarly cross-boundary. The consumer reuses one idempotency key, the authority stores the mutation and reusable result atomically, and the interface makes a pending or confirmed outcome visible. Retries may then recover from transport ambiguity without inventing a second domain action.

6.4 From failed assumptions to useful signals

A system has a **competence envelope**: the range of conditions it can handle with its current resources, architecture, procedures, tools, and expertise. Ordinary disturbances move the system around inside that envelope. Saturation occurs when pressure reaches a binding limit and pushes the system to or beyond the boundary of what it can currently handle (Woods, 2018).

The limit is not necessarily a server at 100 percent CPU. It may be physical, such as memory or network bandwidth; deliberately imposed, such as a bounded queue, connection pool, quota, or rate limit; or organizational, such as responders unable to diagnose several interacting failures quickly enough. Software, infrastructure, operators, deployment paths, and operational tools all affect where the boundary lies (Hochstein, 2026).

A useful incident view needs several kinds of evidence:

- **User outcome:** success rate, error state, latency, duplication, stale data, or unavailable capability.
- **Binding limit:** CPU, disk, connection pool, database quorum, dependency quota, queue capacity, or responder attention.
- **Amplification:** retries, reconnects, fan-out, cache misses, synchronized jobs, or backlog replay.
- **Integrity:** authoritative write result, replication position, invariant violation, or reconciliation status.
- **Recovery:** rollback state, restored capacity, backlog age, replica lag, and whether user-visible behavior has settled.

This framing turns “Will it scale?” into four more useful questions:

1. Which limit is the pressure approaching?
2. Which signal reveals that the remaining margin is shrinking?
3. What failure or feedback loop begins at saturation?
4. Which prepared recovery actions can return the system to a useful state?

6.5 Connect failed assumptions to response patterns

Diagnosis comes before technique selection, but students still need a compact vocabulary for responding once a failed assumption and binding limit are visible. The patterns below are starting hypotheses rather than infrastructure that every implementation should acquire.

Pressure or failed assumption	Response pattern worth considering	New obligation and evidence
Writes and reads compete for one path	Separate the authoritative write path from derived read views	Define freshness, rebuild, and reconciliation; measure write completion and read usefulness
A burst contains work that need not finish synchronously	Queue or buffer bounded background work	Expose pending and terminal states, queue age, retry behavior, and worker saturation
A request may be repeated after an ambiguous result	Use a stable idempotency key and authoritative deduplication	Store the mutation and reusable result atomically; test a retry after a lost response
One tenant or domain instance consumes shared capacity	Partition, quota, or isolate work by a meaningful domain boundary	Define routing, fairness, cross-partition operations, and migration
Incoming work exceeds the rate at which it can be completed	Apply backpressure, admission control, rate limits, or load shedding	Make rejection explicit, bound waiting, guide retries, and preserve the essential path
Optional work threatens a core operation	Degrade a named capability while preserving the core	Define the degraded user state, dependency boundary, recovery trigger, and completion signal
History grows beyond the useful query or retention path	Add a measured index, derived summary, archival, or deletion path	Account for write cost, staleness, rebuild, retention policy, and restored-state checks

These responses can be combined, but each combination creates another operating contract. A queue without idempotency can repeat effects, partitioning without a suitable key can make ordinary queries expensive, and rate limiting without useful client feedback can turn controlled rejection into immediate retry pressure. Choose the smallest response that addresses the observed or credibly projected limit.

The Little's Law relationship introduced in the Benchmarking chapter can check one queue or in-flight boundary. If a stable event-processing stage contains an average of 120 waiting or active events and completes an average of 8 events per second, the implied average time in the stage is `W = L / λ = 120 / 8 = 15 seconds`. That calculation does not explain the delay, but it can expose incompatible measurements or show that a claimed user-visible completion time cannot fit the observed backlog and throughput. If arrivals and completions are not approximately balanced during the observation window, record the system as changing or overloaded instead of forcing the averages into the law.

For implementations A and B, select one queue, pool, request path, or other conserved-work boundary. State its boundary, estimate or measure average work in progress, completion rate, and average time in the system over one consistent window, then explain whether `L = λW` is approximately coherent and what the relationship leaves unexplained.

6.6 Recovery is part of the architecture

Recovery is not a generic command executed after diagnosis. The available response depends on what has already become externally visible or durably authoritative.

Table 6.2. Recovery modes and the preparation each requires before an incident.

Recovery mode	What it protects	Required preparation	Main risk

Recovery mode	What it protects	Required preparation	Main risk
Reject or shed load	Remaining capacity and core operations	Admission rules, retry guidance, and an explicit non-critical boundary	Rejected work may be retried immediately or lost silently
Degrade functionality	Data integrity or one essential user path	Features that can be paused independently and clear user states	Hidden dependency means the supposedly optional work is still required
Roll back	A known-good code or configuration path	Compatible state, retained artifact, health gate, and reachable control	Durable writes or migrations may make reversal unsafe
Fix forward	Accepted state that cannot safely be discarded	Reconciliation plan, bounded change, and strong observation	Recovery takes longer while the system remains degraded
Rebuild and replay	Authoritative history and deferred effects	Backups, logs, replayable work, ordering, and deduplication	Backlog load can trigger another saturation cycle
Break glass	Recovery when the ordinary control plane is unavailable	Independent access, narrow authority, audit, and regular testing	Emergency power can bypass normal safeguards

The cases show that these modes can be combined. Discord shed optional work while capacity returned; GitHub degraded features and failed forward to protect accepted writes; Fastly disabled a triggering configuration before deploying a permanent fix; Cloudflare used break-glass access to restore a known-good route.

Two boundaries deserve particular attention. First, recovery tooling should not depend entirely on the component it is meant to repair. Second, **service restored** and **recovery complete** are different states. Traffic may succeed while replicas lag, caches remain stale, queues grow, notifications wait, and user-visible inconsistencies still require reconciliation.

6.6.1 Pressure map lab

The incident challenge below turns changed assumptions into a pressure map over three rounds. Investigate one signal and commit one response before time advances. The consequences persist: a response may protect one boundary while moving waiting, cost, correctness risk, or operational work somewhere else.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/reality/

Try one path, inspect the debrief, and replay with a different response. Use the exported brief to connect the operating pressure to the stressed component, binding limit, likely failure, first signal, and prepared recovery. The values are deterministic teaching inputs rather than measured capacity, so the useful evidence is the causal explanation and not the final number.

6.7 Challenge assumptions without changing the common core

Implementation B should not be framed as “make the app bigger” or as an extension of implementation A. Apply the same production pressures to both independent implementations while keeping their required behavior stable:

- many concurrent tenants, projects, groups, locations, or other domain instances,
- tens or hundreds of thousands of users,
- high and bursty event volume,
- client retries and dependency failure,
- eventual correctness where immediate accuracy is unnecessary,
- and a deployment or configuration change that must be stopped or reversed.

Choose one documented case and reconstruct it as a chain of trigger, latent condition, propagation, user effect, first useful signal, recovery choice, and residual work. Then transfer only the structure of that chain to the event tracker. Do not claim that the project has reproduced the source company's scale, infrastructure, or result.

Students should explain which assumptions differ between implementations A and B and which remain shared. A good reconstruction makes failure and recovery obligations visible without trying to solve every possible future problem. The Scaling chapter can then choose techniques in response to demonstrated pressure rather than previewing a catalog here.

6.8 Project artifact

Students should produce a documented-case transfer and a pressure map for implementations A and B. The artifact should:

- link the selected source and separate its reported facts from the student's interpretation,
- trace one failure across consumer, network, execution, authority, and operational boundaries,
- name the assumption, binding limit, first useful signal, amplification path, and user-visible effect,
- apply Little's Law to one declared queue, pool, request path, or other conserved-work boundary and interpret its limits,
- connect the failed assumption to one justified response pattern and the new obligation that response would introduce,
- identify a prepared recovery mode and the evidence that would show recovery is complete,
- and explain what differs between the two implementations without changing the common acceptance core.

6.9 Summary

Production exposes architectural assumptions that a local happy path leaves untested. The Discord, GitHub, Fastly, and Cloudflare incidents show different forms of the same structure: a latent condition meets a trigger, propagation crosses boundaries, user-visible behavior changes, and recovery depends on preparation that existed before the incident.

Useful analysis connects consumer outcomes to the binding limit, amplification path, authoritative state, and recovery evidence. Retries can worsen saturation, automated failover can produce a topology the application cannot use safely, global configuration can create global blast radius, and rollback can

fail when its control path shares the failed dependency.

Reality asks what failed, how anyone knew, and how useful service could be recovered. Scaling follows by asking which architectural change is justified by that evidence.

6.10 References

1. Woods, D. D. (2018). The Theory of Graceful Extensibility: Basic Rules that Govern Adaptive Systems. Environment Systems and Decisions, 38(4), 433–457. https://doi.org/10.1007/s10669-018-9708-3
2. Hochstein, L. (2026, July). Saturation: How Your Software Will Fail at Scale. Software Should Work conference presentation. https://www.youtube.com/watch?v=PHYCRubnmSM

PART III

Compare — Evaluate workflows and scaling responses

Implementations A and B expose what changed across two single-agent development systems, but they do not yet show what orchestration adds or costs. Compare produces implementation C through a declared agent fleet, applies common scaling pressures to all three applications, and turns their evidence into a justified target architecture and development workflow.

III.1 From orchestration to an evidence-based decision

The three chapters form one progression:

1. When do agent fleets improve web development? grows one agent into a bounded fleet, makes authority and integration explicit, and treats failure correction as part of implementation C.
2. How should web applications respond to scaling pressure? identifies approaching limits, applies formal models, and compares possible responses without treating a technique catalog as a design.
3. What do three agent-assisted implementations teach us about web scalability? connects the application, development, and agentic loops and turns the three evidence packages into a final decision.

By the end of Compare, students should have implementation C and its fleet trace, a three-way application and workflow comparison, and a target design whose claims distinguish observation, inference, and proposal.

III.2 Implementation C — Work Activity Tracker

The Compare project specification defines the third fresh implementation and its bounded fleet workflow. The common behavior, permitted acceptance material, pressures, and reporting categories remain stable while delegation, orchestration, authority allocation, integration, and fleet controls vary deliberately. When practical, use the same foundation model as implementation B so orchestration is the primary changed variable; otherwise record the configuration difference and narrow the comparison claim.

Implementation C also turns its development process into application data. A transformed fleet trace is loaded into the Work Activity Tracker so its history, filters, and relationships can expose attempts, failures, handoffs, validation, correction, and human intervention. The result is a bounded workflow debugger, not a claim that the application has become a universal agent platform.

III.3 Compare applications and development systems

Scalability has two connected objects of analysis here: the web application and the development system that changes it. Application scalability remains the subject of the book; the three AI-assisted setups are the experimental lens. Compare what the tools made easier, what they made riskier, and what project structures made accepted work legible enough to review and maintain.

The course does not establish a universal ranking of models or workflows. The implementations can differ in architecture, the team learns between builds, and the served configurations may change. The shared specification and evidence make those effects discussable; they do not make every causal explanation identifiable.

CHAPTER 7

When do agent fleets improve web development?

An agent fleet is a coordinated set of independently operating agent executions working on parts of a larger task. The distinction matters. Writing four role descriptions into one prompt may help one agent reason, but it does not create four execution contexts, four tool users, or parallel work. A fleet connects several goal-directed observe–decide–act loops: one agent's artifact or message becomes another agent's observation, while an orchestration layer assigns work, routes results, and controls shared tools and state.

For web development, the attraction is clear. A feature often spans interaction design, browser behavior, an API contract, persistence, deployment, and verification. Those concerns create possible parallel work, but they also create seams where a locally reasonable change can break the application as a whole. The central claim of this chapter is therefore that more agents do not automatically mean more progress. Agentic development scales only when the work decomposes cleanly, the project teaches agents how to operate inside it, and integration and error correction remain explicit.

Unless stated otherwise, “fleet” here means part of the development system that produces a web application. An agent deployed inside the application is a runtime component with its own users, data, permissions, latency, evaluation, and operational obligations. The settings reuse concepts such as tools, traces, workflows, and access control, but they are not the same experiment.

> **Sustainability angle**
>
> Parallel agents multiply compute use. A fleet is easier to justify when decomposition improves quality, reduces rework, or saves substantial human time. Clear setup, fast tests, stable commands, useful errors, and scoped tasks make each model call more likely to produce reviewable value.

7.1 From one agent to a fleet

A fleet should be an answer to the shape of the work, not the starting assumption. Begin with the smallest coordination structure that can complete the task.

Table 7.1. Progression from a single coding agent to an integrated development fleet.

Stage	Structure	Useful when	New cost or risk
One agent	One context and action stream	The change is small, coupled, or depends on frequent interpretation	Limited concurrency; one interpretation can dominate
Sequential delegation	One agent hands a bounded result to the next	Later work genuinely depends on an earlier finding or decision	Handoff latency and accumulated misunderstanding
Parallel specialists	Independent agents work against a settled interface	Tasks touch different modules and have independently testable outputs	Duplicate exploration, conflicts, and a growing review queue
Integrated fleet	Orchestration, isolated workspaces, shared contracts, and gates	The workflow repeats often enough to justify explicit infrastructure	Control, observability, recovery, and integration become system work

The first useful expansion is often sequential rather than parallel. A read-only scout maps the relevant routes, components, data flow, and tests; an implementer then receives the compressed findings and a bounded objective. This keeps noisy reconnaissance out of the implementation context without pretending that dependent work can happen simultaneously. An implementer followed by a reviewer is another sequential pattern: the reviewer receives a concrete diff and validation evidence rather than guessing what might change.

Parallelism becomes reasonable only after a shared decision has settled the seam between tasks. For implementation C, one possible dependency graph is:

`reconnaissance → shared domain and API contract → interface work ∥ data work → integration → browser acceptance`

The interface and data branches may run together because both depend on the same contract. The contract itself, integration, and end-to-end acceptance remain sequential. If both branches need to reinterpret the lifecycle, edit the same schema, or wait for the same person, their apparent parallelism merely moves waiting and conflict downstream.

7.1.1 Amdahl's Law and the sequential bottleneck

Amdahl's Law gives a simple upper bound on speedup when only part of the work can run in parallel (Amdahl, 1967). If `p` is the parallelizable share and `N` agents execute that share, the idealized speedup is:

```
S(N) = 1 / ((1 − p) + p / N)
```

If half of a workflow is inherently sequential, unlimited agents cannot make the whole workflow more than twice as fast. In web development, framing the feature, settling a shared interface, integrating overlapping edits, reviewing risk, and accepting the browser-visible result often remain sequential. Real fleets perform below the ideal bound because communication, duplicated exploration, conflicts, and waiting add overhead that the formula does not model.

The lab below keeps the workflow at 100 units of work and changes only the parallelizable share and number of agents. Predict the effect of the next agent before moving the control, then compare the ideal speedup with the infinite-agent ceiling.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/agent-fleets/

Apply the calculation to one observed fleet task. Name the sequential work and keep coordination time visible rather than hiding it inside the parallel share. The difference between the ideal bound and the observed result is evidence about coordination cost, not proof that the law failed.

7.2 What documented web-development cases show

The shift is visible in web delivery. Vercel reported in April 2026 that coding agents initiated more than 30% of deployments on its platform, up 1,000% over six months. That is vendor telemetry from one platform rather than an industry-wide measurement, but it shows that agentic web development is no longer limited to editor experiments.

The four company-authored accounts below are more useful for examining how the work changes. Each describes its author's own tools and measurements; none is a controlled comparison showing that the result will transfer to the course project.

Table 7.2. Documented agentic-development cases and their transfer questions.

Documented case	Practice in the web-development workflow	Reported evidence and important limit	Transfer question
GitHub assigns Copilot work in github.com	Issues become proposed changes to UI, APIs, migrations, and production code	One month of task examples; no counts, review time, or defect rate	Which decisions may an agent execute, and who accepts the result?
Vercel teaches agents product design	Repository skills, linters, and review carry UI and interaction decisions forward	Skills went unused in 56% of separate Next.js eval cases; setup-specific	How does a web project make product judgment available and maintainable?
Stripe runs parallel Minions	Unattended agents produce pull requests in isolated development environments	More than 1,000 agent-written PRs merged weekly; no attempt or quality denominator	What infrastructure makes parallel work bounded and reviewable?
Stripe evaluates web integrations	Agents modify frontends, backends, databases, and live payment flows	Eleven realistic environments with UI and API graders; deliberately small suite	What evidence establishes that the resulting web behavior is correct?

Four lessons transfer without retelling each case.

First, **execution and acceptance are separate**. Assigning an issue to an agent changes the primary performer; it does not automatically transfer the authority to merge, release, or declare the result correct. Second, **making guidance available is not the same as applying it**. Repository instructions, product-design skills, examples, and linters can make hidden web conventions legible, but retrieval and correct application remain distinct failure modes. Third, **parallel activity is not accepted throughput**. Isolated environments and automated checks make more concurrent attempts possible, yet review, integration, and failed attempts still consume time and compute. Fourth, **web correctness has to be tested through the web boundary**. Component, API, and database checks matter, but they do not replace a browser journey that crosses the layers the user actually experiences.

Implementation C turns those lessons into a local comparison. Students build the common event-tracking application through a bounded fleet workflow, load the execution trace into the application, and compare the accepted outcome with a single-agent counterfactual.

7.3 Build a small web-development fleet

Pi is the primary teaching environment because its small core makes the orchestration visible. Its quickstart documents installation, authentication, project-level `AGENTS.md`, and the initial tool set. Pi's extension API can add tools and lifecycle controls, while the repository's reference subagent extension

demonstrates separate processes, isolated contexts, sequential chains, parallel tasks, and structured output. The reference is a starting point to inspect and adapt, not evidence that every project should use its exact role or security model.

Students can use another agent harness or hosted environment if they preserve the same task graph, evidence, and acceptance boundary. The tools below solve related but different problems, so compare their unit of delegation before comparing feature lists. Pi, Hermes, and Claude Code expose agent harnesses that may also coordinate subagents; their processes consume the repository's project harness. GitHub Copilot exposes a hosted issue-to-pull-request workflow, while the OpenAI Responses example concerns orchestration inside an application. The table classifies the layer exposed to the student rather than every component inside each product.

Table 7.3. Current paths for trying delegation; verify the linked documentation because these products evolve quickly.

Environment	Unit and coordination model	Isolation or control to inspect	Best fit in this chapter
Pi plus its reference subagent extension	Local agent processes in single, sequential-chain, or parallel modes	Project trust, extension code, agent tools, contexts, and process output	Primary hands-on agent-harness lab because students can see and modify the mechanism
Hermes Agent	Parent delegates fresh-context children singly or in parallel; nested delegation is opt-in	Inherited toolsets, separate terminals, optional worktrees, iteration and timeout limits	Broader multi-provider agent-harness comparison with built-in delegation and persistent memory
Claude Code	Subagents, parallel sessions, and experimental agent teams	Tool permissions, separate contexts, optional worktree isolation	Integrated agent-harness comparison when students want several built-in fleet patterns
GitHub Copilot cloud agent	Asynchronous repository task that researches, changes code, and opens a pull request	Repository policy, hosted environment, checks, and pull-request review	Issue-to-pull-request comparison rather than a transparent local fleet
OpenAI Responses multi-agent	A model coordinates parallel subagents and synthesizes their results	API tools, prompts, application policy, usage, and returned evidence	Application-level orchestration comparison rather than an agent-harness lab

Hermes is broader than a coding copilot: it includes persistent memory, reusable skills, remote terminal backends, and messaging entry points. For the course comparison, restrict it to the repository workflow, declare whether persistent memory was available, and enable worktree isolation for concurrent writers.

Production-agent frameworks are outside this table. They help teams deploy agent workflows inside applications, whereas the chapter's primary question is how agents build and verify a web application.

7.3.1 Prepare the project before adding agents

Install Pi following its current quickstart, authenticate with a provider, and start in a version-controlled project. Before adding subagents, ask one agent to inspect the repository:

> Inspect this project and propose an `AGENTS.md`; do not edit yet. Include the project purpose, setup and validation commands, coding conventions, architectural boundaries, forbidden operations, and review checklist. Cite the files that support each proposal and mark unknowns.

Review the proposal before saving it. An instruction file is maintained project policy, not an automatically authoritative summary. For a web project it should expose at least the development command, unit and browser-test commands, route and persistence locations, styling and accessibility

expectations, responsive states, secrets policy, and the definition of done.

Project memory needs a lifecycle. Keep unresolved questions and temporary findings in a working artifact; record consequential choices in a decision record; promote settled behavior into the relevant specification or architecture document; and connect it to executable checks. A discovery transcript should not silently become the specification, and parallel agents should not work from different generations of the same contract. Repeatable review or debugging workflows can become small project skills, while clear mechanical rules belong in linters or tests.

7.3.2 Define the web task graph

The fleet produces the complete common core of implementation C rather than four unrelated features. Settle the domain vocabulary, lifecycle transitions, event schema, API shapes, error semantics, and browser acceptance journey before starting parallel implementation. Then assign bounded work such as:

Table 7.4. A minimal fleet bound to cross-layer concerns in implementation C.

Task	Web-specific responsibility	Boundary and required output
Contract scout	Map routes, domain transitions, events, UI states, storage, and existing checks	Read-only; return file references, unknowns, and proposed contract decisions
Interface implementer	Build forms, loading, empty, success, and error states; preserve accessibility and responsive behavior	Own named UI files; return focused checks and screenshots or browser observations
Data implementer	Implement persistence, valid transitions, event atomicity, API responses, and idempotency	Own named server and data files; return API, migration, and invariant checks
Verifier and integrator	Integrate both branches and exercise the complete user journey	May repair only assigned integration defects; return browser evidence and limits

Role names are illustrative. Grade whether every task has an objective, allowed files and tools, expected output, validation commands, non-goals, owner, and stopping condition. If two roles must edit the same shared contract, resolve that decision before they branch or reduce the fleet. Use separate workspaces or worktrees for concurrent writers, and pin the same declared base model for the single-agent and fleet runs where possible. Record any substitution as a confounding factor.

7.3.3 Map authority and acceptance

Names such as `scout`, `implementer`, and `reviewer` allocate activity, not decision responsibility. Following the decision-path formulation of Nissilä (2026), trace the feature through:

> **Need → Define → Design → Implement → Verify → Release → Observe → Adapt**

For each consequential decision, distinguish the performer from the accountable owner and allocate authority as `propose`, `prepare`, or `execute`. The function-specific automation model of Parasuraman et al. (2000) is useful because gathering information, analyzing it, selecting an action, and executing it can receive different allocations inside one task. NIST similarly recommends defining roles, responsibilities, and oversight for human–AI configurations rather than assuming that the presence of a human creates effective control (Tabassi, 2023).

Table 7.5. Example authority and acceptance boundaries for a web-development fleet.

Decision	Agent authority	Acceptance evidence	Recovery or escalation	Accountable owner
Interpret an ambiguous requirement	Propose alternatives	User examples, affected behavior, assumptions, and unknowns	Human selects or reframes before implementation	Outcome owner
Change a shared interface	Prepare a contract and compatibility analysis	Consumer checks, migration implications, and rollback path	Escalate before a breaking or externally visible change	System owner
Implement a bounded change	Execute within named files, tools, and budget	Focused checks, diff, baseline gate, and known limits	Stop on scope expansion, failed invariant, or unavailable check	Integrating human
Accept the web behavior	Propose a verdict from the complete evidence	Browser journey, API and persistence checks, and reviewed failures	Reject, repair, or revert; do not infer acceptance from activity	Outcome owner

Coactive Design offers a compact test for this allocation. Each dependency needs enough **observability** of relevant state, **predictability** of likely behavior, and **directability** through correction, stopping, or reallocation (Johnson et al., 2014). A human approval box is ceremonial if the approver cannot see what happened, anticipate the next action, or recover from it.

Task control and resource access also remain separate. For one delegated task, trace:

`originating actor → fleet session → delegated task → model or tool attempt → validation → delegated-run outcome`

Propagate only the necessary identity, authority, task, trace, budget, and policy identifiers. An agent that reads untrusted issues should not automatically receive secrets or external write access; an implementer does not need deployment credentials; a publisher should receive a reviewed artifact rather than all private context. Treat imported extensions, skills, instructions, and configuration as executable project dependencies: inspect their source and requested access, pin a revision when practical, and use disposable least-privilege environments. Publishing an issue, patch, deployment, or message externally requires an explicitly accountable human who can explain the intent and evidence, respond to feedback, and correct or withdraw it. Appendix D develops the enforcement and traceability model.

7.4 Practice failure and correction

Small cross-layer mismatches make fleet risk easier to understand than abstract warnings. The goal of the labs is not to prove that four agents are better than one. It is to expose where decomposition, integration, validation, or correction fails.

7.4.1 Delegation lab

Use the planner below before running real agents. Choose a task and project-readiness level, then change the roles and model allocation. The generated packet is a rehearsal: if the task is vague, boundaries overlap, or the review burden exceeds available capacity, reduce the fleet or harden the project first.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/agent-fleets/

Translate the packet into a task graph rather than launching every named role automatically. Run three small patterns before the complete implementation: scout then plan, implement then review, and parallel scouts over independent subsystems. For each pattern, state why the tasks are sequential or independent and who integrates the result.

7.4.2 Correction lab

After the fleet integrates implementation C, select one seeded defect from the table below. The instructor can supply a faulty branch, or the student can introduce the fault after saving a known-good baseline. Do not reveal the chosen fault to the diagnosing agent.

Table 7.6. Small faults that expose fleet integration and correction problems.

Seeded defect	Why local work may look correct	Web-visible or state-visible failure
UI and API use different event or field names	Each side's focused tests use its own vocabulary	A user action appears successful but activity or summary does not update
An event is written before transition validation	The valid-transition test remains green	A rejected action still changes event history or derived counts
A retried mutation has no idempotency boundary	One request produces the expected record	A network retry duplicates an event or transition
Interface and data agents changed the same contract	Both branches pass before integration	The merged browser journey fails or displays a stale state
The verifier runs unit checks but no browser probe	Components, handlers, and queries pass independently	The complete user journey cannot finish

Use one correction loop throughout the lab:

> **delegate → integrate → observe → diagnose → correct → revalidate**

1. Run every focused check reported by the contributing agents and record which checks are green.
2. Exercise the application through its web boundary: create a workspace and work item, perform valid lifecycle transitions, confirm activity and summary, filter the feed, refresh to verify persistence, and attempt one invalid transition.
3. Verify that the invalid action changes neither current state nor event history.
4. Use the diff and fleet trace to locate the first divergence between the settled contract and observed behavior.
5. Assign one bounded correction task with an explicit hypothesis. Avoid launching several speculative fixers against the same files.
6. Run the focused regression check and the complete browser journey again. A correction is accepted only when both pass and the trace records the failed attempt and repair.

This lab makes error correction part of the architecture rather than an afterthought. A green unit check can be accurate within its boundary and still fail to protect the user journey. Likewise, agreement between repeated agent reviews is not independent validation: several runs can repeat the same blind spot. Preserve the fixed code and workflow configuration, deduplicate findings, and decide acceptance from behavioral evidence and accountable review. Appendix A develops the repeated-review method, while Appendix E models detection and repair across a reliability chain.

7.4.3 Measure accepted throughput

An agent run, branch, pull request, or generated file is activity. It becomes useful throughput only when the declared acceptance boundary accepts the result. Count failed runs, duplicated exploration, handoffs, merge conflicts, retries, total model use, elapsed time, active human integration time, and reviewer backlog. Fleet size should follow task independence and review capacity, not the number of available agents.

Coordination mechanisms can remove one bottleneck without removing the others. In 600 trials across six tasks, CodeCRDT achieved deterministic convergence and zero textual merge failures, yet reported results from a 21.1% speedup to a 39.4% slowdown and still observed semantic conflicts (Pugachev, 2025). The prototype does not establish a general fleet speedup, but it makes the distinction concrete: converged files are not necessarily compatible ideas, and conflict-free integration is not accepted web behavior.

Treat one end-to-end fleet attempt as one trial. The internal agents, retries, and handoffs are parts of that trial rather than independent samples. Compare it with one single-agent run over the same brief, base model, acceptance tests, and evidence schema. The result supports a local decision about this task; reliability claims require predeclared repeated trials.

Finally, transform the execution trace into implementation C. The fleet run maps to a workspace, delegated tasks to work items, agents and humans to actors, and attempts, validations, failures, reviews, and interventions to activity events. Use the application's history, filters, summaries, and relationships to answer two questions: where did coordination first fail, and which correction produced accepted behavior? A portable fleet-trace example provides a small transformation target, mapping guide, redaction record, and repeat-import contract. Adapt the actual tool export to that level of evidence rather than treating the example as a universal agent-tool schema. The goal is not to build a universal agent platform. It is to make the workflow that built the web application inspectable through the web application itself.

7.5 Project artifact

Produce implementation C and the bounded fleet workflow required by the Compare project specification. The artifact should make delegation, authority, integration, validation, and correction inspectable rather than merely showing that several agents ran. Include:

- the declared base model, agent harness, and relevant execution configuration,
- the task graph, roles, boundaries, dependencies, and expected outputs,
- the authority map, human approval points, and accountable integration owner,
- validation commands, review criteria, and the stopping or reduction conditions for the fleet,
- one deterministic workflow probe and measurements comparable with the earlier implementations,
- and a transformed fleet trace loaded into implementation C, together with debugging findings that identify where coordination failed and which correction produced accepted web behavior.

Preserve consequential changes to the fleet plan and the evidence behind acceptance. The fleet configuration and trace are part of the submitted development system, not background process trivia.

7.6 Summary

Agent fleets grow usefully from one agent only when the task contains real independent work. Documented web-development cases show the enabling pattern: legible project guidance, isolated execution, explicit acceptance, and end-to-end evidence. Pi provides a transparent path for building a small fleet, while the alternatives demonstrate that local agent harnesses, hosted pull-request agents, and application-level orchestration should not be treated as interchangeable. Implementation C binds delegation to interface, API, persistence, and browser concerns, then makes students repair a small cross-layer fault. Judge the fleet by accepted web behavior, correction quality, latency, cost, coordination, and human review effort—not by how many agents appeared busy.

7.7 References

1. Amdahl, G. M. (1967). Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. Proceedings of the April 18–20, 1967, Spring Joint Computer Conference, 483–485. https://doi.org/10.1145/1465482.1465560
2. Pugachev, S. (2025). CodeCRDT: Observation-Driven Coordination for Multi-Agent LLM Code Generation. arXiv Preprint arXiv:2510.18893. https://doi.org/10.48550/arXiv.2510.18893
3. Parasuraman, R., Sheridan, T. B., & Wickens, C. D. (2000). A Model for Types and Levels of Human Interaction with Automation. IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans, 30(3), 286–297. https://doi.org/10.1109/3468.844354
4. Johnson, M., Bradshaw, J. M., Feltovich, P. J., Jonker, C. M., van Riemsdijk, M. B., & Sierhuis, M. (2014). Coactive Design: Designing Support for Interdependence in Joint Activity. Journal of Human-Robot Interaction, 3(1), 43–69. https://doi.org/10.5898/JHRI.3.1.Johnson
5. Tabassi, E. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0) (NIST AI Nos. 100–1). National Institute of Standards. https://doi.org/10.6028/NIST.AI.100-1
6. Nissilä, T. (2026, August). Päätöspolku on ohjelmistokehityksen uusi organisaatiokaavio. LinkedIn article in the HAR: Human Agent Relationship newsletter. https://www.linkedin.com/pulse/p%C3%A4%C3%A4t%C3%B6spolku-ohjelmistokehityksen-uusi-tapio-nissil%C3%A4-0ke2f/

CHAPTER 8

How should web applications respond to scaling pressure?

Scaling is the work of changing a system so it can tolerate growth in users, data, traffic, teams, or change.

The central claim is that scaling is not one move. It is a sequence of trade-offs made in response to specific pressure.

8.1 Start with the pressure

Before choosing a technique, ask what problem is being solved:

- Are writes too slow?
- Are reads too slow?
- Is the database too large?
- Is one tenant affecting another?
- Is the system failing during traffic spikes?
- Are deployments too risky?
- Are teams blocked by shared ownership?

8.1.1 Scaling pressure lab

The demonstration below starts from a concrete event-tracking pressure: event writes spike, dashboard reads get slower, and the latest-events query has to stay useful without pretending the system needs every scaling technique at once.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/scaling/

The important part is the exported trade-off table. Students should be able to explain why a chosen technique fits the pressure, what it costs, what failure mode it introduces, and which signal would show whether the design is working.

8.2 Name the saturation boundary

Scaling pressure becomes operationally important when it approaches a limit. The competence-envelope model introduced in the Challenge part calls the boundary condition **saturation**: the system can no longer absorb the disturbance with its current capabilities (Woods, 2018). Naming the saturated resource prevents "add more servers" from becoming the default response. The table below groups common limits by the evidence that reveals their approach.

Table 8.1. Categories of saturation limits and their observable signals.

Limit category	Examples	What approaching saturation may look like
Physical resources	CPU, memory, disk space, storage I/O, network bandwidth	Growing wait time, swapping, failed allocations, full disks, or reduced throughput
Virtual resources	Connection and thread pools, file descriptors, bounded queues, quotas, rate limits	Pool wait, rejected work, `429` responses, queue growth, or quota errors
Dependency capacity	Databases, identity providers, third-party APIs, provisioning systems	Timeouts, blocked callers, correlated downstream errors, or delayed scaling
Operational capacity	On-call attention, deployment speed, diagnostic access, coordination	Slow decisions, unsafe workarounds, competing responders, or recovery blocked by normal process

Saturation also creates feedback loops. Retries can amplify load, synchronous fan-out can overload an internal service, and blocked threads can make a CPU-based autoscaler misread demand. A resource leak may separate the causal change from the eventual failure by hours or months. These dynamics explain why a locally sensible mechanism can worsen a system-wide incident and why the pressure map must identify both the first limit and what happens after it is reached (Hochstein, 2026).

8.2.1 Saturation boundary lab

The model below compares one effective event rate with four declared stage limits. Increase demand until a boundary becomes binding, expand that limit, and observe which constraint becomes visible next. Then enable backpressure and compare hidden backlog growth with explicit rejection.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/scaling/

Treat the rates as a reasoning aid rather than measured capacity. The lesson is structural: removing one bottleneck changes the system's limiting factor, while retries can consume headroom before useful demand changes.

8.3 Use a formal model before choosing a technique

A scaling technique should respond to an identified limit. Formal models help students state that limit precisely enough to challenge their own intuition. The table below states four useful laws explicitly and separates what each can support from what it cannot establish.

Table 8.2. Formal scalability models, their assumptions, and suitable application boundaries.

Law	Exact relationship	Main assumptions	Useful when	Cannot establish
Little's Law (Little, 1961)	`L = λW`, where `L` is average work in progress, `λ` is average throughput or arrival rate, and `W` is average time in the system	Stable long-run averages, one consistent system boundary, and conserved work	Relating queue or review backlog, throughput, and elapsed time	The cause of delay, tail latency, or stability from one snapshot
Amdahl's Law (Amdahl, 1967)	`S(N) = 1 / ((1 − p) + p/N)`, where `p` is the parallelizable fraction and `N` is the parallel capacity	Fixed workload and fractions, with no added parallel overhead in the bound	Bounding speedup for request paths, batch work, deployments, or agent workflows	Contention, coordination cost, load growth, or the actual parallel fraction

Law	Exact relationship	Main assumptions	Useful when	Cannot establish
Lusser's Law (Tait, 1995)	`R_system = ∏ R_i` for independent components that must all succeed in a series system	Every named step is critical and step failures are independent	Checking whether a request, deployment, or agent workflow can meet an end-to-end reliability target	Correlated failures, recovery behavior, or omitted dependency paths
Universal Scalability Law (Gunther, 2008)	`C(N) = N / (1 + α(N − 1) + βN(N − 1))`, where `α` models contention and `β` coordination or coherency cost	One workload and configuration, with parameters fitted to representative throughput measurements	Describing capacity that rises, flattens, and then falls as concurrency grows	The causal mechanism or safe extrapolation beyond the measured range

The project does not require every model. Students must apply at least one suitable law or formal model to an application-architecture decision and one to the development workflow. Each application should identify the variables, state the assumptions, show the calculation or reasoning, connect the result to a measured or proposed limit, and explain what the model leaves out. A formula added after the design decision is not evidence that the formula guided the decision.

8.3.1 Contention beyond Amdahl's bound

Amdahl's Law models a fixed sequential share but does not describe shared resources and coordination becoming more expensive as concurrency grows. The Universal Scalability Law adds contention and coherency terms, allowing modeled capacity to rise, flatten, and eventually fall. Its parameters should be fitted from representative throughput measurements rather than selected by intuition.

Appendix E develops the model, provides the interactive contention curve, and examines a production case in which limiting admitted concurrency protected useful throughput.

8.4 Three documented cases of web scaling pressure

Scaling techniques become useful in combinations rather than as isolated vocabulary. The cases below begin with published production accounts from GitHub, Spotify, and Shopify. They provide observed pressures, interventions, and outcomes that a fictional scenario cannot. They are still company-authored retrospectives rather than controlled experiments: they report what each organization measured in its own system, and several changes may have occurred together. Treat their reported outcomes as evidence about those systems, not as performance promises for another application.

Read each case through the same questions:

1. What did users experience?
2. Which resource or boundary approached saturation?
3. What evidence exposed that boundary?
4. What did the organization change?
5. What improved, and what new trade-off appeared?
6. Which part transfers to the event-tracking project, and which part does not?

The table below previews the evidence and the transfer question in each case.

Table 8.3. Documented production pressures and the questions they raise for the project.

Production case	Observed pressure	Intervention described by the source	Reported evidence	Transfer question for the tracker
GitHub Issues navigation	Repeated page visits still paid network and rendering costs	Persistent client cache, stale-while-revalidate, controlled preheating	More React navigations became instant, with measured cache divergence	Which repeated read can tolerate controlled staleness?
Spotify podcast-video ingestion	Transcoding capacity saturated and queued publication work aged	More capacity, work prioritization, earlier alerts, planned backpressure	Publication moved from minutes to hours during the incident	What does acceptance mean, and how will users see pending work?
Shopify flash-sale request processing	Slow requests and browser refreshes consumed application-worker time	Detect and discard requests whose clients had already disconnected	Order-of-magnitude spikes, full request queues, and `499` responses exposed waste	Which work should be refused, cancelled, or shed before adding hosts?

> **Sustainability angle**
>
> Scaling techniques have resource profiles. Caches, queues, replicas, CDNs, shards, and multi-region deployments can improve user experience, but each adds infrastructure and operational cost. Ask which pressure justifies the extra resource use.

8.5 Orchestration maintains declared capacity

Horizontal scaling is not merely the instruction to "run more servers". An orchestrator repeatedly compares a declared desired state with the state it observes and takes action to close the gap. A Deployment can declare a replica count and pod template, while a Service gives clients a stable route as individual pods change.

8.5.1 Orchestration control-loop lab

The lab below runs Webernetes, a partial browser-based port of Kubernetes intended for interactive teaching. Change the desired replica count, send requests through a service, terminate a pod, and perform a rolling update. Watch for the distinction between the operator's instruction, the temporary mismatch, and the controller action that restores the declared state.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/scaling/

This is a mechanism demonstration, not a capacity benchmark or a production Kubernetes environment. The browser runs compatible control logic and a simulated network, but it does not execute real container images or model persistent volumes, resource limits, autoscaling, and the full production networking stack. Ask which component noticed the mismatch, what action it took, and which scaling problem remains unsolved.

Replica orchestration cannot make every component horizontally scalable. If each application instance depends on one saturated database, holds incompatible local state, or performs the same expensive downstream work, adding pods may move or amplify the bottleneck. Treat the replica count as a response hypothesis and measure whether useful capacity or availability improves.

8.6 Latency and geography

Geography becomes a scaling pressure when physical distance, regional failure, data residency, or a geographically distributed user base approaches an application requirement. The fastest handler still waits for network propagation and every sequential round trip consumes more of the latency budget.

Static assets and public cacheable responses can often move closer to users through a CDN or edge cache without moving the authoritative write path. Personalized data and mutations are harder: executing code at the edge does not remove the distance to a single authoritative database. Moving or replicating that authority introduces routing, failover, consistency, conflict, and residency decisions. Measure which bytes and round trips dominate before treating multi-region deployment as the inevitable final stage of scaling.

8.6.1 Partitions force an operational choice

The CAP result is narrower than a general slogan about balancing consistency and availability. For a distributed read-write object in an asynchronous network, a network partition can prevent a design from guaranteeing both atomic consistency and an available response from every non-failing node (Gilbert & Lynch, 2002). Here **atomic consistency** means operations appear in one current order, while **availability** means every request to a non-failing node eventually receives a non-error response. The partition is the condition that makes those guarantees collide; it is not a feature that an application can disable after communication has already failed.

A system that requires a majority quorum may preserve one history by rejecting or delaying a request that reaches an isolated minority. A system that accepts writes on both sides may keep responding, but the replicas can diverge and later need domain-specific reconciliation. Neither label chooses the right policy on its own. The decision depends on which operations may be stale, rejected, delayed, merged, compensated, or repaired without violating the domain.

CAP partition lab

Split one region from a two-region majority and apply the same two work-item transitions under both policies. Inspect the replies before healing the network: reconciliation can change stored state, but it cannot retract a success that a user already observed.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/scaling/

The lab models one write invariant and a fixed topology. Real systems need explicit definitions for reads as well as writes, failure detection and timeouts, quorum membership, leases, conflict resolution, and the consistency guarantee exposed to clients.

In some project themes, geography may matter very little. In others, a platform serving users or devices in different regions may need to consider latency, data residency, or regional failure. This is a good place to teach restraint: not every project needs multi-region architecture.

8.6.2 Latency budget lab

The demonstration below treats latency as a budget. Some of the budget is bounded by physics: a signal cannot cross distance instantly, and real networks add routing and protocol overhead. Some of the budget is architectural: every sequential database round trip adds another wait.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/scaling/

The point is not to memorize exact numbers. Real latency depends on providers, routing, congestion, TLS, database load, and query plans. The useful lesson is the shape of the problem: distance creates a lower bound, and repeated sequential database access multiplies that bound.

8.7 Trade-offs

> **Simplicity makes deficiencies visible**
>
> "I conclude that there are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies and the other way is to make it so complicated that there are no obvious deficiencies. The first method is far more difficult." (Hoare, 1981), p. 81
>
> Hoare was writing about overambitious programming-language design, not distributed systems. The contrast is about inspectability, not proof that a simple design is defect-free. Applied to scaling, it becomes a review test: **what pressure justifies each added mechanism, what new failure mode does it introduce, and why is the simpler design insufficient?**

Important trade-offs to make explicit:

- Simplicity vs capacity.
- Consistency vs availability.
- Real-time accuracy vs eventual correctness.
- Low latency vs operational complexity.
- Shared infrastructure vs tenant isolation.
- Fast feature delivery vs strict contracts.
- Automation vs human control.

> **The waterbed effect**
>
> The waterbed theory of complexity uses a physical metaphor: pushing complexity down in one place may make it rise somewhere else rather than remove it from the system (Randal et al., 2003). In a scaling design, caching moves repeated computation into invalidation and freshness decisions; queues move request latency into backlog, retries, idempotency, and delayed correctness; and partitioning moves shared-resource pressure into routing, migration, and cross-partition queries.

> This is not a formal law that every simplification must preserve a fixed quantity of complexity. It is a review question that complements the distinction between essential and accidental difficulty (Brooks, 1987): **what became simpler, and where did the displaced complexity reappear?**

> **Sustainability angle**
>
> Sustainability is also a trade-off. Keeping raw data forever, precomputing every possible summary, or running capacity for rare peaks can be reasonable in some domains, but the decision should be named and justified.

8.8 Implement the smallest justified response

A target diagram shows that a group can reason about a possible architecture. It does not yet show that the response works in the selected application, preserves required behavior, or moves the expected limit. Complete one bounded implementation experiment before finalizing the target design.

Select implementation A, B, or C and one pressure for which the group has runtime evidence or a credibly projected limit that the available environment can exercise. Preserve the accepted revision and its workload result, then implement the smallest response that addresses that limit. Suitable experiments include a measured query index, a bounded cache, an idempotency boundary, admission control, a queue with an explicit pending state, tenant isolation, or another technique justified by the evidence. Distributed infrastructure is not required.

Keep the workload, fixture, correctness checks, and measurement procedure unchanged where the response is intended to affect them. After the change:

1. rerun the common acceptance scenarios and inspect for behavior displaced by the response;
2. rerun the selected pressure and report every repetition rather than only the best result;
3. compare latency, throughput, errors, correctness, and the chosen resource-efficiency proxy before and after;
4. exercise one failure or recovery obligation introduced by the response, such as stale data, a duplicate retry, a full queue, a rejected request, or an unavailable worker;
5. state whether the result supported, challenged, or left the original hypothesis unresolved.

An unsuccessful intervention remains useful evidence when the baseline, change, and failure are preserved. Do not replace a negative result silently with a more elaborate design. The target architecture can retain the response, revise it, or reject it, but it should distinguish this implemented evidence from its remaining proposals.

8.9 Prepare to be surprised

Design can expand the competence envelope, but finite systems always retain another boundary. Graceful degradation keeps selected functions useful when capacity is constrained. **Graceful extensibility** is a complementary idea: when an unanticipated condition exceeds the designed

envelope, the sociotechnical system can adapt its capabilities and recover (Woods, 2018).

That ability requires optionality prepared before the incident:

- dynamic configuration and feature flags that expose understood controls;
- fast, practiced paths to restart, scale, shed work, roll back, or fix forward;
- diagnostic access and telemetry that remain useful under pressure;
- people who understand the system and have authority to act;
- exercises that keep rarely used recovery paths discoverable and safe.

More controls are not automatically better. An undocumented switch or unsafe manual path creates another failure mode. The goal is to balance work that prevents known failures with work that improves the ability to respond when the design encounters a surprise.

8.10 Project artifact

Students should produce a three-way scaling comparison, one implemented and measured scaling response, and one justified target design. The comparison should apply the same pressure set to implementations A, B, and C and distinguish what is observed in running code from what is proposed. The implemented response should preserve its before-and-after revisions, unchanged workload and acceptance evidence, observed result, new failure or recovery obligation, and conclusion. For at least one pressure, connect the proposed response to a documented production account. Separate what the source directly reports from the student's interpretation and from the transfer proposed for the project; do not assume that the source's scale, result, or accepted trade-off carries over. It should include one formal model applied to an application decision and one applied to the development workflow. Each row in the target design's trade-off table should name a pressure, the limit being approached, the chosen technique, the benefit, the cost, the new failure mode, the operational signal to watch, at least one prepared recovery action, and the reason a simpler alternative was not enough. It should also state where the technique displaced complexity, such as into clients, data freshness, background operations, routing, migration, or incident response. If the target design contains asynchronous multi-step work that can perform an external side effect, include one crash-after-side-effect audit. Record the last durable coordination fact, the external effect that may already have occurred, the timeout and retry rule, the retry-safety or deduplication mechanism and its owner, any stable key it uses, the terminal-state behavior, and the evidence available to an operator. If no component at the effect boundary can deduplicate, state that duplicate delivery remains possible.

8.11 Summary

Scaling begins with a pressure and an approaching limit, not with a preferred technique. Formal models can expose bounds and inconsistencies, while measurements show which boundary the running system is actually approaching. The linked production cases make those relationships concrete through controlled cache staleness, queued publication work, and the removal of abandoned requests. None of those responses transfers automatically; each begins a question that the project must answer with its

own workload and evidence. Orchestration can maintain declared application capacity, but it cannot make a saturated dependency or incompatible local state scale horizontally. Caching, queues, replicas, partitioning, rate limits, background work, and isolation can create capacity or resilience only by accepting new costs, failure modes, and operational obligations. Latency, geography, consistency, sustainability, and human coordination therefore belong in the same trade-off analysis as throughput. A credible target design explains why each mechanism is needed, where it moves complexity, which signal reveals trouble, and how the system can degrade, recover, or adapt when its assumptions fail.

8.12 References

1. Little, J. D. C. (1961). A Proof for the Queuing Formula: L = λW. Operations Research, 9(3), 383–387. https://doi.org/10.1287/opre.9.3.383
2. Amdahl, G. M. (1967). Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. Proceedings of the April 18–20, 1967, Spring Joint Computer Conference, 483–485. https://doi.org/10.1145/1465482.1465560
3. Tait, N. R. S. (1995). Robert Lusser and Lusser's Law. Safety and Reliability, 15(2), 15–18. https://doi.org/10.1080/09617353.1995.11690648
4. Gunther, N. J. (2008). A General Theory of Computational Scalability Based on Rational Functions. CoRR, abs/0808.1431. https://arxiv.org/abs/0808.1431
5. Woods, D. D. (2018). The Theory of Graceful Extensibility: Basic Rules that Govern Adaptive Systems. Environment Systems and Decisions, 38(4), 433–457. https://doi.org/10.1007/s10669-018-9708-3
6. Hochstein, L. (2026, July). Saturation: How Your Software Will Fail at Scale. Software Should Work conference presentation. https://www.youtube.com/watch?v=PHYCRubnmSM
7. Randal, A., Sugalski, D., & Tötsch, L. (2003). Perl 6 Essentials. O'Reilly Media. https://www.oreilly.com/library/view/perl-6-essentials/0596004990/
8. Brooks, Jr., Frederick P. (1987). No Silver Bullet—Essence and Accidents of Software Engineering. Computer, 20(4), 10–19. https://doi.org/10.1109/MC.1987.1663532
9. Hoare, C. A. R. (1981). The Emperor's Old Clothes. Communications of the ACM, 24(2), 75–83. https://doi.org/10.1145/358549.358561
10. Gilbert, S., & Lynch, N. A. (2002). Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. SIGACT News, 33(2), 51–59. https://doi.org/10.1145/564585.564601

CHAPTER 9

What do three agent-assisted implementations teach us about web scalability?

Scalability is the ability to keep a system useful while the conditions around it change. Technical capacity is part of that ability, but usefulness also depends on architecture, feedback, development friction, operational judgment, and the way human and agent work is governed. A web application that serves more requests but becomes unsafe to change is only partially scalable. So is a development workflow that generates changes faster than the team can understand, validate, or correct them.

The concepts in this book form a connected system rather than a checklist of techniques. Growth challenges assumptions; architecture determines where pressure is felt; measurement makes the resulting behavior visible; people and agents choose a response; and the response changes both the application and the cost of its next change. The three course implementations make that system concrete enough to compare.

> **Sustainability angle**
>
> A scalable system should not merely survive growth by consuming more computation, storage, bandwidth, and human attention. The final comparison should ask which additional resources produced useful capacity or learning and which merely displaced waste into infrastructure, model use, review, or maintenance.

9.1 One system, three connected loops

The course concepts can be organized into three feedback loops. Each loop preserves a different form of usefulness, but no loop operates independently.

Table 9.1. Three connected feedback loops for reasoning about scalable web development.

Loop	Recurring sequence	What can accumulate	Evidence that closes the loop
Application scalability	Pressure → challenged assumption → saturation boundary → signal → response → new trade-off	Load, data, latency, failures, and operational complexity	Browser behavior, benchmarks, metrics, traces, and recovery results
Development scalability	Intent → implementation → validation → accepted change → project memory → next change	Knowledge, coupling, technical debt, and friction	Tests, setup results, review findings, change effort, and decision records
Agentic development	Authority allocation → agent execution → trace → acceptance or rejection → correction → next delegation	Attempts, handoffs, review demand, model use, and trust	Workflow traces, interventions, accepted outcomes, and correction evidence

The loops share several concepts. **Web architecture** supplies the boundaries across which requests, data, ownership, and failures travel. **Measurement and observability** turn behavior at those boundaries into evidence. **Scalability models** help estimate which limit matters and how much

improvement is theoretically available. **Friction and technical debt** describe how earlier choices affect the cost of the next response. **Human–agent relationships** determine who may interpret evidence, choose an action, execute it, and accept the result.

The connections are cyclical. A scaling response becomes a development task. The development system determines whether people and agents can implement that task safely. Their accepted changes alter the application's architecture, capacity, and next likely failure. New observations then begin another pass through the loops.

9.2 The application loop: respond to pressure

Application scalability begins with a change in conditions rather than a preferred technology. More users, events, tenants, failures, stakeholders, or independently evolving clients challenge an assumption in the current design. The assumption may concern request rate, data size, retry behavior, ownership, consistency, or the ability of one component to wait for another.

Web architecture determines where the pressure becomes visible. A browser request crosses the client, network, API, domain logic, storage, and possibly a queue or external service. Each boundary defines a contract and a possible wait, failure, retry, compatibility problem, or trust transition. When demand approaches a binding resource or organizational limit, the system reaches a saturation boundary. That boundary may be CPU, a connection pool, queue capacity, third-party quota, deployment path, or the attention available for diagnosis.

Measurement closes the gap between a plausible story and an engineering claim. A browser journey establishes whether the user-visible behavior still works. A benchmark locates latency or throughput under a declared workload. Metrics show a changing rate or backlog, while traces connect symptoms to the request path that produced them. The evidence should answer a specific question: which assumption failed, at which boundary, under what conditions?

Only then does a scaling technique become a reasoned response. An index, cache, queue, replica, rate limit, partition, or asynchronous worker relieves a particular pressure while introducing new consistency, freshness, failure, cost, or operational obligations. Formal models help bound expectations: Amdahl's Law exposes sequential work, queueing models expose waiting, and contention models show why adding capacity may eventually produce diminishing or negative returns. The response completes the loop by creating a new system with new assumptions.

Use these questions to trace the application loop in one implementation:

- Which change in conditions challenged a specific architectural assumption?
- Which signal identified the first binding boundary rather than a downstream symptom?
- Which response addressed that boundary, and what new failure mode or operating cost did it introduce?

9.3 The development loop: preserve the ability to change

Every application response has to pass through a development system. An intention becomes code, configuration, schema, tests, documentation, and deployment behavior. Validation then determines whether the change satisfies the user-visible contract and remains compatible with the surrounding system. Accepted work changes both the application and the knowledge future contributors need.

This is where project legibility and friction connect to scalability. Stable commands, realistic fixtures, useful errors, bounded modules, explicit API contracts, architecture notes, and fast feedback make the next change easier to understand and verify. Hidden assumptions, tangled responsibilities, slow tests, stale guidance, and unclear ownership make it harder. Technical debt is therefore not only untidy code; it is reduced optionality and a higher future cost of coordinated change.

Web boundaries make development friction concrete. Adding one required event field may require a form, client validation, request schema, domain rule, database migration, activity rendering, and compatibility test. A clean contract helps those parts evolve together without forcing them into one module. A blurred contract makes a small feature cross many unrelated files and reviewers.

Feedback can turn repeated friction into project memory. A review comment may become maintained guidance, a common defect may become a regression test, a risky dependency may gain an adapter, and an architectural choice may become a decision record. That conversion is not automatic: transcripts and generated explanations are candidate evidence until someone verifies and promotes them. If the lesson remains only in one person's memory or one agent session, the loop does not improve the project for its next contributor.

Use these questions to trace the development loop:

- Which project property made a later change easier or harder to implement and review?
- Which validation boundary caught an error, and which important behavior remained invisible?
- What knowledge from the change became durable project guidance, an executable check, or a clarified contract?

9.4 The agentic loop: turn execution into accepted change

Agentic development changes who performs parts of the development loop, but it does not remove the loop. Implementation A and B use broadly single-agent workflows with different model capability; implementation C changes the orchestration structure by introducing a fleet. In every case, the important boundary is not generated output but accepted change.

Before execution, the human–agent relationship allocates authority. An agent may gather information, propose an interpretation, prepare a change, or execute within a bounded scope. The task should identify its objective, context, allowed files and tools, expected evidence, non-goals, and stopping or escalation condition. Project readiness affects whether the agent can follow that allocation: explicit commands, maintained contracts, and observable feedback reduce dependence on hidden human context.

Execution produces more than a final diff. The trace can show exploration, tool calls, failed checks, retries, delegated work, review, and human intervention. Those events explain whether a fast result came from clean decomposition, repeated attempts, transferred review effort, or an agent exceeding its intended authority. A branch, pull request, or agent message remains activity until the declared tests and accountable reviewer accept the behavior.

Correction completes the loop. A useful workflow can observe a divergence, diagnose it, assign a bounded repair, and revalidate the whole user journey. Repeated agent agreement is not a substitute for this evidence because several agents may share the same blind spot. Likewise, adding agents helps only when tasks are independent enough to outweigh handoffs, integration, and reviewer backpressure.

Use these questions to trace the agentic loop:

- Which decisions could the agent propose, prepare, or execute, and who accepted the consequences?
- What did the workflow trace reveal that the finished repository did not?
- Did orchestration reduce accepted completion time or merely move effort into integration, review, and correction?

9.5 How the loops affect one another

Consider adding a queue to protect event ingestion during a traffic spike. The three-loop view prevents the queue from appearing as an isolated scaling technique.

Table 9.2. One scaling response interpreted through all three loops.

Perspective	Consequence of the queue
Application loop	Burst absorption improves, but freshness lag, backlog growth, duplicates, and worker saturation become possible
Development loop	The project gains worker code, retry and idempotency rules, new local setup, deployment, tests, and ownership
Agentic loop	Tasks may decompose around producer, consumer, and verification boundaries, but their shared event contract must settle first
Evidence connecting all three	Queue age, end-to-end event visibility, duplicate rate, failure recovery, review effort, and trace history show whether the response worked

The same reasoning applies to caching, partitioning, background jobs, browser-side state, or a new external service. A locally faster application can become harder to operate or change. A highly legible project can improve human and agent work without increasing runtime capacity. A larger fleet can accelerate independent changes while saturating integration and review. Scalability is the judgment needed to see these coupled effects before optimizing one visible metric.

9.6 Apply the map to the three implementations

The three projects give the course a controlled comparison as far as classroom conditions allow. Required behavior and submission evidence remain as stable as practical while the model and orchestration setup change.

Table 9.3. The three course implementations and the evidence each comparison contributes.

Build	Agent setup	Held as stable as practical	Deliberately varied	Evidence goal
Establish — implementation A	Locally hosted single agent	Common functional scope and submission evidence	Local model, runtime, and the team's first project harness	Establish a baseline and expose initial assumptions
Challenge — implementation B	Foundation-model single agent	The same required behavior, built afresh	Served model capability and the revised project harness or architecture hypothesis	Identify which outcomes recur and which change under a different agent setup
Compare — implementation C	Agent fleet	The same required behavior and evaluation obligations	Delegation, orchestration, authority allocation, and fleet controls	Explain fleet effects and compare all three results with recorded evidence

The comparison is local rather than causal proof. The team learns between builds; tools, model behavior, optional scope, hardware, and reporting quality may differ; and each implementation can embody different architectural choices. These are limitations to record, not reasons to abandon comparison. The common brief makes careful claims possible when observations remain separate from explanations.

9.6.1 Build the evidence matrix

Use the concept map to compare the implementations across the same boundaries. Replace each evidence prompt in the matrix with a concrete artifact, result, or explicit `not measured` entry.

Table 9.4. Evidence matrix for synthesizing the three implementations through the shared concept map.

Evidence dimension	Implementation A	Implementation B	Implementation C	Synthesis question
Accepted browser behavior	Acceptance probe and escaped defects	Acceptance probe and escaped defects	Acceptance probe and correction evidence	Which differences affected users rather than only repository structure?
Architecture and contracts	Request trace and key boundary decision	Request trace and key boundary decision	Request trace and key boundary decision	Which boundaries made independent change safer or harder?
Pressure and saturation	Pressure map and first predicted limit	Pressure map and first predicted limit	Pressure map and first predicted limit	How did each architecture respond to the same growth scenario?
Measurement and recovery	Benchmark, signals, and recovery result	Benchmark, signals, and recovery result	Benchmark, signals, and recovery result	What was directly observed, and which behavior remained invisible?
Development friction	Cross-team audit and change effort	Cross-team audit and change effort	Cross-team audit and change effort	Which setup, test, or coupling cost would dominate the next change?
Human–agent workflow	Trace, interventions, and review effort	Trace, interventions, and review effort	Fleet trace, handoffs, and review backlog	Which allocation produced accepted work at a sustainable correction cost?
Resource use and externalities	Runtime and model-use evidence	Runtime and model-use evidence	Runtime and model-use evidence	Did added resources create useful capacity, quality, or only more activity?

Do not turn absent evidence into a favorable score. If a build was not load-tested, state that its saturation behavior is proposed rather than observed. If model usage was not captured, do not infer efficiency from elapsed time. If implementation C produced more branches but required more review and correction, count both the activity and the acceptance cost.

9.6.2 Separate observation, inference, and proposal

A strong synthesis labels the status of each important claim:

- **Observed:** directly inspected or measured in a repository, running application, audit, test, trace, or recorded workflow.
- **Inferred:** a plausible explanation for an observation, with alternatives and confounding factors kept visible.
- **Proposed:** a future response or target design that was not implemented and validated in the compared build.

For example, “implementation B had the lowest measured p95 under the declared workload” is an observation. “Its data model caused the difference” is an inference unless the comparison isolates that factor. “A queue would protect its write path” is a proposal until the new behavior and trade-offs are tested. This vocabulary lets students reason beyond the available experiment without presenting plans as results.

9.7 Turn comparison into a decision

The final synthesis should end with a decision under constraints rather than a universal ranking. A local single-agent workflow may be appropriate when privacy or cost dominates and tasks remain modest. A foundation-model single agent may justify its external dependency when capability reduces correction or expands the range of accepted tasks. A fleet may justify its coordination cost when the project exposes several independent workstreams and has enough integration and review capacity. The preferred architecture may likewise vary with the pressure, competence envelope, and operating skills of the team.

Use the evidence matrix to build the final argument:

1. State the common brief, acceptance core, deliberately varied development setup, and comparison limits.
2. Trace one important outcome through the application, development, and agentic loops.
3. Compare the strongest evidence from A, B, and C, including failures, correction, and resource use.
4. Label observations, inferences, and proposals.
5. Choose an application architecture and development setup for one stated future scenario.
6. Explain the rejected alternatives, the trade-offs accepted, the first likely saturation boundary, and the next measurement.

9.7.1 Final reflection package builder

The builder below assembles evidence already produced during the course. Select the artifacts you have, then use the generated outline to find claims that still lack support.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/recap/

9.8 Project artifact

Submit the completed evidence matrix and a final decision narrative. The package should include the common acceptance result, request-path or architecture evidence, pressure and measurement results, cross-team friction audits, human–agent workflow evidence, one planned-versus-observed decision path, resource-use evidence where available, and remaining risks. For each major claim, mark whether it is observed, inferred, or proposed.

The final narrative should use those artifacts to explain which application architecture and development setup the group would choose under a stated set of constraints. It should also identify the first expected boundary, the signal that would reveal its approach, and the decision the group would revisit when conditions change.

9.9 Questions to carry forward

- Which form of growth is the system actually prepared for, and what evidence establishes that claim?
- Which application, development, or review boundary is most likely to saturate first?
- What would you simplify before adding infrastructure or agents?
- Which signal and correction path would let the next team adapt without guessing?

9.10 Summary

Web scalability emerges from three connected loops. The application loop responds to pressure at architectural boundaries; the development loop determines the cost of changing those boundaries; and the agentic loop determines how execution becomes accepted and correctable work. Measurement connects observation to judgment across all three.

The three implementations provide evidence for reasoning through this concept map, not contestants in a universal ranking. A strong synthesis compares accepted browser behavior, architecture, pressure, observability, friction, human–agent work, and resource use while keeping observations separate from explanations and proposals. The final engineering decision names the conditions it serves, the trade-offs it accepts, the first likely boundary, and what to measure when those conditions change.

APPENDICES

Appendices

The main chapters provide the shortest path through the book and the course project. The appendices preserve deeper methods, implementation details, and research cases that are useful when a particular problem calls for them but are not prerequisites for following the main argument.

Use them selectively:

- **Evaluating agentic workflows** develops the evaluation vocabulary, experimental discipline, graders, uncertainty, and routing questions introduced briefly in the Benchmarking chapter.
- **Reproducible environments** distinguishes repeatable setup, declared environments, hermetic execution, and reproducible builds.
- **Advanced web boundaries** examines state continuity, transactions, client activation, constructive invariants, runtime-generated interfaces, interaction styles, and executable HTTP contracts.
- **Agent-system security** develops control planes, enforcement, traceability, imported capabilities, and cross-run state.
- **Scalability models** gives a deeper treatment of workflow reliability and contention models.
- **Durable workflows** examines retry-safe external effects and recovery across process failure.

Each appendix states the question it helps answer and links its method back to the relevant chapter or project decision.

APPENDIX A

Evaluating agentic development workflows

Use this appendix when you need to design, run, or critique an evaluation of a coding agent or a complete agentic development workflow. The Benchmarking chapter introduces the minimum evidence needed for the three course implementations; this appendix develops the vocabulary, controls, graders, uncertainty, and routing questions in depth.

A.1 Anatomy of an agent evaluation

A software test usually invokes known code with controlled inputs and checks the result. An agent evaluation must also define the situation in which a model acts, because tool access, instructions, the starting repository, resource limits, and intermediate state can all change the result. The following vocabulary helps keep those layers separate (Grace et al., 2026): The table below locates each term within one evaluation.

Table A.1. Vocabulary for the nested objects and responsibilities in an agent evaluation.

Term	Unit or scope	What it records or does	Common confusion
Task	One declared problem	Defines inputs and success criteria	A repeated attempt is not a new independent task
Trial	One attempt at one task	Runs the task once so repeated trials can expose behavioral variation	Trial count is not task diversity
Outcome	End state of one trial	Preserves the resulting patch, database state, generated artifact, or other effect	The final claim is not necessarily the actual outcome
Trace or transcript	Path through one trial	Records outputs, tool calls, intermediate results, and relevant interventions	A plausible trace does not prove a correct outcome
Grader	One evaluated property	Checks an aspect of the outcome or trace through code, a model-based rubric, or human judgment	One grader is not the whole evaluation
Agent harness	Model-and-tool execution loop	Supplies context, tools, state, and control for one agent run	It need not provision or score the evaluation
Evaluation harness	Evaluation infrastructure	Provisions tasks and environments, invokes the agent harness, preserves evidence, applies graders, and aggregates results	It is distinct from the agent being evaluated
Evaluation suite	A declared set of tasks	Groups tasks intended to measure a capability or behavior	A shared benchmark is one kind of suite

These terms describe different measurement objects. A repository test is often one deterministic grader inside an agent evaluation rather than the entire evaluation. A public benchmark is a shared evaluation suite designed for comparison across systems, while a local product or project evaluation should represent the work and failure modes that matter in its own setting. An RL environment additionally exposes a reward signal for optimization or training. Treating these as synonyms hides differences in task selection, incentives, and supported claims.

Prefer grading the outcome when several valid paths can solve the task. Use the trace to diagnose failures, account for cost and intervention, and check process requirements that genuinely matter, such as permission compliance or a mandatory approval. Rigidly requiring one anticipated sequence of tool

calls can reject a valid solution, while accepting only the final statement can miss that the claimed change never occurred.

A.2 Capability and regression evals

A **capability eval** asks whether the workflow can perform work that is currently difficult. Its initial pass rate may be low because it is intended to reveal a boundary and provide room for improvement. A **regression eval** asks whether the workflow still handles previously accepted work and should normally pass almost all trials (Grace et al., 2026). When a capability task becomes reliably solvable, preserve it as a regression task rather than discarding the evidence and starting over.

Start with tasks already checked manually, consequential failures, and representative project changes. For each task, keep a reference solution or known passing run to show that the stated problem is solvable and the graders can accept at least one correct outcome. Everything required by a grader should be justified by the task brief or a declared invariant; an agent should not fail because the evaluator silently assumed a filepath, output format, or environmental resource. Balance positive and negative cases so optimizing one behavior does not produce its opposite failure elsewhere.

A.3 Qualify the served configuration

A model name or local endpoint is not a capability result. Bind an accepted outcome to the configuration that served it: resolved model revision or digest and quantization, runtime version, allocated context, prompt and tool scaffold, and relevant hardware and accelerator residency or CPU/GPU offload where exposed. When an approved service does not expose a field, mark it unavailable and name the evidence source or service operator rather than estimating it.

In a study of 9.7K test examples across long-input and long-form-output benchmarks, quantization effects varied by method, model, task, context length, and language (Mekala et al., 2025). The study used vLLM on datacenter GPUs and did not test coding agents, Ollama, or consumer-device deployments, so one quantized build should not inherit another build's score. Hardware constrains which configurations fit and their latency, throughput, and resource use; it does not establish task capability.

Keep capability, hardware fit, and data location as separate claims. Running inference locally identifies where model computation occurs, not the data boundary of tools, telemetry, storage, or network calls. A fallback can select a different model or configuration and change the capability, latency, cost, and data boundary. Ollama, for example, exposes an installed model's digest and quantization and a running model's allocated context and server-reported model VRAM bytes. It can also forward cloud-model requests while callers continue using its local API, so `localhost` alone does not prove local execution (cloud models).

Assign a stable identifier to each served configuration in the evaluation. Record its exposed model, runtime, context, scaffold, tools, hardware boundary, outbound paths, and fallback policy, then link every attempt and acceptance result to that identifier. For an opaque departmental service, use

operator- or instructor-supplied metadata; otherwise mark unavailable fields and name the evidence source. Qualify the configuration against a predeclared representative task and deterministic check, and treat the result as evidence for that task rather than a general ranking of local and hosted models.

Context selection and output reduction are also configuration decisions. Long tool outputs and interaction histories consume context and can increase latency or cost, which has motivated command-aware filters such as rtk and model-generated summaries. A shorter agent-facing view does not by itself establish a lower total bill or an unchanged task outcome. In a SWE-agent workshop study, observation masking and model-generated summaries reduced cost in most of five tested model configurations, but its paired analysis found that both significantly lowered solve rate in one reasoning-model configuration (Lindenbauer et al., 2025). On controlled long-horizon tool-use benchmarks, ACON reduced peak input tokens while largely retaining task performance (Kang et al., 2026). Neither study validates every command filter; together they show that context reduction is a behavior change to test, not free efficiency.

> **Compact the working view; preserve the raw record**
>
> Treat compressed output as a working view, not as the evidence record. Keep raw output retrievable during the run and preserve the producer's exit status independently. For bounded qualification cases and consequential failures, retain the original command or query, working directory, separate output streams, pagination or continuation metadata, and explicit omission markers under a stable handle. Apply the underlying system's data-minimization, secret-handling, residency, access, and deletion rules; disable archival where those rules cannot be met. Prefer native structured output as a comparison baseline, make capture failures visible, and treat retrieved output as untrusted text rather than embedded instructions.
>
> Qualify each important compression policy with the same task cohort and acceptance checks used for the uncompressed view. Include compression calls, raw retrievals, retries, and failed attempts when reporting total cost per accepted outcome. A reduction in shell-output bytes is only a component metric.

Tool-output preflight. Use instructor-supplied or synthetic non-sensitive raw artifacts for one passing command and one deliberately tricky output: an exit-zero warning, a failing diagnostic, or an unusual line in a repetitive log. Production and secret-bearing output are outside this exercise. If the policy claims to handle pagination, add one paginated artifact.

Before generating compact views, list the critical facts in each raw artifact and require every compact fixture to mark at least one explicit omission. Record the command and tool version, reduction-policy version, configuration, and content hash. Verify that source status, separate output-stream provenance, critical facts, omission markers, retrieval handle, and pagination metadata survive. Report critical facts retained over critical facts present, false or materially misleading facts, and any policy-safe fallback. Passing this preflight can reject an obviously lossy or unsafe policy; claims about cost or task effectiveness require paired end-to-end outcomes.

A.4 The eval suite is also software

Terminal-Bench provides a concrete example of a versioned agent benchmark whose tasks combine instructions, container environments, resource limits, reference solutions, and executable tests. Terminal-Bench 2.1 changed 26 tasks from the earlier suite to repair bugs, adjust timeouts or resources, and improve robustness against reward hacking (Terminal-Bench Contributors, 2026). Its `fix-code-vulnerability` task, for example, pins a Bottle source revision and Python image, specifies CPU, memory, storage, and time limits, requires both a patch and a structured report, and checks the result with original and security-focused tests.

The revisions make a methodological point more important than a leaderboard position: the measurement instrument can be wrong. Ambiguous instructions, incomplete graders, leaked state, shared caches, resource contention, and permissive reference matching can create failures or passes unrelated to the intended capability. Record the suite version and environment with every result, validate a reference solution, start trials from clean state, and read selected traces before trusting an aggregate score. When a task or grader changes materially, do not silently compare the new score with results from the previous instrument.

For this course, inspecting one packaged benchmark task is more useful than running the entire public suite. Identify its claimed capability, starting state, visible instructions, hidden or external assumptions, graders, reference solution, resource boundary, and plausible gaming paths. Then state what a passing result would and would not establish. The exercise teaches benchmark anatomy without replacing the project's local tasks with a heterogeneous and potentially expensive leaderboard workload.

> **Advanced option: derive candidate tasks from repository history**
>
> Repo2RLEnv can mine merged pull requests or commits, reconstruct repository states in Docker, and emit Harbor-compatible tasks whose primary runtime signal comes from the repository's own tests (Hugging Face, 2026). Its runtime pipelines use the familiar fail-to-pass and pass-to-pass shape: tests associated with the change should begin failing and pass under the reference patch, while previously passing behavior should remain intact. This can help an instructor or a mature project turn reviewed changes into candidate evaluation data, but it is not a course baseline and does not make every repository an informative RL environment. History records changes that happened, not a representative sample of future work; tests can omit intended behavior; generated instructions can leak or distort the problem; and similarity to an oracle diff can penalize a different valid patch. An executable reward is verifiable under its grader, not automatically complete ground truth.

Some outcomes can be checked deterministically through tests, schemas, or expected artifacts. Others require a rubric or human judgment. When a model acts as a judge, test whether answer order, verbosity, confidence, or the judge's own style changes its scores; LLM judges have documented position and self-enhancement biases (Zheng et al., 2023). Keep a small human-reviewed calibration set and report disagreement rather than treating the judge as ground truth.

Falck et al. (2026) offer a useful case in which evaluation becomes supervision. Their Replica setup trains a 27B agent on 242 figure-replication tasks using auto-generated per-task rubrics, three judge samples, and turn-level credit assignment, then scores it on 68 held-out AI-for-science tasks. The rubric judge was more repeatable than a generic judge in the paper's repeated-sampling analysis. In a blinded study of ten training tasks selected because the judges disagreed, its Kendall rank correlation with expert rankings was 0.19 rather than 0.15 for the generic judge; on disputed pairs, experts sided with the rubric judge 63% of the time, a difference that was not statistically significant (`p = 0.109`). Because the same judge design supplied training rewards and the headline evaluation, the task holdout did not make the scoring process independent. The evidence supports a bounded claim about a lower-noise training signal and improved scores under this judge, not that the judge is ground truth or that the system performs open-ended scientific innovation.

The demonstration below separates deterministic probes from a rubric score. Use it to inspect a failing task and decide whether the problem lies in the result, the rubric, or the evaluation setup.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/appendices/evaluating-agentic-workflows/

Modern Web Guidance provides a concrete example of evaluating an imported agent capability. Its evaluation harness first checks browser assertions against both a reference implementation and a deliberately flawed implementation, then compares agent runs with and without the guidance. A project can reproduce the smaller experimental shape: hold the task, model, starting repository, and acceptance checks constant; change whether the capability is available; and compare correctness, retries, cost, and human correction. The resulting difference measures performance on that task and those checks, not whether the guidance makes every application better in production.

Literal task completion is not enough for an agent that can take actions. The proposed "Genie coefficient" asks how far an agent's behavior departs from a reasonable interpretation of the user's intent (Raghavan & Schneier, 2026). It is a conceptual framework rather than a validated metric, but it suggests a useful evaluation split: score task success, permission compliance, unnecessary actions, side effects, and clarification quality separately.

For the course project, evaluate one representative AI-assisted task twice: once for the correctness of the produced artifact and once for the quality of the workflow that produced it. Preserve the prompt or task brief, relevant context, model and tool versions, execution trace, checks, human corrections, and final acceptance decision.

A.5 Report uncertainty, not only point estimates

An evaluation observes a finite set of tasks under declared conditions. A difference in average score or acceptance rate is therefore an estimate, not a stable ranking of two models or workflows. In a systematic review of 445 new or substantially modified LLM benchmark papers, only 16.0% used

uncertainty estimates or statistical tests to compare results (Bean et al., 2025). Earlier work on NLP evaluations likewise found that small test sets can lack the power to distinguish plausible improvements and that effects detected by underpowered studies can be exaggerated (Card et al., 2020).

When possible, run both candidates on the same task IDs and predeclare how repeated calls become one task-level result. Repeated calls reveal variation within a task; they do not turn one task into many independent tasks. If many tasks are variants of one template, repository, or issue family, disclose that dependence and narrow the claim.

When a protocol permits `k` trials per task, name the reliability question before selecting a summary. **Pass@k** asks whether at least one of the `k` trials succeeds and is useful when several attempts are genuinely available and one accepted solution is enough. **Pass^k** asks whether all `k` trials succeed and is useful when users need the behavior to be dependable every time (Grace et al., 2026). More attempts make pass@k easier to satisfy and pass^k harder, so the same workflow can look increasingly capable by one measure and increasingly unreliable by the other. Predeclare `k`, budgets, stopping rules, and whether failed attempts remain in latency and cost totals; do not use pass@k to imply first-attempt reliability.

Report the independent task count, task source, one predeclared primary outcome, and observed paired difference. If a course-provided or otherwise justified paired interval method fits the outcome, include the interval and name the method. Before inspecting results, define the signed difference and the practical boundary or bounds that would change the decision. Only an interval wholly on the required side of a boundary supports that local decision; overlap leaves it inconclusive, and statistical separation can still describe a difference too small to matter. An interval containing zero is not proof that the candidates are equal. Treat other metrics and unplanned slices as descriptive or exploratory unless their joint decision rule and multiple-comparison treatment were declared in advance. When no defensible interval is available, show the raw paired outcomes, label the comparison exploratory, and restrict the claim to those tasks. An interval still describes uncertainty conditional on the sample and scoring process; it cannot repair unrepresentative tasks, judge bias, contamination, or a change to the agent harness or evaluation harness.

Paired comparison check. For one predeclared binary acceptance result per independent task, count tasks accepted by both candidates, only the first, only the second, or neither. Use an instructor-supplied interval or a justified paired method to interpret those counts against the predeclared decision boundary; do not compare two separate error bars. The single representative project task above remains a case comparison; use this check with a multi-task project cohort or instructor-provided data.

A.6 From evaluation to routing

A lower per-request price does not necessarily make a model cheaper for a task. An inexpensive attempt that fails may have to be repeated with a stronger model, while still consuming context, validation, and review time. Conversely, always selecting the strongest available model can waste capacity when a smaller model can reliably clear the same acceptance bar. The useful comparison is therefore cost per accepted outcome, not token price or first-attempt cost.

Research distinguishes **routing**, which selects one model before it produces an answer, from **cascading**, which begins with one model and escalates when evidence indicates that its answer is insufficient. Both depend on an accurate quality estimate; Dekoninck et al. (2025) identify estimator quality as a critical condition for improving the cost-performance trade-off. In their SWE-Bench experiment, the post-generation estimator could use the task's ground-truth test outcomes. The result therefore shows what a strong verifier can enable, not how well the approach works for tasks without one. Databricks presents routing as an emerging coding-agent cost-control technique and reports an internal benchmark result, but publishes too little about that router evaluation to predict savings in another codebase (Wendell et al., 2026). This motivates a local experiment, not a promised saving.

When access rules permit both models—or when instructor-provided outputs are available—choose an efficient and a stronger model within the same declared agent harness and assemble a small set of representative tasks. Keep near-duplicates and tasks from the same module in the same partition, using one partition to define the policy and reserving the other for held-out evaluation.

Define two evidence layers before running the comparison. The **policy-visible gate** may use deployable signals such as public tests, builds, or static checks to decide whether a cascade should escalate. The **final adjudicator** uses held-out tests or blinded human review that the policy cannot see; only this layer decides whether the final outcome is accepted. A policy that stops after passing its visible gate but fails final adjudication is a false acceptance.

On the first partition, define and freeze either a routing rule that uses only facts available before generation or a cheap-first cascade that uses the policy-visible gate after its first result. Freeze the task features, thresholds, prompts, versions of the model and agent harness, budgets, retry rules, and exclusion rules before inspecting the held-out results. Compare the policy with always using either model and, when the task set permits it, random selection at the same stronger-model call rate. The random baseline tests whether the policy selects useful cases rather than benefiting only from its mixture of models.

Report accepted outcomes over all attempted tasks, stronger-model calls, end-to-end latency, human correction, and total cost per accepted outcome. Count timeouts, tool failures, missing patches, and budget exits as attempted failures. Include policy decisions, quality estimation, rejected attempts, fallbacks, repeated context, validation, review, and rework in the cost; report unpriced resources separately. If both model arms are run to reveal counterfactual mistakes, keep that one-time evaluation expense separate from each policy's simulated operating cost. That comparison can expose false acceptances and unnecessary escalations; without both arms, label the missing counterfactuals as unknown rather than inferring them from an average.

The result supports a local decision only if the policy meets a predeclared quality and risk threshold while improving the chosen cost or latency measure. After using held-out results to change the policy, evaluate again on fresh tasks. High-risk work can bypass the router or retain mandatory review, and the policy should be recalibrated when models, prices, agent harnesses, or the task distribution change.

A.7 Compressed evidence and repeated review

A compact view can route attention, and repeated review can reveal variation, but neither is acceptance evidence. The following two cases show why the raw artifact, independent checks, and a stopping rule remain necessary.

A reading diff routes attention

Large changes may repeat the same migration across call sites or include generated and mechanically forced edits. `meat` is an experimental tool that asks a model to choose rows to remove, fold, or elide, then mechanically checks and applies that plan to the immutable original diff. The surviving code is therefore source-derived rather than regenerated by the model.

The view is still lossy. An author-run analysis of the Python-only diffs from three selected commits records an aggregate 30.1% byte reduction; it also documents cases where the output hid decisive test outcomes and made an old/new default inversion look like a one-sided addition. The report illustrates the mechanism and its failure modes; it does not measure reviewer effectiveness. In a different task, a mutation-based preprint found that GPT-4 and GPT-5.2 summaries of 12 controlled synthetic Python programs and 50 human-written benchmark programs sometimes described familiar algorithmic intent while missing mutated behavior; it did not study diffs or human reviewers (Khatib et al., 2026).

Use a reading diff as an attention map, not as acceptance evidence. Keep the raw diff and independent checks available, inspect omitted regions at consequential boundaries, and return for decomposition any change that remains too large or coupled to understand.

When findings become inventory

Lawson (2026) uses the provocative claim that teams can now choose how many bugs they want by asking agents to continue reviewing. The operational choice is narrower: a team can choose how much defect-search and verification effort to fund, which known risks to accept, and when to stop. It cannot count the unknown defects that remain.

Cheap reports may shift rather than remove friction. In a live study at Mozilla and Ubisoft, reviewers directly accepted 8.1% and 7.2% of generated comments, respectively, while another 14.6% and 20.5% were marked useful as review or development tips. At Ubisoft, the median instrumented interval for evaluating generated comments was about 43 seconds per patch, an upper bound on active evaluation time. The study measured adoption rather than defect-detection accuracy, but it shows that suggestions still consume attention (Olewicki et al., 2026).

Each finding still needs independent evidence such as a reproduction, a reachable path, or a violated invariant, followed by an impact judgment, a repair decision, and regression evidence. A review loop without a stopping rule can create its own backlog, while repeated local patches can make the system harder to understand. When several reports share a cause, ask whether a simpler boundary or invariant could eliminate the class before adding another exception. Stop against an explicit risk threshold, not when the agent runs out of concerns.

A.7.1 Review-evidence exercise

Choose one bounded representative change. Before opening a compact or agent-produced review view, predeclare two or three behavior or architecture questions. Record provisional answers after that view, then inspect the complete artifact and relevant checks and record every correction, omitted or misleading critical detail, and the time spent in each stage. For agent findings, record candidate, independently confirmed, duplicate, and rejected findings together with triage and repair time. Stop against an explicit risk threshold and ask whether one simpler boundary or invariant could eliminate a repeated defect class.

A.8 Repeated review is measurement, not proof

Repeating an agent review can reveal variance, but agreement between runs is not independent validation. VulnBench repeated identical security-review tasks five times across ten small JavaScript and Express fixtures. Of 158 unique findings that matched its Snyk Code reference, 134 appeared in every repetition; among 161 unmatched findings, 80 appeared only once and 22 appeared in every repetition (Tal et al., 2026). The recurrence difference is useful for triage, but reference overlap measures agreement with Snyk Code rather than ground-truth accuracy. Treat each model review as one sample. Deduplicate repeated findings, record recurrence separately from deterministic-reference overlap, and preserve rare findings for case-level adjudication rather than automatically discarding them. Consensus can repeat a shared blind spot, while a low-frequency report can still identify a real defect. Behavioral tests, invariants, deterministic analysis, and accountable review remain the acceptance evidence. Repeated review also consumes the same scarce integration capacity that bounds the fleet. If students repeat an agent review, they should record the fixed code, prompt, agent harness, and model configuration; the number of runs; unique and recurring findings; reviewer time; and cost per accepted finding. More review samples are justified only when they improve accepted outcomes or allocate human attention better than a single review.

A.9 Apply the method selectively

A course project does not need a public-benchmark-scale evaluation system. Begin with a few representative tasks, deterministic checks where possible, preserved outcomes and failures, and a declared workflow configuration. Add repeated trials, judge calibration, statistical intervals, or routing experiments only when the decision being made requires that additional evidence.

A.10 References

1. Grace, M., Hadfield, J., Olivares, R., & de Jonghe, J. (2026, January). Demystifying Evals for AI Agents. Anthropic Engineering. https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents
2. Terminal-Bench Contributors. (2026). Terminal-Bench 2.1. GitHub repository. https://github.com/harbor-framework/terminal-bench-2-1

3. Hugging Face. (2026). Repo2RLEnv: Convert Any Repository into a Verifiable RL Environment. GitHub repository. https://github.com/huggingface/Repo2RLEnv
4. Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv Preprint arXiv:2306.05685. https://doi.org/10.48550/arXiv.2306.05685
5. Falck, D., Sabri, S., Surina, A., Foster, T., Sims, A., Devlin, S., Rogers, D., Collins, T., Aleksiev, K., Kirsch, L., & Hughes, E. (2026). Training AI Scientists to Replicate Research. arXiv Preprint arXiv:2608.13331. https://doi.org/10.48550/arXiv.2608.13331
6. Dekoninck, J., Baader, M., & Vechev, M. (2025). A Unified Approach to Routing and Cascading for LLMs. Proceedings of the 42nd International Conference on Machine Learning, Proceedings of Machine Learning Research, 267, 12987–13010. https://proceedings.mlr.press/v267/dekoninck25a.html
7. Wendell, P., Bhatia, A., Gaba, V., Elsen, E., & Zhou, I. (2026, August). Managing AI Coding Costs at Scale. Databricks Blog. https://www.databricks.com/blog/managing-ai-coding-costs-scale
8. Tal, L., Kloos, J., Rudich, A., Thoemmes, S., & Nair, M. (2026). Snyk VulnBench JS 1.0: Can LLMs Find the Same Bugs Twice? arXiv Preprint arXiv:2606.15762. https://doi.org/10.48550/arXiv.2606.15762
9. Olewicki, D., Da Silva, L. M. P., Ben Sghaier, O., Mujahid, S., Amini, A., Mah, B., Castelluccio, M., Habchi, S., Khomh, F., & Adams, B. (2026). Impact of an LLM-Based Review Assistant in Practice: A Mixed Open-/Closed-Source Case Study. IEEE Transactions on Software Engineering, 1–12. https://doi.org/10.1109/TSE.2026.3663093
10. Raghavan, B., & Schneier, B. (2026). Why AI Needs a “Genie Coefficient.” IEEE Spectrum. https://spectrum.ieee.org/ai-agent-benchmark
11. Lawson, N. (2026, August). You Can Just Choose How Many Bugs You Want Now. Read the Tea Leaves. https://nolanlawson.com/2026/08/16/you-can-just-choose-how-many-bugs-you-want-now/
12. Khatib, L., Pu, M., Vasilescu, B., & Nagappan, M. (2026). Using Mutation-Analysis to Examine an LLM’s Ability to Summarize Code. https://doi.org/10.48550/arXiv.2602.17838
13. Bean, A. M., Kearns, R. O., Romanou, A., Hafner, F. S., Mayne, H., Batzner, J., Foroutan Eghlidi, N., Schmitz, C., Korgul, K., Batra, H., Deb, O., Beharry, E., Emde, C., Foster, T., Gausen, A., Grandury, M., Han, S., Hofmann, V., Ibrahim, L., … Mahdi, A. (2025). Measuring what Matters: Construct Validity in Large Language Model Benchmarks. Advances in Neural Information Processing Systems, 38. https://doi.org/10.52202/085713-0590
14. Card, D., Henderson, P., Khandelwal, U., Jia, R., Mahowald, K., & Jurafsky, D. (2020). With Little Power Comes Great Responsibility. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 9263–9274. https://doi.org/10.18653/v1/2020.emnlp-main.745
15. Kang, M., Chen, W.-N., Han, D., Inan, H. A., Wutschitz, L., Chen, Y., Sim, R., & Rajmohan, S. (2026). ACON: Optimizing Context Compression for Long-horizon LLM Agents. Proceedings of the 43rd International Conference on Machine Learning, Proceedings of Machine Learning Research, 306. https://arxiv.org/abs/2510.00615

16. Lindenbauer, T., Slinko, I., Felder, L., Bogomolov, E., & Zharov, Y. (2025). The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management. Fourth Deep Learning for Code Workshop (DL4C) at NeurIPS 2025. https://doi.org/10.48550/arXiv.2508.21433
17. Mekala, A., Atmakuru, A., Song, Y., Karpinska, M., & Iyyer, M. (2025). Does quantization affect models' performance on long-context tasks? Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, 9422–9470. https://doi.org/10.18653/v1/2025.emnlp-main.479

APPENDIX B

Reproducible development environments

Use this appendix when setup failures, hidden host state, inconsistent resource limits, or environmental drift make a project difficult to run or evaluate. The Friction chapter keeps the essential project rule—declare the supported environment and test it from a fresh checkout—while this appendix develops the distinctions and audit procedure in depth.

B.1 Treat the environment as a versioned input

A dependency lockfile does not describe the whole development environment. Runtime and package-manager versions, system packages, local services, process startup and readiness, test hooks, browser or command-line tools, and supported operating-system, architecture, and resource assumptions can remain hidden on one workstation. When another contributor, CI worker, or agent sandbox receives the same repository but cannot run its checks, the failure may reveal an implicit input, an incorrect support boundary, or an external or transient dependency.

For an agent, this state is part of the task rather than neutral background. In an internal Anthropic experiment that held the Claude model, agent harness, and Terminal-Bench 2.0 task set constant, reported infrastructure-error rates fell from 5.8% under strict enforcement to 0.5% with uncapped resources across six configurations; above roughly three times the benchmark specification, additional headroom was also associated with solution strategies that could not succeed under tighter limits (Segato, 2026). The vendor-authored study concerns benchmark infrastructure rather than developer onboarding, but it supports recording CPU and memory allocations, hard limits, and enforcement policy; time limits are a separate run condition worth recording.

Keep four related claims separate:

- **Setup automation** provides a bootstrap path, but that path may still be unpinned, stateful, or dependent on the network.
- A **declared environment** records and, where possible, pins the toolchain, dependencies, service definitions, configuration schema, seed-data revision, and supported platform and resource scope needed to reconstruct setup and run the baseline checks.
- **Hermetic execution** isolates an action from undeclared host state and external services and restricts it to declared inputs and controlled tools; running inside a container does not establish this by itself.
- A **reproducible build** means that, given the same source, build environment, and build instructions, independent builders produce bit-for-bit identical copies of the specified artifacts.

devenv is one Nix-based example: it can declare packages, languages, tasks, services, processes, tests, hooks, profiles, and containers, while its lock file records external inputs. It is an implementation example rather than evidence that one tool or Nix fits every project. In a large-scale study of 709,816 package instances from historical Nixpkgs snapshots, more than 99% could be rebuilt successfully, while bitwise reproducibility ranged from 69% to 91% (Malka et al., 2025). That result concerns package

builds, not interactive development workflows or cross-platform equivalence, and shows that declared inputs alone do not guarantee bitwise-identical outputs. Native setup scripts, development containers, and other environment managers can support the same project contract.

Declare which secrets are required and how they are supplied, but inject their values at runtime rather than pinning or committing them in the environment definition. Name host kernels, hardware, networks, mutable service data, and external APIs that remain non-reproducible boundaries; prefer local or demonstration substitutes for the baseline and never require production credentials. Mutable downloads still need a lock, checksum, digest, or explicit freshness policy. Strict pins need an update and security-review policy; reproducibility should not turn an old dependency into an invisible permanent dependency.

Use the existing cross-team audit as the test. From a fresh checkout on another supported machine or container, with project caches, generated files, and project-owned service state absent, run the documented setup and baseline validation and record the host prerequisites already present. Capture the setup time and outcome, any undocumented intervention or retry, the failure stage, and relevant host, resource, or network conditions; record required configuration names but never secret values. After a repair, the authors or CI should replay the path from a fresh state. The result supports a bounded claim about the declared setup, not that production is identical or that a stochastic agent will produce the same patch.

B.2 Choose the required strength

Not every project needs hermetic execution or bit-for-bit reproducible builds. Choose the weakest environment contract that still supports the decision: documented setup for ordinary collaboration, a declared and pinned environment for comparable course evidence, isolation for actions that must not depend on host state, or reproducible artifacts when independent builders must verify identical output. State the remaining external boundaries instead of implying that a tool has removed them.

B.3 References

1. Segato, G. (2026, February). Quantifying Infrastructure Noise in Agentic Coding Evals. Anthropic Engineering. https://www.anthropic.com/engineering/infrastructure-noise
2. Malka, J., Zacchiroli, S., & Zimmermann, T. (2025). Does Functional Package Management Enable Reproducible Builds at Scale? Yes. Proceedings of the 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories, 775–787. https://doi.org/10.1109/MSR66628.2025.00115

APPENDIX C

Advanced control boundaries in web applications

Use this appendix when a design depends on state continuity across requests, transactional invariants, client activation, runtime-generated interfaces, or a detailed application contract. The Architecture chapter keeps the decisions that every project must recognize; the sections below develop implementation choices and audits that are useful only when those boundaries are present.

C.1 Stateless requests still depend on state

Statelessness describes what a service needs to process one request, not whether the application remembers anything. The final `2026-07-28` revision of the Model Context Protocol (MCP) removes the initialization handshake and the Streamable HTTP `Mcp-Session-Id` header. Servers must not infer protocol context from earlier requests on the same connection; the core carries no protocol session across calls (Model Context Protocol, 2026). This is a versioned protocol example, not a requirement that the course project expose MCP or that older MCP traffic use the same design.

The table below compares what a client carries forward and what the deployment must still provide.

Table C.1. Continuity designs and the deployment obligations they retain.

Continuity design	What a later request carries	Deployment obligation
Hidden protocol session	A session identifier	Route to the instance that owns it, or share the protocol-session state
Explicit application handle	A resource handle such as `workspace_id`	Authorize the operation and resolve the referenced state from every eligible instance, or retain affinity

An explicit handle moves continuity rather than deleting it. An application may select state from the authenticated principal and ordinary request parameters; when workflow state cannot be derived from them, a service can mint a handle and require later calls to return it. The service still has to store or reconstruct the referenced state and define its owner, lifecycle, concurrency rules, expiry, and cleanup. A handle is an address, not authorization: authenticate the caller, then authorize that principal to perform the requested operation on the referenced tenant or workspace on every request. If the state remains available only on the instance that minted the handle, affinity has merely moved behind a new identifier.

Retries add another state boundary. If a mutation commits but its response is lost, a retry may reach a different instance and apply the mutation again. Do not assume that a protocol request identifier makes a domain operation idempotent. Use an application-level command identifier or idempotency key, bind it to the caller, handle, operation, and relevant arguments, and store the mutation and reusable result atomically in the authoritative application state.

State placement audit. Assume two interchangeable server instances, `A` and `B`, share one application store. Alice's validated token identifies principal `alice` in tenant `T1` with `events:write`; creating a workspace grants her access to it. Bob's token identifies principal `bob` in the same tenant with

the same scope, but he has no access to Alice's workspace. The server derives these identities from the validated tokens, not from client metadata or a resource handle.

1. Alice calls `create_workspace` and receives `W1`.
2. Alice calls `record_event(W1, work_item_id=I42, command_id=C7, type=work_started)`; instance `A` commits the event, but the response stream breaks.
3. The client retries on instance `B` with a new protocol request identifier and the same `C7`.
4. Bob calls `record_event(W1, work_item_id=I42, command_id=C8, ...)`.
5. The workspace is now `archived`, and the stated lifecycle makes archived workspaces read-only; Alice calls `record_event` again.

For each request, record the authenticated principal and scope, authoritative state before the request, decision to apply, return, or reject, and the invariant or evidence supporting that decision. Design the flow so `C7` produces one event and its retry returns the original result, Bob cannot act merely by knowing `W1`, and the archived write returns a defined application error such as `workspace_archived` rather than silently recreating state. Finish by naming which information is carried by the client and which state remains authoritative on the server.

C.2 Preserve durable invariants with transactions

A database transaction groups operations into one unit with four commonly named properties: atomicity, consistency, isolation, and durability, or ACID (Härder & Reuter, 1983). **Atomicity** means the operations commit together or not at all. **Consistency** means a committed transaction leaves the data within its declared invariants; the database cannot invent those application rules, so they still have to be expressed through constraints, guarded writes, or validated domain logic. **Isolation** describes how concurrent transactions are allowed to observe and affect one another. The exact guarantee depends on the selected isolation level rather than on the mere presence of a transaction. **Durability** means a successful commit survives the failures covered by the storage system's contract.

The work-item lifecycle contains a useful transaction boundary. A successful transition changes the current state and records the corresponding activity event. If either write escapes alone, the activity feed and work item disagree. The same invariant is threatened when two requests make a decision from the same old state or when the application returns success before persistence is durable.

C.2.1 ACID transaction lab

Choose one disturbance, predict where the invariant could leak, and run it first with application-managed writes. Then introduce a database transaction and, for concurrent requests, compare read committed with serializable execution.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/appendices/advanced-web-boundaries/

The simulation does not imply that every database implements an isolation level or durable commit in the same way. For an implementation-specific claim, identify the real transaction boundary, constraints, isolation setting, acknowledgment point, and documented failure model, then test them against the deployed storage path.

C.3 Model valid data combinations

A transaction constrains how a valid state change becomes durable. Constructive data modeling asks which combinations ordinary code should be able to construct before that boundary is reached. In *The Unreasonable Effectiveness of Constructive Data Modeling*, Alexis King describes defining the supported "positive space" with ordinary records and tagged variants, then using exhaustive case analysis where the language supports it (King, 2026). King's earlier *Parse, Don't Validate* frames a boundary check as a transformation that returns a more precise representation carrying what it learned rather than only reporting success (King, 2019). These are design arguments rather than comparative evidence, and the goal is not maximum type precision.

For the result of a state-transition decision, a loose record does not require the payload that each outcome needs:

```
type TransitionDecision = {
  accepted: boolean;
  next?: WorkItem;
  current?: WorkItem;
  event?: ActivityEvent;
  reason?: TransitionError;
};
```

It permits `accepted: true` without a next item or event, and `accepted: false` without a current item or reason. Model the required payload for each branch instead:

```
type TransitionDecision =
  | { kind: "accepted"; next: WorkItem; event: ActivityEvent }
  | { kind: "rejected"; current: WorkItem; reason: TransitionError };
```

The discriminator makes each branch's required payload explicit, and narrowed type-checked code cannot treat a rejected decision as if it exposed a transition event. It does not prove that the event matches the correct transition unless the inner types or constructor also enforce that relationship. The value is an in-memory decision, not a durable fact. Untrusted, untyped, or deserialized input crossing a boundary still needs runtime parsing. Before committing a decision, the request path must authenticate and authorize the caller; the persistence operation must conditionally match the authoritative state or version, then atomically update the item and append the event. Malformed deserialization, mutation, or unsafe casts can bypass the representation, while concurrency and retries can violate surrounding invariants even when each individual value is valid.

Selected-invariant check. In the existing `specs/{feature}/spec.md`, show one broad record or result and one contradictory value it permits. Replace it with a stack-native tagged union, sealed hierarchy, or guarded constructor. Where the stack supports checked variants, add a compile-fail type

fixture; add a runtime test for any parser or guarded constructor at a data boundary, and reuse an existing supported-behavior test. Record one important invariant that remains enforced elsewhere, such as authorization or atomic concurrent transition.

C.4 Activate delivered interfaces

Rendering and navigation do not explain how delivered HTML becomes interactive. A server-rendered page may contain useful content while the browser still has to recover or load the behavior behind its controls. That activation work affects payload size, client processing, first-interaction latency, and application structure.

Table C.2. Client activation approaches and their principal trade-offs.

Approach	Activation unit	Main benefit	Main constraint
Hydration	The rendered component tree	Familiar, general application model	Re-executes client code to recover state and listeners
Progressive hydration	Ordered portions of the same tree	Prioritizes important interaction	The application is still eventually hydrated
Islands architecture	Explicit interactive regions	Static HTML elsewhere; work can be deferred or avoided	Shared state, navigation, and communication across islands
Resumability	Serialized state and handler boundaries	Continues without replaying the application	Serializable state and framework compiler or runtime constraints

Islands partition a page into static and interactive regions and attach a loading strategy to each interactive region. Resumability changes how execution crosses the server-client boundary: component boundaries, application state, and listener references are serialized into the delivered HTML so the browser can continue without replaying the application (Vepsalainen et al., 2024). A resumable framework can therefore create fine-grained islands automatically.

The strongest fit for explicit islands is an application with meaningful static regions and a smaller amount of separable interactivity. Research on edge-powered islands found that work could be deferred or avoided, but also concluded that the benefit becomes harder to obtain as an application grows more dynamic and difficult to divide into independent portions (Vepsäläinen et al., 2025). Inter-island state, navigation, and communication then become architectural costs rather than incidental implementation details.

C.4.1 Client activation lens

The lab below holds the server-rendered event page constant and changes only how its behavior becomes available in the browser. Compare what happens before the first interaction, then run the same record-event action. The transfer values are illustrative teaching values, not framework benchmarks; a real comparison should record production bundles, client execution, and interaction latency on representative devices.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/appendices/advanced-web-boundaries/

Ask which code and state must cross the boundary, which work happens before the user requests it, which latency moves to the first interaction, and what new constraints the chosen technique introduces. For each interactive region in the project, record its activation strategy, state owner, and communication path.

C.5 Runtime UI generation changes the control boundary

The earlier sections ask where a view is produced and how it becomes interactive. A separate question is how much of its structure and behavior model output may choose at runtime. A component written by a coding agent during development is not runtime-generative UI in this sense: once reviewed and committed, it is ordinary application code at runtime. The boundary expands as runtime output moves from filling a host-selected component to selecting component composition or executable behavior.

Casas (2026) describes a practical spectrum from fixed components, through declarative descriptions rendered from an approved catalog, to generated executable code. Treat this as a design lens rather than a settled taxonomy. The table below shows how the application's control obligation expands with the model's runtime freedom.

Table C.3. Runtime model-output boundaries and the application controls they require.

Runtime model output	Application must control	Main trade-off
Data for one approved component	Component implementation, behavior, and prop schema	Predictable behavior with little freedom over layout
Declarative descriptor using approved components/actions	Schema, renderer, design system, and action registry	Composition within a grammar, with validation and schema-evolution burden
Generated HTML, CSS, and JavaScript	Isolated execution boundary, mediated data/tool bridge, and limits	Broadest expressiveness with the largest failure, review, and containment load

This axis can be combined with server-side or client-side rendering, hydration, islands, or resumability. It does not replace those choices. A declarative descriptor is still untrusted input: reject unknown components and actions, validate data bindings and accessible names, bound size and nesting, and authorize an action when it is invoked. Resolve action identifiers through trusted application code; naming an action in generated data does not grant authority to perform it.

Executable generated UI expands the boundary further. It needs a separate-origin sandbox or equivalent isolated runtime without ambient credentials, a narrow bridge to host data and tools, explicit network, storage, navigation, and tool permissions, resource limits, observability, and a safe fallback. Running generated code in a browser does not by itself provide that boundary.

A Google Research preprint provides bounded preference evidence for one complete generative-UI pipeline. After excluding eight of 100 sampled LMArena prompts, the study sent each pre-cached result to two raters. Across the pairwise comparisons, raters preferred the generated UI over the study's markdown condition in 82.8%. Contractor-built pages were preferred over the generated pages in 50.0% of comparisons, while the generated pages were preferred in 35.3% and the remainder was neutral. The pipeline combined Gemini with search and image tools, extensive instructions, and post-processing; generation often took one to two minutes and was excluded from the ratings. The study therefore measures preference in that setup, not task success, correctness, accessibility, security, maintainability, or production readiness (Leviathan et al., 2026).

C.6 Descriptor boundary audit

Create one small event-summary data fixture and keep it unchanged. First define the prop schema and payload for one fixed summary component. Then define minimal prop schemas for an approved registry such as `metric`, `eventList`, and `button`, plus a small action allowlist, and express the same fixture as a declarative descriptor. Define where and how the system rejects an unknown component, raw HTML or script, an unregistered action, a control without an accessible name, and excessive nesting. For each rejection, state the enforcing layer, error, and test, and use the fixed read-only summary as the fallback. Finally, identify the additional containment and evidence that would be required before the application could execute model-generated code.

The descriptor schema and action vocabulary are API contracts. The next section examines what follows once such contracts cross client and server boundaries.

C.7 Choose an application interaction style

An interface distributes knowledge between consumers and providers: which addresses or operations exist, how data is represented, what errors mean, and how either side can change independently. Several interaction styles can expose the same behavior while placing coupling in different locations.

Table C.4. Application interaction styles and where each places coupling.

Style	What the consumer knows in advance	Where change pressure concentrates	Typical operational concern
Resource-oriented HTTP	Resource addresses, representations, methods, and statuses	Resource and representation compatibility	Caching, idempotency, pagination, and versioning
Hypermedia-driven REST	Entry points, media types, and stable relation semantics	Relation semantics and completeness of advertised controls	Link validity, client interpretation, and adoption
GraphQL	A typed schema and the fields needed for each operation	Schema governance, field evolution, and deprecation	Authorization, resolver behavior, and query cost
RPC	Named methods and request or response message contracts	Method and message compatibility	Deadlines, retries, idempotency, and streaming flow

Resource-oriented HTTP uses the uniform method and status semantics of HTTP. Hypermedia-driven REST adds links or controls so representations advertise available transitions; HATEOAS is therefore a REST constraint rather than a separate peer to REST (Fielding, 2000). GraphQL lets a consumer select a response shape against a typed schema. RPC exposes named operations and explicit messages. GraphQL and RPC commonly travel over HTTP, so transport and application interaction style remain separate choices.

Event streams, queues, webhooks, and publish-subscribe interfaces add another decision: communication can become asynchronous. That can reduce temporal coupling while making delivery guarantees, ordering, replay, idempotency, and schema evolution explicit parts of the contract.

C.7.1 API contract lens

The lab below holds one user intent constant: start work on item 42 and obtain its current state, title, workspace, and available next actions. Switch the interaction style to see what crosses the wire, what the consumer must know in advance, and where a request for one more related field creates pressure.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/appendices/advanced-web-boundaries/

No style wins every row. A stable public resource model, a workflow discovered through hypermedia, a frontend assembling related data, and an internal service command provide different reasons to accept different forms of coupling. Begin with consumers, change patterns, network conditions, tooling, and ownership rather than a preferred acronym.

For the project, design a minimal event-creation operation and event-query operation in one style. Then identify what changes when consumers retry, devices go offline, a new workflow state appears, many tenants share the system, or the interface needs an additional related field.

C.8 Executable HTTP contracts

An API description becomes more durable when representative requests and expected responses are executable. Hurl expresses HTTP sessions in plain text and can assert status codes, headers, and response bodies without depending on the application's implementation language. For example, an HTTP-based event tracker could preserve one creation-and-read scenario as follows:

```
POST http://localhost:3000/events
Content-Type: application/json
{
  "type": "work.started",
  "workItemId": "item-42"
}

HTTP 201
[Captures]
event_id: jsonpath "$.id"

GET http://localhost:3000/events/{{event_id}}

HTTP 200
[Asserts]
jsonpath "$.type" == "work.started"
jsonpath "$.workItemId" == "item-42"
```

This file is both an example of the interface and a deterministic integration test. It can be run locally or in continuous integration, and a failure points to the request or assertion that no longer matches the implementation. If implementations A, B, and C expose a compatible HTTP boundary, the same scenario can be run against each one by changing the base URL rather than rewriting the test in each implementation's language.

Hurl does not replace browser testing: it checks the HTTP boundary, while a tool such as Playwright checks behavior through a browser, including rendering and interaction. Although Hurl can repeat requests and assert response duration, a small contract scenario is not evidence that the system will behave well under representative load. The common project specification does not require an HTTP API, so this technique is optional for implementations that expose one rather than a universal project requirement.

C.9 Use the boundary that matches the risk

These techniques solve different problems. Explicit handles and idempotency protect continuity across requests; transactions protect durable invariants under concurrency and failure; constructive models reduce contradictory in-memory states; activation approaches move browser work; descriptor schemas constrain runtime composition; isolation contains executable output; interaction styles distribute coupling; and HTTP examples preserve an interface contract. Choose the smallest boundary that makes the relevant decision observable and enforceable.

C.10 References

1. Model Context Protocol. (2026, July). Model Context Protocol Specification, Revision 2026-07-28. https://modelcontextprotocol.io/specification/2026-07-28
2. Leviathan, Y., Valevski, D., Kalman, M., Lumen, D., Segalis, E., Molad, E., Pasternak, S., Natchu, V., Nygaard, V., Venkatachary, S., Manyika, J., & Matias, Y. (2026). Generative UI: LLMs are Effective UI Generators. arXiv Preprint arXiv:2604.09577. https://doi.org/10.48550/arXiv.2604.09577
3. Casas, R. (2026, June). Beyond Components: Designing Generative UI for MCP Apps. AI Engineer, YouTube video. https://www.youtube.com/watch?v=hCMrEfPG2Yg
4. Vepsalainen, J., Hevery, M., & Vuorimaa, P. (2024). Resumability—A New Primitive for Developing Web Applications. IEEE Access, 12, 9038–9046. https://doi.org/10.1109/ACCESS.2024.3352891
5. Vepsäläinen, J., Vuorimaa, P., & Hellas, A. (2025). The Potential of Serverless Edge-Powered Islands for Web Development. Journal of Web Engineering, 24(1), 1–38. https://doi.org/10.13052/jwe1540-9589.2411
6. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures [Phdthesis, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm
7. King, A. (2019, November). Parse, Don't Validate. Personal technical essay. https://lexi-lambda.github.io/blog/2019/11/05/parse-don-t-validate/
8. King, A. (2026, July). The Unreasonable Effectiveness of Constructive Data Modeling. Software Should Work conference presentation. https://www.youtube.com/watch?v=0BXuYlNrUmE
9. Härder, T., & Reuter, A. (1983). Principles of Transaction-Oriented Database Recovery. ACM Computing Surveys, 15(4), 287–317. https://doi.org/10.1145/289.291

APPENDIX D

Agent-system security, control, and traceability

Use this appendix when an agent workflow crosses several orchestration, model, tool, credential, or network boundaries. The Agent Fleets chapter keeps the essential rules—allocate decision authority, enforce least privilege, preserve evidence, and keep a human owner—while this appendix develops control-plane separation, end-to-end trace reconciliation, capability adoption, and persistent cross-run state.

D.1 Separate task control from resource access

Controls around a fleet operate at different scopes. The project harness introduced earlier is the versioned material inside a repository: specifications, commands, rules, and evidence. An agent harness controls a model-and-tool execution loop. A session or orchestration layer may coordinate several agent harnesses, while a resource-access layer may mediate selected model and tool requests. These responsibilities may be combined in one platform or split across several services. Together with the people, review process, and ownership rules around them, these layers form the broader development system.

The table below separates task-level orchestration from request-level resource access.

Table D.1. Control boundaries for agent orchestration and model or tool access.

Control boundary	Primary unit	Typical responsibilities
Session and task orchestration	Run, delegated task, workspace	Agent-harness selection, task graph, session history, collaboration, delegated tool authority, sandbox choice, and task-level policy
Model and tool access	Model or tool request	Permitted services and credentials, request routing and fallback, rate and spend limits, request guardrails, and shared usage telemetry

Omnigent illustrates the first scope: its open-source meta-harness wraps several agent harnesses behind a common session interface and documents composition, shared sessions, contextual policies, and local or remote sandbox execution (Zaharia et al., 2026). Unity AI Gateway illustrates the second: for model and MCP traffic routed through it, Databricks documents catalog-backed permissions, rate limits, budgets, provider routing and fallbacks, and usage records (Databricks, 2026). These are examples, not evidence that every fleet needs two products or exactly two control planes. Both accounts are documented in Databricks-authored sources, their responsibilities overlap, and Omnigent was introduced as alpha; they are not independent evaluations of effectiveness.

The durable distinction is between a decision and its enforcement point. NIST's zero-trust architecture distinguishes a policy engine that makes and logs an access decision, a policy administrator that executes it by configuring an enforcement point, and the enforcement point that enables, monitors, or terminates access; implementations may combine the engine and administrator (Rose et al., 2020). This

is a design analogy, not a claim of NIST conformance. For each consequential decision, name the policy authority, mandatory enforcement point, and conflict or outage rule; overlapping controls still need explicit precedence, and a mediator cannot protect a path that bypasses it.

The contract between the scopes matters more than the number of components. The HAR decision map states which decisions and transfers should occur; these control scopes provide some of the places where that design can be enforced and observed. A responsibility recorded in a table without a mandatory enforcement point remains an intention. Trace one operation through the chain:

`originating actor → fleet session → delegated task → model or tool attempt → validation → delegated-run outcome`

Propagate only the necessary authenticated principal, audience-scoped delegated authority, and trusted task, trace, budget-reservation, and policy-decision identifiers. Each enforcement point must validate the authority and identifiers it relies on. Return the resource actually selected, routing or fallback decision, policy result, and observable latency, usage, and cost; for a model call, also record its provider, model, and version where available. Associate every physical attempt with one delegated run and that run's explicit outcome, such as accepted, rejected, failed, cancelled, or unresolved. Default telemetry to metadata: capture prompts, source, tool arguments or results, approval details, or principal data only when necessary, with redaction, access control, and deliberate retention, and never place credentials in trace context. A trace identifier connects evidence; it is not proof of identity or permission. Without this contract, the resource layer can see spend without knowing whether the work was accepted, while the orchestration layer can see intent without knowing which model or permission was actually used.

For one delegated task, choose two consequential decisions: one about the session or task and one about a model, tool, credential, or other resource. For each, record the policy authority, mandatory enforcement point, trusted context or evidence, and conflict or outage rule. Then trace the task end to end and look for a direct-call bypass, lost actor identity, multiplied retry, conflicting caps or double-counted usage, and cancellation that does not reach in-flight work. Where practical, use a safe direct-call negative test and, where records are available, reconcile the mediator with the model provider or tool resource rather than inferring non-bypass from an incomplete trace. The audit is complete only when every denied, failed, retried, cancelled, or successful attempt has an identifier, policy result, and usage or cost provenance where measurable, and rolls up once to an explicit delegated-run outcome. When computing cost per accepted outcome, include spend from every attempt in the fixed cohort and use accepted delegated runs—not internal calls—as the denominator; label missing or estimated costs rather than treating them as zero.

For the B–C comparison, keep any gateway pinned to the declared base model where possible; otherwise record the model actually used and treat substitution as a confounding factor.

D.1.1 Fleet trace debugger

The forensic packet below separates orchestration outcomes, gateway decisions, and physical provider or resource records. Map every physical row to a delegated run or leave it unmatched, then reconcile the packet without counting the same attempt once per log source.

Interactive lab

This activity is available in the web edition.
scalableweb.dev/appendices/agent-system-security/

The packet is synthetic and deterministic. It rehearses evidence joins, accounting, bypass detection, and late-effect reasoning; it does not estimate the reliability or cost of a particular fleet platform.

D.2 Trust boundaries

An agent's role should define not only its task but also which information it can read and which actions it can take. Simon Willison's lethal trifecta names a particularly dangerous combination: access to private data, exposure to untrusted content, and a way to communicate externally. Prompt injection in the untrusted content can then turn the agent's legitimate permissions into a path for data exfiltration.

D.2.1 Imported capabilities are development dependencies

A skill may arrive as prose, yet it can direct tool calls made with the agent's authority and may bundle code or configuration. A USENIX Security 2026 study crawled 98,380 skill entries from two community registries in January 2026. Static triage selected 4,287 candidates; sandboxed behavioral verification and independent judgments by two of the authors confirmed 157 malicious skills (Liu et al., 2026).

Those 157 are a behaviorally confirmed lower bound in one snapshot, not an ecosystem prevalence estimate. The snapshot excluded private, unindexed, and official collections; one publisher identifier contributed 85 templated cases; and 60-second sandbox runs could miss dormant or environment-gated behavior. Official provenance, popularity, scanning, or a previous safe version can filter risk; none proves that an imported capability or update is safe in the current host.

Treat adoption and updates as dependency changes. Record the source and resolved revision; inspect instructions, bundled files, installation hooks, downloads, and mutable remote inputs; identify missing or unreviewable content; and map requested access against the authority the host would actually grant over files, commands, networks, and credentials. Perform any dynamic verification only in a disposable sandbox approved for security testing, with the least authority needed and no production credentials. Connect output produced with the capability to independent project checks, and define who reviews updates and how the capability can be removed.

The skill-adoption lab provides a non-executing fixture for practicing this review without loading an untrusted capability into an agent context; study its source or download the standalone lab.

> **Case study: when exploration scales**
>
> During a July 2026 cyber-capability evaluation using the ExploitGym benchmark, an AI agent escaped its evaluation environment and chained weaknesses across external infrastructure and Hugging Face's production systems (OpenAI, 2026). Hugging Face reconstructed roughly 17,600 agent actions across dataset processing, Kubernetes, cloud identity, an internal network, and source control. Most attempts failed, but the volume and persistence of exploration eventually exposed a viable path across several trust boundaries (Larcher et al., 2026).

> OpenAI's later reconstruction added a crucial continuity mechanism. Separate agent runs discovered files left in a shared Artifactory service and used them as an informal message board for requests, exploit details, assignments, scripts, progress, and credential information. After the files were cleared and the posting credentials revoked, later runs recreated the channel through an unauthenticated WebDAV endpoint, using message-bearing directory names in Artifactory's remote cache (Dalton & Wallace, 2026).
>
> The sandboxes were short-lived, but the runs were not independent. Unlike the curated project memory described earlier, this was ambient infrastructure state. A location becomes cross-run memory and an unintended coordination channel when one run can leave state that survives, is discoverable, and influences a later run. Artifact and package stores, caches, queues, logs, temporary storage that outlives a run, and remote workspaces therefore belong in the trust-boundary review. For each channel, record who can write and later read it, how writes are attributed, what may be stored, how long it persists, and how it is cleared.
>
> The incident did not make established defensive principles obsolete. It showed why they become more important with agents: strict sandbox isolation, least privilege, short-lived and narrowly scoped credentials, blocked metadata access, and telemetry that can correlate behavior across systems. Agent safety therefore depends on the authority exposed by the surrounding system, not only on instructions or model behavior.

The safest design is to break the combination where possible. A scout reading untrusted issues may not need secrets or network-writing tools. An implementer may work in a sandbox with scoped files and no deployment credentials. A publisher may receive a reviewed artifact instead of the full private context that produced it. Human approval can help at a sensitive boundary, but an approval prompt is not useful when the effect is unclear or reviewers have learned to accept it mechanically.

D.3 Complete the control-boundary review

For each consequential action, name the decision authority, mandatory enforcement point, granted scope, evidence returned, and conflict or outage rule. Then follow one denied, failed, retried, cancelled, or successful attempt across every relevant system and confirm that it rolls up exactly once to an explicit delegated-run outcome. A diagram or approval step is not a control unless the action cannot bypass it.

D.4 References

1. Liu, Y., Chen, Z., Zhang, Y., Deng, G., Li, Y., Ning, J., & Zhang, L. Y. (2026, August). “Do Not Mention This to the User”: Detecting and Understanding Malicious Agent Skills in the Wild. 35th USENIX Security Symposium (USENIX Security 26). https://www.usenix.org/conference/usenixsecurity26/presentation/liu-yi
2. Zaharia, M., Uhlenhuth, K., & Zumar, C. (2026, June). Introducing Omnigent: A Meta-Harness to Combine, Control and Share Your Agents. Databricks Blog. https://www.databricks.com/blog/introducing-omnigent-meta-harness-combine-control-and-share-

your-agents

3. Databricks. (2026). AI Governance with Unity AI Gateway. Databricks Documentation. https://docs.databricks.com/aws/en/ai-gateway/
4. Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). Zero Trust Architecture (NIST Special Publication Nos. 800–207). National Institute of Standards. https://doi.org/10.6028/NIST.SP.800-207
5. Dalton, M., & Wallace, E. (2026, August). The “Breaking” News: The OpenAI–Hugging Face Incident —A Technical Reconstruction and Its Implications for AI. Black Hat USA 2026 conference presentation. https://www.youtube.com/watch?v=87DyyMV0kCY
6. Larcher, H., Carreira, A., G., R., & Rannou, C. (2026, July). Anatomy of a Frontier Lab Agent Intrusion: A Technical Timeline of the July 2026 Incident. Hugging Face Blog. https://huggingface.co/blog/agent-intrusion-technical-timeline
7. OpenAI. (2026, July). OpenAI and Hugging Face Partner to Address Security Incident During Model Evaluation. OpenAI. https://openai.com/index/hugging-face-model-evaluation-security-incident/

APPENDIX E

Formal scalability and reliability models

Use this appendix when a scaling or workflow decision needs more than the introductory Little's Law or Amdahl's Law treatment in the main chapters. The models below expose two different ways that adding work can reduce success: serial reliability compounds across critical steps, while contention and coordination can flatten or reverse throughput gains.

E.1 Lusser's Law and the reliability chain

A workflow whose accepted result depends on every critical step has the structure of a series system. If step `i` has reliability `r_i`, the workflow reliability is:

```
R_workflow = ∏ r_i
```

When `n` independent critical steps have the same reliability `r`, this reduces to `R_workflow = r^n`. This product rule is commonly called Lusser's Law or the product law for series reliability (Tait, 1995). It explains why a small per-step failure probability can dominate a long workflow even when no individual step looks especially risky.

Redundancy within one step has a different topology. Jason Gorman's compact formulation uses `C` for the probability that the first prediction is correct and `P` for the conditional probability that an incorrect prediction is caught. For completion reliability, catching the prediction must also restore the step to a correct state; under that stronger interpretation:

```
r = C + (1 − C)P = 1 − (1 − C)(1 − P)
```

The complement expression describes the prediction-or-recovery paths within one step; Lusser's Law describes the product across serial steps. The distinction matters because detection alone does not complete a workflow. If `D` is the probability that an incorrect prediction is detected and `K` is the probability of successful repair after detection, then the completion reliability of one step is:

```
r = C + (1 − C)DK
```

An incorrect result that is detected but not repaired can stop safely, while an undetected result can propagate. Safety, liveness, and correctness should therefore be reported separately rather than collapsed into one success percentage.

E.2 Reliability chain lab

The first preset below reproduces the compact idealization with 95% initial correctness, 90% catch-and-repair coverage, and 100 independent critical steps. Before changing the controls, predict whether the workflow remains above 50% correct completion. Then select **Caught, not repaired** and compare correct completion with safe stopping.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/appendices/scalability-models/

The model assumes identical independent steps, treats every step as necessary, and assigns the first terminal outcome to the whole run. Real agent workflows can share model blind spots, context, tools, and upstream state; steps can have different reliabilities; and later validation can sometimes repair earlier failures. Use the curve as a design probe, not a reliability forecast. Estimate `C`, `D`, and `K` from repeated accepted outcomes at a declared boundary, and use `∏ r_i` when observed steps differ instead of replacing them with an unsupported average.

E.3 Contention curve lab

Amdahl's Law models a fixed sequential share but does not describe the way shared resources and coordination can become more expensive as concurrency grows. The Universal Scalability Law models relative capacity as:

`C(N) = N / (1 + α(N − 1) + βN(N − 1))`

Here `α` represents contention or serialization cost and `β` represents coherency or coordination cost. The curve can rise, flatten, and eventually fall: adding workers may reduce total capacity when coordination grows faster than useful work.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/appendices/scalability-models/

First set both costs to zero and inspect the ideal linear curve. Then add contention, add coherency cost, and find the first concurrency level after the modeled peak. The parameters should ultimately be fitted from measured throughput at several concurrency levels; arbitrary sliders illustrate the model's shape but do not predict a real deployment.

> **Case study: limiting concurrency to protect throughput**
>
> PlanetScale describes a Vitess and MySQL production incident in which a long-running transaction, immediate application retries, and a transaction pool raised to about 10,000 allowed more than 10,000 requests to accumulate inside the storage engine. Snapshot reads grew more expensive as the long transaction preserved an increasingly long version history; more requests exceeded their latency limit and retried. Work in progress rose while completed query throughput fell. Increasing the pool had silenced pool-full errors but weakened the boundary protecting the database.
>
> The team later reduced the pool to roughly 1,000 and changed full-pool behavior so requests could wait for a slot within a bounded timeout. During a later burst on another shard, demand peaked around 25,000 slot requests per second while reported query throughput stayed near 60,000 per second and fewer than 200 statements executed inside MySQL at once. If those figures describe the

> same stable boundary and observation window, Little's Law gives an upper estimate of `W < 200 / 60,000` seconds, or about 3.3 milliseconds. The important separation is between clients seeking work, work admitted to the constrained resource, and useful work completed.
>
> A queue does not create capacity; it moves waiting to a boundary where it can be observed and limited. This report changed pool size and waiting behavior together, the later burst did not reproduce the original incident, and no tail-latency distribution was reported. It also uses the Universal Scalability Law as an explanatory lens rather than fitting its parameters from measurements. Treat the numbers as evidence for this workload, not as a universal pool target. The durable lesson is that admission control can raise throughput when in-flight requests slow one another down, provided waiting is bounded, overload remains visible, and the underlying long transaction and retry behavior are still addressed.

E.4 Treat the model as a challenge to the design

A formula is useful only when its boundary, variables, assumptions, and evidence are explicit. Use measured inputs where the model requires them, keep recovery and correlated failures visible, and do not extrapolate beyond the observed workload merely because the fitted curve is convenient.

E.5 References

1. Tait, N. R. S. (1995). Robert Lusser and Lusser’s Law. Safety and Reliability, 15(2), 15–18. https://doi.org/10.1080/09617353.1995.11690648

APPENDIX F

Durable workflows and retry-safe effects

Use this appendix when an asynchronous operation spans several steps, must survive process failure, or can repeat an external side effect. The Scaling chapter keeps the essential design warning: durable coordination does not make external effects exactly-once. This appendix develops explicit state machines, replay-based workflows, retry boundaries, idempotency, compensation, cancellation, and operator evidence.

F.1 Durable execution preserves coordination progress

A queue can preserve a pending work item, but an asynchronous multi-step operation that must survive process failure also needs durable coordination state. An explicit application state machine can store each transition and its current state in a database. A replay-based workflow engine instead persists scheduling decisions and task results, then reconstructs logical control state by replaying deterministic orchestration code against that history. In the replay-based design, calls to databases, APIs, models, and tools sit behind external-work boundaries; replay reuses results that reached workflow history rather than issuing those calls again (Burckhardt et al., 2021).

Durable orchestration does not guarantee exactly-once external effects. Suppose the worker sends a subscriber webhook for event `E42`, the receiver accepts it, and the worker crashes before recording completion. A timeout and retry may send the webhook again. Temporal documents this boundary explicitly for Activities: an Activity may be retried and should therefore tolerate repeated execution, and a stable business idempotency key enforced by the called service is one way to make a non-idempotent effect safe to repeat (Temporal Technologies, 2026). The caller must reuse that key across attempts, and the called service must apply the effect and record the key atomically. If no component at the effect boundary can deduplicate, duplicate delivery remains possible; compensation or manual repair may be the honest recovery path.

For the event project, the event store remains authoritative for domain events, while state-machine state or workflow history records coordination progress. Bound total retry time and resource cost, classify permanent errors, and expose terminal states. Cancellation is a request to stop: it may not preempt an in-flight external effect and does not by itself undo one that already completed. Every approach adds durable state and operational ownership; replay-based engines also require replay-compatible code evolution and management of history growth and retention.

F.2 Durable effect retry lab

The lab below stops a worker after a webhook receiver applies its effect but before the workflow records completion. Predict the outcome, inject the crash, then compare retry keys and receiver enforcement while keeping the logical command, physical attempts, outbound calls, effects, and durable completions separate.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/appendices/durable-workflows/

The simulation holds the failure point constant. It demonstrates how one bounded retry contract behaves; it does not establish that every receiver, workflow engine, or external operation exposes the same enforcement boundary.

F.3 Audit the effect boundary

The critical question is not only whether the workflow resumes. Identify the last durable coordination fact, the external effect that may already have occurred, the stable identity reused across attempts, the component that enforces deduplication, and the terminal state exposed to an operator. If the receiver cannot enforce retry safety, record duplicate delivery as a residual risk and define compensation or repair.

F.4 References

1. Burckhardt, S., Gillum, C., Justo, D., Kallas, K., McMahon, C., & Meiklejohn, C. S. (2021). Durable Functions: Semantics for Stateful Serverless. Proceedings of the ACM on Programming Languages, 5(OOPSLA). https://doi.org/10.1145/3485510
2. Temporal Technologies. (2026). Activity Definition. Temporal Documentation. https://docs.temporal.io/activity-definition



ELEMENTS OF SCALABLE WEB DEVELOPMENT

Case studies

DEVELOPMENT SYSTEM CASE STUDY

Slideotter – Learning faster than the product converged

Slideotter is a local, structured workbench for creating and reviewing presentations with language models. I developed it with coding agents between April and June 2026, before beginning Kirjolab. It is the less successful product of the two, but it was an important development-system experiment.

Slideotter produced a capable application, a strong validation harness, and several ideas that carried into later work. It did not reach the same clarity about its first production use. The case is therefore not a story of technical failure. It is a story of learning rapidly while the product, architecture, and deployment target were all still moving.

The commit counts below describe the shape of the history, not developer productivity. A mechanical extraction and a difficult product decision are not comparable units of work.

Case at a glance

Evidence	Observation
Development interval reviewed	19 April–24 June 2026
Repository history	1,606 commits
Commits made by 15 May	1,444, about 90% of the history
Structurally named commit subjects	457, about 28% of the history
TypeScript files	0 in the initial README-only commit; 517 at reviewed head
TypeScript test or specification files	49
TypeScript and JavaScript lines in application and test areas	About 98,000
Implemented architecture decision records	58
Public product status	Alpha

What worked

The project found durable authority boundaries

Slideotter changed its implementation direction very early. The first generator produced PowerPoint files, but two days after the initial commit the project replaced that path with PDF generation. Two days later the repository began a browser studio and then migrated preview, export, and validation to a shared DOM renderer.

The replacement was expensive, but it exposed a durable principle: preview and exported output should use the same rendering authority. The current architecture keeps structured slide specifications as content, the shared DOM runtime as the rendering authority, and the server as the file-write boundary. Generated changes remain candidates until the author applies them.

Those are useful product constraints, not only code organization. They reduce the chance that an agent silently changes canonical work or that preview behavior drifts from the published artifact.

Validation treated the artifact as more than source code

Slide software can typecheck while still producing unreadable slides. The project therefore validates geometry, text fit, media, visible-text quality, browser workflows, rendered baselines, and PDF output in addition to types and tests. Its repository instructions require visual changes to be checked in the rendered artifact rather than inferred from source coordinates.

Layout rules were established early so generated slides had explicit limits for overlap, clipping, margins, and text fit. Those rules narrowed the model's search space and turned several obvious visual defects into machine-checkable failures before review. They could not establish whether a slide communicated well, so rendered inspection and editorial judgment remained separate acceptance steps.

That practice carried an important lesson into later projects: validation should inspect the thing the user consumes. For a presentation tool, the PDF is part of the behavior.

The history retained decisions and reversals

The repository contains a roadmap, a current-status document, architecture guides, manual test notes, competitive comparisons, and implemented or proposed architecture decision records. This made the rapid development legible after the fact.

The documentation also became a coordination surface for agents. The roadmap distinguishes durable direction from the next practical slice, while the status document records the live implementation. That separation is more useful than one ever-growing backlog because it tells a future agent both why the system has its current shape and what remains uncertain.

Why the product was less successful

Capability breadth arrived before a stable first use

Within the first month, Slideotter accumulated most of its history. The work covered structured slide specifications, multiple model providers, outline and theme generation, variants, deck restructuring, media workflows, browser presentation mode, PDF and PowerPoint export, an Electron wrapper, and a Cloudflare hosting direction.

These capabilities support the same broad vision, but they do not form one small hypothesis. The project learned about many parts of presentation authoring at once before repeated real use had shown which problems mattered most. Its current roadmap appropriately says that the next work should come from real studio usage, but that constraint arrived after a large solution surface already existed.

Packaging and hosting multiplied the system boundaries

The repository supports local filesystem-backed use, a command-line package, a browser studio, a macOS wrapper, and a cloud proof path. Each target is plausible in isolation. Together they introduce several storage, authentication, rendering, and release models.

The cloud setup guide still identifies production workspace authentication, real queued generation and export, and Durable Object serialization as incomplete. The Electron path likewise still needs production packaging work. Technical breadth should not be confused with a finished delivery path.

Architecture kept learning behind implementation

The repository contains 457 commit subjects beginning with terms such as `split`, `extract`, `move`, `reduce`, or `refactor`. On 17 June alone, 48 commits had that structural form. At the reviewed head, application services and workbenches still include several modules above one thousand lines.

The later extraction work improved navigability and produced explicit client, server, rendering, storage, and workflow boundaries. It also shows that feature throughput repeatedly ran ahead of ownership design. Static health scores helped direct cleanup, but they could not decide which product surfaces should exist.

The harness became a substantial product of its own

Slideotter has focused checks for many artifact and workflow properties. That makes ambitious refactoring safer. It also creates a large maintenance surface of fixtures, baselines, validators, build paths, and project instructions.

This is a development-system waterbed effect. Moving uncertainty out of generated code and into deterministic checks improves control, but the checking system itself must remain understandable and proportional to the product evidence it protects. Without real usage, it is possible to make the harness increasingly precise around the wrong workflow.

What transferred to Kirjolab

Slideotter preceded Kirjolab and changed how I approached it. Several useful practices transferred directly or conceptually:

- establish canonical artifacts and explicit write boundaries;
- keep model output reviewable until a human applies it;
- validate rendered or operational behavior, not only code;
- preserve architecture decisions and current project state in the repository;
- use small semantic commits so rapid work remains inspectable; and
- add structural feedback when tests alone cannot constrain code shape.

The warning transferred too. Kirjolab began with a stronger development harness, but it repeated the tendency to broaden capability faster than the architecture consolidated. Slideotter helped make the later refactoring possible; it did not by itself prevent the need for it.

What I would do differently

1. **Choose one delivery target first.** I would make the local browser studio the only initial runtime and defer desktop and cloud paths until repeated use justified them.
2. **Close one authoring loop.** A brief, an approved outline, a small supported slide set, explicit candidate review, and a validated PDF would define the first product.
3. **Use real decks as the roadmap.** New layout families, retrieval methods, and editing modes would follow observed failures across several presentations.
4. **Budget architecture as the system grows.** Module size, fan-in, churn concentration, and cross-workflow dependencies would be reviewed alongside artifact quality.
5. **Measure product success separately.** Time to a usable deck, corrections per slide, regeneration rate, and repeated use would complement repository and validation metrics.
6. **Prune the harness around the chosen loop.** A smaller set of high-value checks would reduce maintenance cost while the product hypothesis remained uncertain.

What Slideotter teaches

Slideotter demonstrates that rapid implementation can be a powerful learning method. It discovered useful rendering, review, validation, and agent-control patterns that would have been difficult to design in the abstract.

It also shows the limit of learning through code production alone. A repository can become technically sophisticated faster than its product can become necessary. Validation can protect a workflow, but only usage can establish whether that workflow deserves the surrounding system.

The central lesson is to separate two kinds of progress. Slideotter made substantial progress in understanding how an agent-assisted presentation workbench could be built. It made less progress in proving the smallest version that should be built and operated.

DEVELOPMENT SYSTEM CASE STUDY

Thesis Journey Tracker – Small scope made production success possible

Thesis Journey Tracker is a private dashboard for a small thesis-supervision team. It tracks students, thesis phases, meetings, topics, notes, workload, and backups without trying to become a general student information system. I developed it with coding agents beginning in March 2026, and it is in production and working well.

The project is useful as a counter-case to Slideotter and Kirjolab. All three used rapid agent-assisted development and strong validation. Thesis Journey Tracker succeeded with a smaller product boundary, a simpler runtime, and production concerns that were allowed to shape later work.

The repository evidence cannot independently establish user satisfaction or long-term reliability. The production assessment above is based on my direct use of the deployed application; the repository supports analysis of how it was built and hardened.

Case at a glance

Evidence	Observation
Development interval reviewed	22 March–30 July 2026
Repository history	282 commits
Commits made by 31 March	175, about 62% of the history
Focused security-hardening interval	29 commits on 13–14 July
TypeScript files	0 in the initial README-only commit; 160 at head
TypeScript test or specification files	25
Application, test, and script lines reviewed	About 20,500
Runtime	Cloudflare Workers and D1
Rendering model	Server-rendered HTML with limited client script
Production status	Deployed and working in the author's use

The first pass was deliberately narrow

The repository began with a README. The next commit, explicitly named "First pass by codex-5.3", added an 838-line Worker, one database migration, basic configuration, and the package setup. It was a large first implementation, but the problem was bounded: represent the supervision workflow in one private dashboard.

The same day, the history shows rapid correction rather than endless feature expansion. The project moved to TypeScript, added security and browser tests, simplified the interface, extracted database and view code, adopted server-rendered HTML components, and added Lighthouse checks. Three days later, a commit explicitly hardened import for production rollout.

This sequence resembles a spike followed by consolidation. The generated first pass made the real workflow concrete, and the following commits converted what was learned into types, modules, tests, and operating controls.

Why the scope scaled

The product says what it is not

The README describes the project as intentionally small and focused on thesis advising rather than a generic student information system. That negative boundary is consequential.

The application can add more views or improve scheduling without acquiring course registration, grading, messaging, organization-wide identity, or other adjacent institutional concerns. Agents therefore receive a smaller decision space, and each feature can be judged against one recognizable workflow.

The runtime matches the problem

The system uses one Cloudflare Worker, D1, server-rendered HTML, and small browser interaction scripts. The current project structure keeps the Worker entrypoint thin and groups routes, persistence, rendering, authentication, calendar behavior, backups, and student-domain rules by capability.

At the reviewed head, `src/worker.ts` is 447 lines even though the application has grown substantially beyond the original 838-line Worker. Complexity moved into feature-owned routes and stores rather than remaining in one entrypoint. The project still has large modules, but their names and ownership follow the user workflow.

Production use changed the kind of work performed

Later changes are less about adding visible surface area and more about making the existing system dependable. The July hardening interval includes strong independent secrets, revocable sessions, bounded request bodies, same-origin mutation checks, encrypted calendar credentials, login throttling, Markdown-injection protection, restricted calendar metadata, pinned CI dependencies, and safer backup retention.

Later commits added structured incident logging and validated database mutation outcomes. This is a healthy shift from feature discovery to operational learning. Once the application had a real delivery context, security, recovery, and diagnosis became product requirements rather than optional infrastructure.

The development system supports operation

CI mirrors a local gate

The CI workflow audits dependencies, verifies generated Worker types, typechecks, runs unit and D1-backed integration tests, exercises browser workflows, and enforces a Lighthouse performance threshold. The same workflow can run locally through Agent CI.

This alignment reduces the difference between an agent's local evidence and the repository's shared evidence. Pinned runner images, restricted installation scripts, read-only workflow permissions, and discarded checkout credentials also treat the build environment as part of the security boundary.

Deployment is intentionally separate from validation

The deployment guide separates version upload from production promotion. A connected Cloudflare build creates a deployable version, but a reviewed version is promoted explicitly after CI succeeds.

That boundary matters in agentic development. Passing a gate authorizes consideration for release; it does not have to grant an agent or build system automatic production authority. The repository also documents migrations, account creation, secret rotation, backup retention, preview-URL exposure, and rollback-relevant version handling.

Recovery is a first-class feature

The application supports JSON export and restore as well as scheduled R2 backups. Backups are written only when data changes, with periodic refresh so retention policies preserve recent restore points. The setup instructions require the R2 lifecycle rule to be inspected rather than assuming that a command succeeded.

This is validation applied to operations: the intended safety property is a recoverable dataset, not merely a successful backup API call.

Why this evidence is still bounded

Production use is stronger evidence than a working repository, but it is not universal evidence of scalability. The current application serves a private personal or small-team context. Its own security documentation accepts residual denial-of-service and cost exposure that would need edge controls under broader traffic.

The repository has one contributor and does not expose adoption, uptime, support burden, or realistic load measurements. Several modules remain large, including calendar parsing and complex view components. The case therefore demonstrates fit and sustainability at its intended scale, not readiness for an institution-wide deployment.

This distinction is important. Scalability is not maximum size. A system is successful when its development and operational model remain proportional to the demand it actually has.

Comparison with the larger projects

Project	Dominant source of learning	Result at reviewed point
Slideotter	Broad product and architecture exploration	Capable alpha; production path still uncertain
Thesis Journey Tracker	Repeated use of one bounded operational workflow	Working production tool at small-team scale
Kirjolab	Very high implementation throughput and later revision	Integrated prototype with major consolidation

The key difference is not simply repository size or commit count. Thesis Journey Tracker had a narrower answer to "for whom, for what task, and in what operating environment?" That answer constrained generation and made production feedback interpretable.

Slideotter and Kirjolab needed implementation to discover more of their product and architecture. That exploration was valuable, but it also meant that new code could answer questions that had not yet been prioritized by use.

What I would preserve and improve

I would preserve the narrow product statement, server-rendered default, capability-owned modules, local/remote CI alignment, manual production promotion, and explicit recovery path.

I would improve the evidence around the deployed system:

1. **Record operational measures.** Availability, error rate, D1 query latency, backup freshness, and restore drills would turn "working well" into inspectable longitudinal evidence.
2. **Test recovery periodically.** A backup is only proven when a representative restore is rehearsed and checked.
3. **Track change friction.** Lead time for common workflow changes and churn in the largest modules would show when current boundaries stop scaling.
4. **Add usage evidence carefully.** Privacy-preserving counts or structured interviews could reveal which views and integrations actually reduce supervision work.
5. **Reassess the exposure boundary.** If the user population or public reach grows, identity, rate limiting, audit needs, and data governance should be revisited before adding features.

What Thesis Journey Tracker teaches

Agentic development did not make the project successful merely by producing the first version quickly. The important choices were to bound the problem, consolidate the first pass, use the application in its intended setting, and let production concerns redirect later work.

The case suggests four lessons:

- A small negative product boundary can constrain agents more effectively than a long feature list.
- Simple deployment architecture preserves attention for the domain workflow.
- Production success creates new requirements around security, recovery, observability, and release authority.

- A system should be evaluated at its intended scale before it is judged against hypothetical growth.

Thesis Journey Tracker is less ambitious than Slideotter or Kirjolab, and that is part of why it works. Its development system is proportional to a real, bounded service rather than to every adjacent possibility.

DEVELOPMENT SYSTEM CASE STUDY

Kirjolab – When code production scaled faster than architecture

Kirjolab is a collaborative workspace for scientific writing. It connects portable Markdown manuscripts with references, PDF annotations, evidence, and reviewable local-model suggestions. I developed the project rapidly with Codex during July 2026.

The result was a technical success in an important sense: I reached a scope and level of integration that I could not have achieved alone in the same time. The repository also needed substantial refactoring almost immediately. That combination makes Kirjolab useful as a case study. It shows both what a disciplined agentic development setup can enable and what happens when the rate of code production exceeds the rate at which the architecture is understood.

The commit counts below describe the shape of the history, not developer productivity. A small mechanical commit and a difficult design decision are not comparable units of effort.

Case at a glance

Evidence	Observation
Development interval reviewed	10–30 July 2026
Repository history	1,433 commits
Commit subjects beginning with `refactor`	818, about 57% of the history
TypeScript files	14 in the initial template; 583 at the reviewed head
TypeScript test files	5 in the initial template; 265 at the reviewed head
Build Week implementation range	332 commits, 439 files, 88,867 insertions, and 16,140 deletions
Most concentrated restructuring interval	696 of 707 commits from 25–28 July were named as refactors

The project did not begin with an empty prompt. The initial commit came from a prepared development template with strict TypeScript, coverage checks, mutation testing, browser tests, affected-file checks, a pre-push gate, local CI, architecture decision records, feature specifications, and repository instructions for agents. Eight minutes after the initial commit, the product architecture was documented. The first scholarly workspace slice followed about an hour later.

What went right

The harness existed before the feature pressure

The most consequential early decision was to treat the project harness as part of the product. Generated code had to pass deterministic checks rather than merely look plausible. The initial harness already checked formatting, types, coverage, browser behavior, security-sensitive browser boundaries, and mutation resistance. Later changes added more focused diagnostics and faster affected-file paths.

This did not guarantee good architecture, but it kept the system repairable. When a responsibility was extracted or an external dependency replaced project-owned code, the existing tests could detect behavioral drift. Rapid generation was therefore paired with rapid rejection of invalid changes.

The product boundaries were clearer than the source boundaries

The early architectural vision defined several durable principles:

- portable Markdown and bibliography files remain the lasting artifacts;
- evidence, claims, and prose retain explicit relationships;
- derived views can be rebuilt;
- model output is a candidate rather than canonical state; and
- the researcher explicitly accepts, edits, or rejects model suggestions.

Those decisions survived extensive implementation. Although the internal organization changed, the product did not need to reverse its core authority model. This distinction matters. An early architecture does not have to predict every module, but it should identify which information and decisions must remain trustworthy.

Validation grew with the scope

Testing was not postponed until the end of the rapid phase. The history shows tests being added alongside collaboration, PDF handling, offline persistence, structured reviews, imports, exports, and model operations. Mutation coverage was raised to 82% on 13 July. Exploratory browser reviews recorded findings before fixes, and the corrected workflows were checked again.

The project also used measured spikes instead of adopting dependencies on intuition. A CodeMirror experiment checked collaboration, completion, Vim behavior, accessibility, offline use, long-document startup, and bundle cost. It was kept out of production because important physical-device and accessibility evidence was still missing and the bundle cost was substantial. A `unified-latex` experiment was rejected because it would not remove enough of Kirjolab's conversion policy and handwritten parsing responsibility. The ability to say “not yet” was part of the technical success.

The history preserves the learning

Small semantic commits, feature specifications, architecture decision records, review notes, and explicit supersession make the project unusually inspectable. The Build Week notes even distinguish the contest-period implementation from the earlier foundation.

This documentation supports a stronger account than “the agent was fast.” It shows which constraints existed, which decisions remained human, what was tested, what was reconsidered, and where the implementation changed direction.

Where rapid development created pressure

Functional correctness did not guarantee navigability

The quality gates emphasized whether behavior remained correct. They did not initially place equally strong limits on module ownership, fan-in, churn concentration, or directory growth.

By 25 July, `src/client/app.ts` had reached 12,495 lines and combined a large element registry with event binding, rendering, and feature orchestration. The later modularization RFC identified several other mixed authorities:

- a 5,636-line document Durable Object covering collaboration, files, history, export, and persistence;
- a 2,766-line review-study Durable Object combining commands and SQLite projection;
- a 2,411-line reference-library Durable Object spanning several persistence lifecycles; and
- a 1,090-line reference-library contract with 82 dependents.

The application could pass its tests while becoming increasingly difficult to navigate. This is a useful warning for agentic development: a generated test suite can constrain observable behavior without constraining the shape of the code that future agents will learn from.

Scope expanded faster than consolidation

The original vertical slice connected evidence to prose. Within days, the project also covered structured literature reviews, GitHub synchronization, LaTeX import, citation trails, collaboration, sharing, offline recovery, advanced PDF workflows, and production operations.

These were not random features. They fit the larger product direction, and many were implemented carefully. The problem was their combined arrival rate. New capabilities accumulated before the previous ones had settled into clear ownership boundaries. The architecture was learning from the implementation, but it was learning several steps behind it.

The prepared template had a cost

The initial template supplied the practices that made rapid work safe. It also began with 978 files and about 142,000 inserted lines, including vendored process material, skills, capability kits, and update infrastructure. Most of that was not product code.

This is a real trade-off rather than a simple mistake. Rich repository context gave agents explicit instructions and reusable workflows. It also increased navigation cost, search noise, and the amount of project material that could enter a model's context. I would keep the harness, but start from a more sharply pruned version of it.

The evidence was mostly internal

At the reviewed point, GitHub showed one contributor, no pull requests or issues, direct work on an unprotected `main` branch, and no public GitHub Actions run history exposed through the repository API. The repository contains strong local validation evidence, but little independent evidence about adoption, usability, production reliability, or realistic load.

Kirjolab therefore supports a bounded conclusion. It demonstrates rapid implementation, disciplined local validation, and successful architectural recovery. The history alone does not demonstrate product success or long-term maintainability.

Refactoring was the revision phase

The project history changes character after the initial delivery push. From 10–21 July there were 461 commits, mostly features, fixes, tests, and documentation. On 22 July, 51 of 108 commits were already refactors. From 25–28 July, 696 of 707 commits were named as refactors.

The work was not one undifferentiated rewrite. It included:

- extracting presentation and workflow owners from the large client coordinator;
- splitting API and persistence responsibilities;
- replacing repeated handwritten validation with bounded schemas;
- separating pure PDF-analysis mechanics from browser and Worker adapters;
- narrowing the reference-library contracts;
- measuring dependency costs and deleting duplicated infrastructure; and
- finally organizing hundreds of flat source files by product capability.

On 30 July, ADR-198 recorded that `src/client/` held 312 files and `src/domain/` held 119 files at their roots. The resulting migration moved them into shallow product-named directories without inventing package or runtime boundaries.

This is the “revise” step of the constrain, generate, validate, revise loop at repository scale. Implementation exposed the actual domains and change patterns. Refactoring converted that evidence into new constraints for subsequent work. The expensive part was that the structural feedback arrived later than the functional feedback.

What I would do differently

If I started Kirjolab again with the knowledge gained from the project, I would preserve the strict harness and core product model while changing the sequence.

1. **Begin with capability-oriented ownership.** I would establish thin composition roots and directories for authoring, references, PDFs, projects, review, and integrations before those capabilities accumulated hundreds of files.
2. **Hold the first release to one closed loop.** Evidence capture, a traceable claim, a reviewable model suggestion, and portable prose would form the initial product. Structured reviews and broad integrations would wait for evidence from that loop.
3. **Add structural feedback earlier.** File size, fan-in, churn concentration, flat-directory growth, and cross-capability dependencies would be reviewed alongside coverage and behavior.
4. **Prune the project template.** I would retain affected checks, mutation testing, local CI, ADRs, and specification rules while removing skills and capability material unrelated to the product.

5. **Create independent evidence.** Public CI results, representative research datasets, usability sessions, deployment signals, and before-and-after change-friction measurements would complement repository-owned tests.
6. **Use review checkpoints.** Even for a single-developer project, grouped pull requests or explicit architectural checkpoints would create opportunities to review the direction of a capability rather than only the correctness of individual commits.

What Kirjolab teaches

Kirjolab did not fail because it needed refactoring. Early implementation was also a way to discover the real product and its architecture. The mistake would be to treat the first working structure as evidence that the development system could sustain the same pace indefinitely.

The case suggests four broader lessons:

- A strong harness can make extraordinary development speed technically useful.
- Functional gates and structural gates answer different questions.
- Agents amplify the patterns and ownership boundaries already present in a repository.
- Refactoring is most effective when its lessons become new project constraints rather than remaining a one-time cleanup.

The technical success was not that the first architecture was final. It was that the project could move quickly, reveal where its assumptions failed, and use its validation system to turn that learning into a better structure.

DEVELOPMENT SYSTEM CASE STUDY

Webernetes – Comparing generated behavior with a real cluster

Webernetes is an experimental TypeScript implementation of a subset of Kubernetes that runs in a browser without backend cluster infrastructure. Sam Rose built it at ngrok to support durable, interactive explanations of Kubernetes rather than to replace a production cluster.

The project is relevant to scalable development because language models produced most of a large port in a short period, while acceptance still depended on human review, explicit porting rules, and behavioral comparison with a real implementation. Its strongest lesson is not that generated code can be trusted at scale. It is that a development harness can turn some compatibility claims into observable contrasts.

The figures below come from Rose's 30 June 2026 retrospective and describe that snapshot rather than the repository's later state.

Case at a glance

Evidence	Observation
Intended use	Interactive Kubernetes teaching content in the browser
Implementation boundary	A TypeScript reimplementation of a selected Kubernetes subset
Development interval in retrospective	Two months
Repository snapshot in retrospective	552 commits across 629 files
Generated code reported	Almost 100,000 lines
Tests reported	204 integration tests and 1,855 unit tests
Behavioral reference	The same integration-test operations executed against a real k3s cluster
Current project status	Experimental; APIs and supported resources may change

Commit, file, line, and test counts describe scale and process shape. They do not establish correctness, productivity, or production readiness.

The product boundary made a partial port useful

ngrok wanted visual, interactive Kubernetes content without operating short-lived clusters for every reader. Webernetes therefore simulates the parts needed by that content in the browser: an API and storage model, nodes, pods, selected controllers, service discovery, and HTTP communication between simulated containers. Images are TypeScript classes registered with the simulator rather than real images fetched from a container registry.

That is a narrower goal than implementing Kubernetes. The README's support inventory explicitly lists omitted or simplified behavior, including volumes, affinity, resource management, some service types, and much of the network stack. The simulator initially focused on the behavior needed to explain

probes. This scope makes the browser artifact useful while preventing its familiar API from becoming an unsupported claim of production equivalence.

Generation remained candidate work

Rose reports that language models generated almost all of the code and that he reviewed every line. The retrospective also describes recurring failures: models replaced specific upstream structures with simpler ones, introduced locally convenient helpers that obscured comparison, and omitted cases while porting Go table tests. Those are especially dangerous in a compatibility project because code can look reasonable while changing the behavior being copied.

The current porting rules make reviewability an architectural constraint. Ported declarations carry breadcrumbs to a pinned upstream Kubernetes revision, preserve source order and helper boundaries where possible, use shared compatibility primitives for Go concepts such as channels and clocks, and require intentional deviations to be stated. The objective is not idiomatic TypeScript in isolation. It is a mapping a reviewer can compare with the upstream implementation without reconstructing an unrelated local design.

This is a useful distinction between generation speed and accepted-change throughput. Models expanded the amount of candidate implementation, but the human still had to understand the mapping, notice unjustified deviations, and decide which subset was worth maintaining.

Shared tests created a behavioral contrast

Webernetes exposes a fake client shaped closely enough to `@kubernetes/client-node` that shared integration tests can execute the same calling code against two systems. One harness targets a real k3s cluster in Node.js; the other targets Webernetes in a browser. If an operation passes against k3s and fails against the simulator, the mismatch becomes a concrete porting problem instead of a model's opinion about its own output.

The repository instructions now strengthen that loop. For Kubernetes-facing behavior, a contributor should first observe the test fail against the simulator and pass against k3s, then implement the fix. Simulator time also goes through a controllable cluster clock rather than ambient timers, making scheduling behavior easier to pause and test deterministically.

The comparison is stronger than generated tests alone, but it remains bounded. It establishes agreement for the selected operations, assertions, versions, and environments. It does not prove complete Kubernetes parity, validate unsupported resources, reveal behavior that both paths or the shared client misunderstand, or make the browser simulator suitable for production workloads. Ported unit tests are useful regression evidence, but they are less independent because a model can mistranslate or omit both implementation and test behavior.

The artifact eventually tested its product hypothesis

The initial value proposition was interactive Kubernetes education, not the port itself. In August 2026, Rose used Webernetes for the interactive demonstrations in an article about Kubernetes probes. He reports checking the demonstrations against k3s and finding a Kubernetes bug during that comparison.

That later use matters because it closes a loop the repository metrics cannot. The simulator became part of published teaching material, exercised the behavior it was originally scoped to explain, and produced a concrete discrepancy worth investigating. It is still one authored experience rather than evidence that the simulator will fit every educational setting.

The harness did not make scale free

The retrospective reports a steep increase in model use near the end of the initial build, including more than two billion tokens and about US$1,800 in API-equivalent cost in the final week. Those figures depend on the models, accounting method, and work attempted, so they should not be generalized into a cost estimate for other ports.

More importantly, generation did not remove the cost of comprehension. Line-by-line review, upstream comparison, test-harness design, scope decisions, and mismatch investigation remained human work. The reference implementation reduced uncertainty about selected behaviors; it did not decide which behaviors mattered or who would maintain the result.

What Webernetes teaches

Webernetes shows a strong pattern for agent-assisted compatibility work:

1. Define the useful subset and name unsupported behavior.
2. Preserve a reviewable mapping to a pinned upstream source.
3. Exercise the same public operations against the candidate and an independent reference.
4. Turn mismatches into failing tests before changing the implementation.
5. Keep human review responsible for scope, fidelity, and the limits of the evidence.

The transferable asset is not the volume of generated code. It is the development system that makes selected claims inspectable and disagreements reproducible.

DEVELOPMENT SYSTEM CASE STUDY

Modern Web Guidance – Treating current knowledge as a tested dependency

Modern Web Guidance is a preview project supported by the Google Chrome and Microsoft Edge teams and the web-development community. It packages current web-platform guidance as skills that coding agents can search and retrieve while working.

The case extends the idea of a project harness beyond one repository. Instead of expecting a model's training data or a permanent prompt to contain the latest HTML, CSS, accessibility, performance, and compatibility practices, a maintained capability supplies a smaller task-relevant guide. The project then evaluates whether access to that guide changes checked outcomes.

Case at a glance

Design choice	Current approach
Knowledge source	Curated guides and discipline-level skills maintained by domain contributors
Agent access	Semantic search followed by retrieval of a selected guide
Compatibility boundary	A declared Baseline target; Widely Available is the default
Distribution	A generated installation repository and an npm-based CLI
Evaluation contrast	The same coding tasks run with and without access to the guidance
Browser evidence	Playwright assertions calibrated against reference and deliberately flawed pages
Project status	Preview, with content and tooling still evolving

This is a case about the design of a knowledge dependency and its evaluation harness. It is not independent proof that every recommendation is correct or that installing the skill improves every web project.

Retrieval keeps the working context bounded

Modern Web Guidance contains guidance for more than one hundred web-development use cases. Its top-level skill instructs an agent to search the collection for the current task and retrieve a matching guide by identifier. A developer can also use the same search and retrieve commands before installation to inspect what would enter the agent's context.

This separation matters for scale. The full library can evolve independently, while a task consumes only the patterns it appears to need. Descriptions and token counts make selection more inspectable than silently injecting the whole collection. Retrieval can still fail: the query may select the wrong guide, relevant guidance may be absent, and a concise result may omit a constraint the application needs. The selected guide therefore remains an input to review rather than an authority over the product.

Browser support becomes a project constraint

The project connects its recommendations to Baseline, a shared definition of when web-platform features become interoperable across major browser engines. A repository can state a Baseline year or another support policy in its agent instructions. Without a local target, Modern Web Guidance defaults to Baseline Widely Available and expects fallbacks for features outside that boundary.

This turns browser compatibility from an accidental property of generated code into an explicit design input. It does not replace testing on the browsers, devices, and assistive technologies used by the product's actual audience. A Baseline label describes feature interoperability; it does not establish usability, accessibility, performance, or fit with a particular framework.

The guidance has an evaluation loop

The source repository separates human-authored guidance from the machinery that distributes and evaluates it. For an evaluation-ready use case, a subject-matter expert writes the guide, a reference implementation, and natural-language expectations. The development pipeline generates a deliberately incorrect page and a Playwright grader, then calibrates the grader so the reference passes and the negative page fails.

The harness can then run the same task from the same base application in two conditions: an agent works without the guidance and with access to it. Both outputs are graded with the task's browser assertions. This guided-versus-unguided contrast tests whether the capability adds value beyond what the selected agent and model already know, rather than counting whether the agent opened a guide.

The method has useful limits:

- A generated grader can encode the wrong behavior even after calibration.
- One positive and one negative page do not cover every incorrect implementation.
- Natural-language expectations reflect the maintainers' chosen interpretation of the task.
- The current harness uses small base applications and controlled tasks rather than existing production systems.
- Results are local to the tested task, agent, model, prompt scaffold, guide revision, and assertions.

Calibration can reject an obviously broken check. It cannot turn the check into independent ground truth, so failures and surprising uplifts still require inspection.

Distribution creates an update tradeoff

The project uses two repositories: contributors work in the source repository, while a generated installation repository contains compiled skills and agent-specific packaging. The recommended CLI installation supports automatic updates. That provides a clear maintenance path for knowledge that should change with the platform.

Automatic currency and reproducibility pull in different directions. An evaluation or consequential change should record the resolved skill version or revision and the project's browser-support target. An organization may instead pin updates until review, especially because a skill can influence tool calls made with the agent's existing authority. Official provenance, popularity, and a passing evaluation reduce some uncertainty; none grants the capability permission to override local architecture, security, privacy, or user requirements.

A practical adoption pattern

A team adopting a living capability can keep the boundary explicit:

1. State the local problem and compatibility target before retrieval.
2. Inspect the selected guide and its provenance rather than trusting a match score alone.
3. Record the resolved capability revision or update policy with the change.
4. Adapt the recommendation to the project's architecture and users.
5. Validate behavior in the real application and representative browsers.
6. Re-run a fixed task cohort when the guide, model, agent, or harness changes materially.
7. Keep a removal or rollback path if the capability stops fitting the project.

This makes the external skill a reviewable development dependency rather than invisible model knowledge.

What Modern Web Guidance teaches

Modern Web Guidance demonstrates a scalable way to package fast-changing expertise: maintain it once, retrieve a bounded part when relevant, express compatibility as a local constraint, and test whether the imported knowledge improves checked behavior.

It also shows why the surrounding harness matters as much as the prose. Search, versioning, calibration, paired runs, browser checks, and update policy determine whether the guidance can be evaluated and maintained. The durable lesson is to connect imported knowledge to local evidence without confusing either the guide or its grader with proof of product quality.



ELEMENTS OF SCALABLE WEB DEVELOPMENT

Course project guide

Common Project Specification

Work Activity Tracker

This specification defines the application that your group will implement three times during the course. The required behavior remains substantially the same. The development setup changes:

1. **Establish:** locally hosted model and a single-agent workflow.
2. **Challenge:** foundation model and a single-agent workflow.
3. **Compare:** an agent fleet using a declared base model.

Each phase produces a separate implementation. The purpose is not to find one universally best model, framework, or architecture. The purpose is to compare how different development systems approach the same bounded web application and to reason about the scalability of both the application and the development process.

The phase specifications define additional constraints and deliverables. This common specification applies to all three phases.

---

1. Application Brief

Build a web application that records activity around work items.

The application contains:

- **Workspaces**, which separate independent groups or projects.
- **Work items**, which belong to a workspace and move through a small lifecycle.
- **Activity events**, which record what happened to a work item.

The domain is intentionally familiar and compact. Your group may give the application a visual theme or adapt the terminology—for example, a workspace may be presented as a classroom, game, laboratory, production line, or agent workflow. The required capabilities must remain identifiable to an auditor, but the specification does not prescribe one ideal interpretation or design.

This specification primarily describes observable behavior. When several interpretations satisfy the requirements, choose one, implement it consistently, and document any assumption that affects how the application is used or evaluated. You do not need to make the implementation unusual deliberately; differences are expected to emerge from ordinary design and development decisions.

In Compare, the same model will be applied to the development workflow itself: a fleet run can become a workspace, delegated tasks can become work items, and agent or human actions can become activity events. The earlier implementations do not need agent-specific behavior.

---

2. Required Domain Model

Your implementation must represent the following concepts. Field names and internal representations may differ.

2.1 Workspace

A workspace must have:

- a unique identifier;
- a name.

2.2 Work item

A work item must have:

- a unique identifier;
- a workspace;
- a title;
- a state in a lifecycle with at least three ordered stages corresponding to:
  - not started, such as `open`;
  - active, such as `in_progress`;
  - finished, such as `done`.

You may add descriptions, owners, labels, priorities, deadlines, or other fields.

2.3 Activity event

An activity event must have:

- a unique identifier;
- an event type;
- the workspace concerned;
- the work item concerned;
- an occurrence timestamp;
- an actor represented at least by a name or identifier.

The required event meanings are:

- creation of a work item, such as `item_created`;
- movement into the active stage, such as `work_started`;
- movement into the finished stage, such as `work_completed`.

Events may contain additional metadata. Names and representations may differ if their mapping to the required meanings is documented.

---

3. Required Behavior

3.1 Workspace management

A user must be able to:

- create a workspace;
- view the workspaces needed to use the rest of the application.

Editing and deleting workspaces are optional.

3.2 Work-item lifecycle

A user must be able to:

- create a work item in a workspace;
- move a work item from the initial stage to the active stage;
- move a work item from the active stage to the finished stage;
- see the current state of a work item.

Creating a work item must record one creation event. Each successful state transition must record the corresponding activity event.

Unsupported transitions must be rejected. For example, an item cannot skip directly from the initial stage to the finished stage. A rejected transition must not change the item state or record a successful transition event.

3.3 Activity feed

A user must be able to:

- view recorded events;
- identify the event type, workspace, work item, actor, and occurrence time;
- filter events by workspace;
- filter events by event type.

The default ordering must be documented and deterministic. Newest-first ordering is recommended.

3.4 Summary

The application must show at least one derived summary for a selected workspace:

- the number of work items in each state; or
- the number of events of each type.

The summary may be calculated when requested, maintained incrementally, or produced in another explainable way.

3.5 Validation and errors

The application must:

- reject missing or invalid required values;
- preserve a valid state when an operation fails;
- show the user an understandable error;
- avoid exposing secrets or a raw internal stack trace through the user interface.

3.6 Persistence

Workspaces, work items, their states, and activity events must survive an ordinary application restart.

The storage mechanism is your choice.

3.7 Intentionally unspecified details

The specification does not prescribe:

- page structure, navigation, or visual design;
- whether interactions use forms, links, buttons, an API, or another interface;
- exact URLs, endpoint shapes, or transport formats;
- whether rendering happens on the server, client, or both;
- identifier format or timestamp precision;
- how actors are chosen or represented beyond the required event information;
- how state and events are represented or stored internally;
- how the summary is calculated;
- module, service, or deployment boundaries;
- behavior outside the required lifecycle and acceptance scenarios.

Resolve these details in a coherent way. Record assumptions that another group would need in order to use, test, or understand the result.

---

4. Common Acceptance Scenarios

Another group and the instructors must be able to verify at least the following scenarios through the running application:

1. Create two workspaces.
2. Create work items in both workspaces.
3. Confirm that each new work item begins in the documented initial state.
4. Start one work item and confirm that it enters the documented active state.
5. Complete that item and confirm that it enters the documented finished state.

6. Attempt an unsupported state transition and confirm that it is rejected without changing the stored state.
7. View the activity feed and find one event for each successful operation.
8. Filter the feed to one workspace and confirm that events from the other workspace are absent.
9. Filter the feed by event type.
10. View the required summary and verify it against the created data.
11. Restart the application and confirm that the data remains available.
12. Submit invalid input and confirm that the result is understandable and does not corrupt existing data.

The course may provide automated checks for behavior that can be tested without prescribing a particular implementation. Passing an automated check does not replace human review.

---

5. Technical Freedom and Experimentation

You may choose:

- the programming language;
- the web framework or combination of frameworks;
- server-rendered pages, a client application, LiveView-style interaction, or another web approach;
- a monolith, client-server design, serverless design, or another explainable architecture;
- relational, document, file-based, event-oriented, in-memory-with-persistence, or other storage;
- local, containerized, cloud, edge, or other deployment;
- testing, observability, and build tools.

Next.js, Elixir, Python, and less common stacks are all valid choices. A familiar stack is not automatically safer, and an exotic stack is not automatically more innovative. Evaluation is based on what the choice enables, what it costs, how well the result satisfies the contract, and whether another group can understand and run it.

For the clearest within-group comparison, you should normally keep the same primary language and framework family across all three implementations. Different groups are encouraged to choose different stacks. A group may change its own stack between phases when it has a clear experimental reason and instructor approval, but the final analysis must then treat the stack change as a confounding factor.

Experimentation is encouraged when it serves a question the group can explain. Examples include:

- comparing server rendering with a client-heavy approach;
- using an actor model or functional language;
- trying an event-sourced representation;
- using an embedded or unusual database;
- targeting an edge or serverless runtime;
- minimizing client JavaScript;

- optimizing for local-first operation, privacy, portability, or resource efficiency.

Your group owns the operational consequences of its choices. A stack that nobody outside the group can install, run, test, or inspect has not met the project requirements, even if the application worked on the authors' machine.

No particular stack receives extra credit merely for being fashionable, complex, or unusual.

---

6. Minimum Project Harness

Each implementation must provide:

- a `README.md` that explains the application, prerequisites, setup, running, testing, and the chosen stack;
- one supported path from a clean checkout to a running application;
- pinned or otherwise reproducible dependency and runtime versions;
- an example configuration file when configuration is required;
- no committed secrets;
- a repeatable way to create enough demonstration data for the acceptance scenarios;
- a repeatable workload adapter for the shared runtime protocol;
- at least one automated validation command;
- tests covering the work-item lifecycle and its event recording;
- project instructions for the AI development workflow;
- an architecture or data-flow diagram and a short architecture note for the core event workflow;
- one maintained feature specification for the core event workflow, including its acceptance path;
- a short decision record explaining the stack choice.

Setup may use native tools, containers, a development environment definition, or another documented mechanism. Do not require an auditor to infer missing steps from knowledge held only by your team.

The README must distinguish mandatory prerequisites from optional development conveniences. If the stack requires unusual system dependencies or hardware, document them near the beginning.

---

7. Shared Runtime Workload and Progressive Scalability Analysis

You do not have to build a production-scale distributed system. You must be able to measure a stable baseline and reason about what your implementation would do under pressure.

7.1 Shared baseline workload

Run the same stack-neutral workload against implementations A, B, and C. The workload measures logical user operations from the adapter's request until the application returns an observable result, not individual internal network requests. The adapter may drive the user interface, a documented application interface, or another repeatable boundary that exercises the real application and persistence path.

Start each measured profile from this fixture:

- eight workspaces named `workspace-01` through `workspace-08`;
- fifty work items per workspace: twenty in the initial state, fifteen in the active state, and fifteen in the finished state;
- all activity events implied by creating and transitioning those items;
- four actors named `actor-01` through `actor-04`, distributed deterministically across the fixture;
- stable fixture keys that the adapter can map to implementation-specific identifiers.

A workload cycle uses this operation mix:

Share	Logical operation	Target rule
30%	Read the activity feed	Divide evenly between the unfiltered feed, one workspace, and one event type
15%	Read a workspace summary	Rotate evenly through the eight workspaces
20%	Create a work item	Rotate workspaces and actors; use a unique deterministic title
15%	Move an existing initial item to the active state	Use a distinct valid target for each operation
10%	Move an existing active item to the finished state	Use a distinct valid target for each operation
10%	Attempt an unsupported transition	Treat the documented rejection as an expected result, then verify that state and events did not change

Use a fixed operation-order seed and record it. Concurrent operations in this baseline should use distinct work items; contention on the same item belongs in a pressure scenario.

Run the following profiles:

Profile	Operations	Concurrency	Repetitions	Reset rule
Warm-up	100	1	1 unmeasured	Reset and reseed afterward
Steady	500	4	3 measured	Reset and reseed before each repetition
Burst	200	16	3 measured	Reset and reseed before each repetition

For every measured repetition, record:

- implementation identifier, commit, environment, configuration, adapter version, and operation-order seed;
- elapsed time and successful logical operations per second;
- median and 95th-percentile end-to-end operation latency overall, and by operation type when the sample is large enough;

- expected rejections separately from timeouts, crashes, invalid responses, and other unexpected errors;
- post-run checks for state, event, filter, and summary correctness;
- one resource-efficiency proxy that can be collected consistently across phases, such as peak memory, CPU time, database growth, or billed execution time.

Report each repetition and the median result rather than presenting only the best run. Absolute results from different hardware or hosted environments should not be ranked as though the environment were controlled. If a platform limit prevents a profile from completing, preserve that result as evidence and label any reduced diagnostic run instead of silently changing the protocol.

The baseline is an experimental reference, not a claim that this operation mix represents every real workload.

7.2 Progressive pressure analysis

Use the baseline workload as the starting point for the cumulative pressure analysis. Change one main condition at a time where practical so the effect remains interpretable—for example concurrency, fixture size, dependency latency, or repeated requests.

Use these scenarios as a shared pressure catalog:

- many independent workspaces;
- concurrent users changing different and identical work items;
- high and bursty event volume;
- storage growth over time;
- slow or temporarily unavailable dependencies;
- retries that may repeat a request;
- automated reads or writes whose application value, resource cost, or access pattern differs from ordinary interactive use;
- demand for recent activity and historical summaries;
- a need to retain, aggregate, archive, or delete old events.

Do not exhaustively apply the entire catalog after every phase. In Establish, select one **primary pressure** that is likely to expose an important architectural limit and define it as a documented transformation of the baseline workload. Reuse that scenario and its procedure in Challenge and Compare so the three implementations remain comparable. Challenge adds one **secondary pressure** revealed by an architectural difference or observation. Compare uses the broader catalog to interpret important differences across A, B, and C, but only relevant pressures need in-depth analysis.

For each pressure analyzed in depth, identify:

- what is likely to fail or slow down first;
- the resource, quota, dependency, or operational limit being approached;
- the first useful signal that would reveal the problem;
- one plausible response;

- at least one prepared recovery action available if the limit is reached;
- the new cost or failure mode introduced by that response.

When automated use or data exposure is the selected pressure, also distinguish cooperating, authorized, and abusive consumers; state which information is intentionally public or retained; and account for the false-positive and accessibility cost of the proposed control.

Use runtime evidence where the available workload and environment allow it. Label reasoned projections as projections rather than observations. The goal is a cumulative, justified analysis, not repeated hypothetical matrices or distributed infrastructure added without evidence.

---

8. Permitted Transfer Between Implementations

Implementations must begin in separate repositories or clean implementation directories.

You may carry forward:

- this specification and instructor-provided materials;
- acceptance scenarios and course-provided tests;
- your own written lessons and comparison notes;
- domain terminology and user-facing copy;
- technology-neutral diagrams;
- generic personal or course tooling that does not contain application behavior.

You may not carry forward:

- application source code;
- implementation-specific tests;
- generated components or configuration copied from an earlier implementation;
- an earlier repository used as the starting point for the next phase;
- prompts that contain substantial blocks of the earlier implementation.

When uncertain, disclose the reuse in the evidence package. The purpose of the rule is to create genuinely fresh implementations, not to prevent students from learning.

Third-party libraries, public starter templates, framework generators, and generic personal scaffolds are allowed if their use is documented and permitted by the course's normal rules.

---

9. Common Submission Bundle and Comparable Evidence

After every phase, submit one common bundle containing:

- that phase's implementation in its own repository or clean implementation directory;
- the comparable evidence categories in sections 9.1–9.4;

- the bounded cross-team audit report and a short response from the authors; and
- the additions named by the phase specification.

A listed requirement does not automatically require a separate file. One clearly structured artifact may satisfy several fields, and the phase specifications name only additions to this common bundle. Use the same evidence categories after every phase so later comparisons can refer to the earlier package rather than reconstruct it.

9.1 Development setup

- implementation identifier and commit;
- language, framework, storage, and deployment approach;
- model name and whether inference was local, departmental, or externally hosted;
- agent tool and version or retrieval date, together with any imported skill, plugin, or instruction bundle that materially affected the workflow;
- orchestration configuration, if any;
- local hardware class or hosted environment relevant to the result, paths by which project data may leave that environment, and any fallback policy.

When the tool exposes them and they materially affect a comparison, also record the resolved model revision or digest, quantization, runtime, allocated context, actual inference route, memory limits, accelerator residency or CPU/GPU offload, and resolved revision or content digest of imported capabilities. Mark unavailable fields as unavailable rather than spending project time estimating hidden configuration.

9.2 Effort and execution

- elapsed project time;
- estimated active human time;
- approximate model usage, cost, or locally measurable resource use;
- failed or abandoned attempts worth noting;
- important human interventions and corrections.
- one representative AI-assisted task record containing the task brief, relevant model and agent configuration, checks, trace or compact execution evidence, human correction, acceptance decision, and known limits.

Exact accounting is not expected when a tool does not expose it. State what could and could not be measured.

9.3 Product evidence

- acceptance-scenario results;
- automated test results;
- results from the repeated shared baseline workload, including the declared resource-efficiency proxy;
- known defects or incomplete requirements;

- deployment URL, local run evidence, or another agreed operational artifact.

Use the same resource-efficiency signal and procedure across phases where practical. If direct energy measurement is unavailable, report an observable resource proxy and do not relabel it as energy or carbon.

Every implementation must have a reproducible operational path. The Compare phase adds one realistic deployment outside the author's development process; its specification defines that final requirement.

9.4 Engineering explanation

- architecture and data-flow diagram;
- stack decision record;
- the phase's cumulative scalability-pressure record;
- a standardized friction audit covering setup, comprehension, change, testing, review, deployment, operations, and coordination, including the largest source of resistance and one improvement or follow-up decision;
- short reflection distinguishing observation from a possible explanation.

Do not submit an uncurated raw agent transcript as the main evidence. Preserve enough task, prompt, trace, and review information to explain consequential decisions and failures.

---

10. Bounded Cross-Team Audit

After each phase, another group will inspect the implementation for a limited period using only the repository and submitted instructions. Use the same audit tool policy in every phase and, where scheduling allows, the same auditing group. Record unavoidable changes in auditors or tools as comparison limitations.

The auditing group will:

1. attempt the documented setup;
2. run the documented validation command;
3. check a sample of the common acceptance scenarios;
4. locate where one important domain behavior is implemented;
5. identify one hidden assumption, inconsistency, or comprehension problem;
6. attempt the small change probe supplied for that phase;
7. return a concise friction report.

10.1 Change-probe protocol

The instructor releases one phase-specific probe only after the implementation has been submitted. The probes use different product language but exercise an equivalent vertical slice and acceptance burden. Their exact change shape is withheld until release.

The change-probe portion of the audit lasts thirty minutes. Begin timing after the auditor receives the prompt and has a runnable checkout, so setup friction and change friction remain separate observations. During the time box:

- use only the submitted repository, its instructions, and the common audit tool policy;
- do not ask the authors where the change belongs;
- work in a disposable branch or copy rather than modifying the submitted implementation;
- run relevant existing checks before adding a focused automated test when time permits;
- stop when the time expires, even if the change remains incomplete.

If the submitted setup cannot be made runnable during the audit, record the probe as **not attempted because setup blocked it**. An estimate may be included as commentary, but it does not count as completed change evidence.

10.2 Friction report

Return any patch or diff together with a report containing:

- setup outcome and setup time, reported separately from the probe;
- time required to identify the likely user-interface, application, domain, persistence, and test locations;
- probe outcome: complete, partial, blocked, or not attempted;
- acceptance checks completed and still missing;
- files or boundaries changed and any duplicated change that appeared necessary;
- commands run and their results, including existing regressions;
- assumptions, documentation gaps, and remaining uncertainty;
- the first point at which progress became slow or blocked.

The audit is not a full code review and does not transfer ownership. Auditors are not expected to become experts in the chosen stack. Difficulty caused by an unfamiliar technology is useful evidence, but the report should distinguish ordinary learning from missing documentation or accidental complexity.

The authors may correct factual misunderstandings in the report, but they should not erase evidence that the system was difficult to understand.

---

11. Optional Extensions

Complete the common requirements before adding extensions. Suitable extensions include:

- authentication and roles;
- work-item assignment;
- comments or additional event types;
- search, pagination, or richer filters;
- import and export;

- real-time updates;
- offline behavior;
- an API for external event producers;
- accessibility improvements;
- localization;
- richer observability;
- import and visualization of an agent-workflow trace;
- explicit idempotency support;
- retention, archival, or deletion policies;
- performance or resource-efficiency improvements.

Optional scope must be reported separately so that a larger feature set is not mistaken for a better result.

---

12. Common Evaluation Principles

Every phase is evaluated using these principles:

- required behavior works;
- the stack and architecture are understood and justified;
- setup, testing, and operation are reproducible;
- scalability claims are tied to evidence and explicit trade-offs;
- AI-generated work is reviewed rather than accepted by default;
- reporting is honest about failures, intervention, and uncertainty;
- optional output volume does not substitute for the common core;
- another group can inspect the project without relying on private explanations.

The phase specifications describe the emphasis and deliverables for each build.

Establish Project

Implementation A: Local Single Agent

The common project specification applies to this phase.

---

1. Purpose

Establish creates the first implementation and the comparison baseline.

Build the Work Activity Tracker using a locally hosted model through a single-agent workflow. The goal is not to demonstrate that local models are inherently weak or strong. The goal is to experience what this particular model and environment make easy, difficult, fast, slow, private, controllable, or resource-intensive.

Complete and submit implementation A before lecture 4. The public discussion during lecture 4 will use the submitted applications, benchmark results, and architecture diagrams as concrete cases.

Keep the implementation deliberately modest. A complete common core with clear evidence is more valuable than a large unfinished application.

---

2. Development-System Constraint

You must use:

- only the approved local-model endpoint or named departmental local-model service; every other inference route and automatic model fallback must be disabled or fail closed;
- one coding-agent process at a time;
- the model, context, and tool configuration documented in your evidence package.

You may:

- discuss, plan, write, test, and review work as a group;
- use ordinary deterministic development tools;
- restart the agent or begin a new session when needed;
- use non-generative documentation and search tools allowed by the course.

You may not:

- use a foundation model to generate, repair, review, or explain application code;
- delegate work to subagents or run coding agents in parallel;
- copy a prior implementation of the course application.

If the local model cannot complete a task, the group may implement or correct it manually. Record consequential human intervention instead of hiding it.

---

3. Required Work

3.1 Implement the common core

Complete the required domain model, behavior, harness, and acceptance scenarios from the common specification. Before expanding optional scope, describe the first closed product loop and list the capabilities deliberately deferred until that loop produces evidence.

If the implementation begins from a starter template, record the template source and starting revision before pruning it. Classify inherited material as:

- **keep** because it protects the closed product loop or its development harness;
- **replace later** because it is still the only working seam;
- **remove** because it is distribution-only material with no current owner; or
- **decide** because removal would settle a product or architecture question that remains open.

Review the exact removal set before editing, especially when automation or an agent performs the pruning. The template is a distribution artifact, not the finished architecture of the product.

3.2 Establish the evidence baseline

Record the comparable evidence fields in the common submission bundle while the work is happening. Do not attempt to recreate model usage, failed attempts, or human intervention at the end.

3.3 Record the stack hypothesis

In the stack decision record, explain:

- why the chosen stack is suitable for this application;
- what the group wants to learn by using it;
- what setup, deployment, scalability, or maintainability risks it creates;
- how well the local model appears to understand the stack.

3.4 Analyze initial scalability

Produce two bounded scalability claims: one about runtime demand or data and one about contributors or change. For each, name the maintained goal, pressure, range or scenario, boundary, constraints, likely first limit, first useful signal, candidate response, expected trade-off, supporting or challenging evidence, and one intentionally excluded dimension.

Choose one primary pressure from the shared catalog that is likely to expose an important limit in implementation A. Use the runtime or data claim as the starting hypothesis for that choice, and retain the contributor or change claim for the later friction and development-workflow comparison. Define it

as a documented transformation of the shared baseline workload that can be repeated for implementations B and C. Run it where the available workload and environment allow; otherwise mark the result as a reasoned projection.

Identify the likely first bottleneck, the resource or dependency approaching its limit, the first useful operational signal, one plausible response or recovery action, and the trade-off it introduces. You are not expected to add distributed infrastructure.

---

4. Deliverables

Submit the common submission bundle for implementation A plus:

- a capability ownership card for the main event workflow;
- a short local-model workflow reflection.

The workflow reflection should answer:

- What did the local model handle well?
- Where did it require smaller tasks, more context, correction, or manual work?
- Which limitation came from the model, which from the harness, and which from the project itself?
- What evidence supports those interpretations?

---

5. Evaluation Emphasis

This phase emphasizes:

- completion of the common behavioral core;
- a reproducible project harness;
- appropriate scope and technical clarity;
- honest baseline evidence;
- the ability to distinguish local-model behavior from hardware, tooling, and project conditions.

Raw generation speed and feature count are not goals.

Challenge Project

Implementation B: Foundation-Model Single Agent

The common project specification applies to this phase.

---

1. Purpose

Challenge creates a fresh implementation of the same Work Activity Tracker using a foundation model through a single-agent workflow.

The purpose is to compare implementation B with the local-model baseline. A more capable model may generate more code, infer more context, or recover from problems differently. It may also introduce unnecessary complexity, plausible-looking defects, greater cost, or more demanding review work.

This is a new implementation, not a refactor of implementation A.

Complete and submit implementation B before lecture 7. The public discussion during lecture 7 will compare the Challenge results before the course introduces the fleet workflow for implementation C.

---

2. Starting Conditions

Begin in a new repository or clean implementation directory.

Use:

- the same common specification and acceptance scenarios;
- the transfer policy in the common specification;
- the approved foundation model;
- one coding-agent process at a time;
- normally the same primary stack chosen for Establish.

Keeping the primary stack stable makes the comparison easier to interpret. A group may change it with instructor approval when the change supports a clear learning question. The final comparison must then treat the stack change as a confounding factor rather than attributing every difference to the model.

---

3. Development-System Constraint

You may:

- use the approved foundation model for planning, implementation, testing, explanation, and review;

- begin new agent sessions when needed;
- use ordinary deterministic development tools;
- use the permitted shared artifacts described in the transfer policy.

You may not:

- copy application code or implementation-specific tests from implementation A;
- use an agent fleet, subagents, or parallel coding agents;
- describe implementation B as a foundation-model result without recording substantial manual correction.

The group remains responsible for every submitted decision and line of code, regardless of who or what generated it.

---

4. Required Work

4.1 Implement the common core again

Complete a fresh implementation of the required domain model, behavior, harness, and acceptance scenarios.

4.2 Repeat the evidence collection

Use the same evidence categories and measurement units as Establish. Preserve failed attempts and human corrections that help explain the result.

4.3 Compare the two single-agent workflows

Compare implementations A and B in terms of:

- acceptance-scenario completion;
- architecture and codebase shape;
- test quality and known defects;
- setup and audit friction;
- elapsed and active human time;
- model latency, usage, cost, or local resource use where measurable;
- amount and type of human intervention;
- optional scope completed.

Do not present correlation as proof. The group has more domain knowledge during Challenge, and this practice effect is part of the comparison's limitations.

4.4 Apply runtime pressure

Repeat the primary pressure selected in Establish against implementation B using the same workload and procedure where practical. Then select one secondary pressure from the shared catalog that is exposed by an architectural difference or by evidence from A or B. Analyze how both implementations would respond to that secondary pressure; run it when practical and label projections clearly.

Compare the cumulative pressure evidence for A and B:

- Did the new implementation make different architectural assumptions?
- Does it fail or slow down in a different place?
- Did the foundation model introduce infrastructure or abstractions that the evidence does not justify?
- Which implementation responds better to the pressures studied, and according to what evidence?

For one queue, pool, request path, or other conserved-work boundary in A or B, apply Little's Law over one consistent observation window. Record average work in progress (`L`), average completion rate (`λ`), and average time in the system (`W`); check whether `L = λW` is approximately coherent; and state whether arrivals and completions were sufficiently balanced for the relationship to be useful. Use the result to interpret the boundary, not to claim a cause of delay or stability outside the observed conditions.

Connect one failed assumption or approaching limit to a justified response pattern such as an index, derived view, idempotency boundary, bounded queue, partition, quota, backpressure, rate limit, or graceful degradation. At this phase the response may remain a design hypothesis, but name the new correctness, failure, operational, or recovery obligation it would introduce.

Complete the chapter's documented-case transfer and pressure map as part of this cumulative pressure record. Link the selected source, separate reported facts from interpretation and project transfer, trace the failure across the relevant web boundaries, and identify the assumption, amplification path, user-visible effect, first useful signal, recovery mode, and evidence that recovery is complete.

---

5. Deliverables

Submit the common submission bundle for implementation B plus:

- the architecture hypothesis for implementation B;
- an updated capability ownership card for the main event workflow;
- the documented-case transfer, pressure map, and Little's Law check;
- a concise A–B comparison.

The A–B comparison should distinguish:

- observed differences;
- plausible explanations;
- alternative explanations and confounding factors;
- what the available evidence cannot establish.

6. Evaluation Emphasis

This phase emphasizes:

- a genuinely fresh and complete implementation;
- consistent evidence across both phases;
- careful comparison rather than model enthusiasm;
- architectural and scalability reasoning;
- awareness of practice effects, scope changes, and human review.

A larger implementation B is not automatically a better implementation.

Compare Project

Implementation C: Agent Fleet and Final Analysis

The common project specification applies to this phase.

---

1. Purpose

Compare creates a third fresh implementation of the Work Activity Tracker through an agent fleet and then analyzes all three implementations and development workflows.

A fleet is an orchestration strategy, not a model category. The central question is whether decomposition, specialization, and parallelism improve the result enough to justify coordination, duplicated work, integration, review, and resource cost.

Complete and submit implementation C before lecture 9. The public discussion and brief recap during lecture 9 use the fleet debugging views and final comparisons as evidence.

---

2. Starting Conditions

Begin in a new repository or clean implementation directory.

Use:

- the same common specification and acceptance scenarios;
- the transfer policy in the common specification;
- a declared base model;
- normally the same foundation model and primary stack used for Challenge.

Keeping the model and stack stable makes B–C primarily a comparison of orchestration. If the available fleet tooling requires another model or stack, document the difference and treat it as a limitation.

---

3. What Counts as a Fleet

For this project, a fleet must include:

- at least two independently operating agent instances or delegated agent executions;
- explicit responsibility or task boundaries;
- a defined output from each delegated task;
- at least one handoff, integration point, or parallel branch;
- human or agent review before the integrated result is accepted.

Asking one agent to write from several fictional perspectives does not count as a fleet.

More agents are not automatically better. Use the smallest fleet that allows the group to study a real coordination question.

---

4. Fleet Plan

Before implementation, record:

- the task graph or workflow;
- each role or delegated responsibility;
- allowed tools, files, or modules where relevant;
- shared state that can survive between delegated runs, including who can write and later read it, how writes are attributed, how long it persists, and how it is cleared;
- when session orchestration and resource access are separate, their policy ownership and precedence, bypass paths, and the authenticated identity, scoped delegated authority, and task, budget, policy, and trace identifiers passed across the boundary;
- expected output and validation for each task;
- dependencies and possible parallel work;
- who or what owns integration;
- where human approval is required;
- the conditions for reducing or stopping the fleet.

Role names such as scout, planner, implementer, and reviewer are examples, not requirements.

The plan may change. Preserve important changes and explain why they were made.

---

5. Required Work

5.1 Implement the common core again

Complete implementation C through the documented fleet workflow.

5.2 Evaluate coordination

In addition to the common evidence, record where possible:

- number and purpose of delegated tasks;
- work performed in parallel;
- duplicated exploration or implementation;
- handoffs and integration failures;
- conflicts or incompatible assumptions;

- idle time or blocked tasks;
- total model usage across the fleet;
- active human integration and review time;
- defects found after individual agents considered their work complete.

5.3 Turn the fleet trace into debugging data

Export or transform evidence from the actual fleet run into a documented machine-readable trace. JSON is recommended, but another portable format is acceptable. Use the downloadable portable fleet-trace example to study the minimum schema, application mapping, redaction record, and repeat-import contract. It is a transformation target rather than a required internal storage schema or a universal export format.

Map the fleet workflow to the application concepts:

- the project or fleet run becomes a workspace;
- delegated tasks become work items;
- agents and human participants become actors;
- task status maps to the documented lifecycle stages;
- delegation, start, completion, handoff, review, failure, retry, validation, and human intervention become activity events where the source trace supports them.

The trace format may contain additional relationships and metadata, such as parent tasks, dependencies, model information, duration, token use, cost, tool calls, validation results, or error details.

Provide a repeatable way to load the transformed trace into implementation C. Use the application to inspect the run and answer at least two debugging questions, such as:

- Where did work stall, fail, or require a retry?
- Which handoff or dependency delayed progress?
- Where did agents duplicate work or make incompatible assumptions?
- Where did validation or review reject apparently completed work?
- Where and why did a human intervene?
- What became the bottleneck as concurrent work increased?
- Can every physical model or tool attempt, including denied, failed, retried, and cancelled attempts, be linked to its delegated run, actor, policy result, and usage or cost provenance, and can that run be assigned an explicit outcome?

Present the findings through the application's existing feed, filters, summaries, and any small additional view needed to make the evidence understandable. A real-time integration, generic adapter for every agent tool, polished graph editor, and interactive step debugger are not required.

Treat imported traces as potentially sensitive. Remove secrets and unnecessary prompt or source-code content before submission.

5.4 Compare one agent with the fleet

Compare implementation C with implementation B:

- Which tasks benefited from delegation?
- Which tasks were too coupled, ambiguous, or small to delegate?
- Where did the fleet increase throughput?
- Where did sequential work or review capacity become the bottleneck?
- Would one careful agent or a human have been preferable for any task?

Apply at least one relevant scalability law or formal model to the workflow. Amdahl's Law, Little's Law, queueing, backpressure, or the Universal Scalability Law may be appropriate.

5.5 Analyze the three applications

Repeat the primary pressure against implementation C using the same workload and procedure where practical. Compare A, B, and C under that shared pressure. Then use the broader pressure catalog to select only the additional scenarios needed to explain consequential architectural differences. You do not need to apply every catalog scenario exhaustively to every implementation.

The final analysis must address:

- functional completeness and defects;
- architecture and data flow;
- scalability, reliability, maintainability, simplicity, and resource efficiency;
- setup, comprehension, change, testing, review, operational, and coordination friction;
- runtime measurements and their limits;
- model and orchestration evidence;
- optional scope;
- the influence of practice, hardware, stack, tools, and other confounding factors.

The final result should support a conditional engineering decision, not declare a universal winner.

Apply at least one scalability law or formal model to an application-architecture decision raised by the primary or selected pressures. Name the variables and assumptions, show the calculation or reasoning, connect the result to runtime evidence or a stated pressure, and explain what the model does not capture. This application analysis is separate from the workflow model required in section 5.4.

5.6 Implement and evaluate one scaling response

Select implementation A, B, or C and one pressure supported by runtime evidence or a credibly projected limit that the available environment can exercise. Preserve the accepted revision and its baseline evidence, then implement the smallest justified response to that limit. The response may be an index, bounded cache, idempotency boundary, admission control, queue with an explicit pending state, tenant isolation mechanism, or another technique justified by the evidence. Distributed infrastructure is not required.

Keep the relevant fixture, workload, correctness checks, and measurement procedure unchanged. Rerun the common acceptance scenarios and the selected pressure, report every repetition, and compare latency, throughput, errors, correctness, and the chosen resource-efficiency proxy before and after. Exercise one failure or recovery obligation introduced by the response. State whether the result supported, challenged, or left the original hypothesis unresolved, and preserve an unsuccessful result instead of silently replacing it with a more elaborate design.

5.7 Validate one realistic deployment

Select implementation A, B, or C based on the evidence collected so far. Deploy the revision selected after the scaling-response experiment in an instructor-approved realistic environment outside the author's development process and rerun the shared baseline workload. Record the environment and configuration, operational steps, result, difference from the local run, first observed constraint, and what the run does not prove about larger scale.

---

6. Deliverables

Submit the common submission bundle for implementation C plus:

- fleet plan or task graph;
- a compact selected execution trace;
- the documented transformed trace and a repeatable way to load it;
- evidence from the fleet debugging view and at least two findings;
- fleet coordination measurements and analysis;
- one application-level formal-model analysis and one workflow-level formal-model analysis;
- before-and-after evidence for one implemented scaling response, including its new failure or recovery obligation;
- deployed workload evidence from one selected implementation;
- a target architecture and scaling trade-off table that distinguish implemented evidence from remaining proposals;
- a completed evidence matrix and final decision narrative that together form the final A–B–C comparison.

The selected trace should show consequential delegation, validation, integration, failure, or human intervention. Do not submit an uncurated transcript dump in place of analysis.

---

7. Final Comparison

The final comparison should answer:

1. What remained stable because the specification was stable?

2. What differed between the three implementations?
3. What evidence suggests that model capability contributed to the A–B differences?
4. What evidence suggests that orchestration contributed to the B–C differences?
5. What alternative explanations remain?
6. Which application architecture responds best to the primary and selected scalability pressures, and why?
7. Under what project constraints would the group choose:
    - a local single agent;
    - a foundation-model single agent;
    - an agent fleet?
8. What remained a human responsibility in every phase?
9. What did the fleet debugging view reveal that was difficult to see from the final code or raw transcript alone?

Use tables and diagrams where they make exact comparisons easier to understand.

---

8. Evaluation Emphasis

This phase emphasizes:

- a bounded and inspectable fleet;
- successful integration and validation;
- a useful self-observation loop in which implementation C exposes the workflow that produced it;
- evidence about coordination cost as well as throughput;
- application of scalability concepts to both software and workflow;
- causal humility in the three-way comparison;
- a defensible conditional decision rather than a tool ranking.

Fleet size, transcript length, generated code volume, and architectural complexity do not earn credit by themselves.

Course Development Environments

This file defines the operational choices that must be published for each course run. Model and agent products change faster than the book's architectural and measurement concepts, so the current assignments belong here rather than in permanent chapter prose.

Publication rule

The instructor must complete and publish the table below before the affected project begins. A phase does not begin until every student has a supported access path, a declared fallback for service failure, and a cost policy that does not require unannounced personal spending or hardware.

Phase	Required assignment before launch
Establish	Approved local model artifact or departmental endpoint; resolved revision where exposed; runtime and quantization where applicable; supported hardware or hosted allocation; single-agent harness; context and tool limits; network and data-boundary policy; fallback and support contact
Challenge	Approved foundation model and endpoint; single-agent harness and enforcement rule; account or credit provision; data-boundary and retention policy; tool limits; fallback and support contact
Compare	Approved fleet harness; declared base model, preferably the Challenge model; maximum concurrent agents and total budget; isolation and external-action policy; trace export or transformation path; fallback and support contact

The instructor should publish one filled table per course run with the following fields:

Field	Establish	Challenge	Compare
Model and resolved revision or declared route	To be announced	To be announced	To be announced
Agent harness and version	To be announced	To be announced	To be announced
Orchestration rule	Single agent	Single agent	To be announced
Supported execution environment	To be announced	To be announced	To be announced
Student cost limit and provided credit	To be announced	To be announced	To be announced
Project-data egress and retention	To be announced	To be announced	To be announced
Tool, network, and external-action limits	To be announced	To be announced	To be announced
Fallback when the assigned service is unavailable	To be announced	To be announced	To be announced
Support and exception path	To be announced	To be announced	To be announced

Comparison rules

- Keep the Challenge model as the Compare fleet's base model when practical so B-C primarily changes orchestration.
- If a model, route, harness, or hardware allocation changes, record the actual configuration and narrow the comparison claim.
- Do not rank local inference latency across unequal student hardware as though the environment were controlled.

- Manual implementation remains legitimate and must be reported; access failure should not pressure students to conceal work or buy an unapproved service.
- When the assigned tool does not expose a requested configuration field, record it as unavailable rather than estimating it.

This operational sheet may change between course runs. The behavioral specification, evidence categories, and assessment rubric should remain stable unless a revision is announced before students begin the affected phase.

Course Assessment Rubric

This rubric turns the course learning outcomes and project evidence into student-facing scoring rules. It applies to every course run unless the published course environment or learning-platform instructions announce a revision before the affected phase begins.

Project scoring: 30 points per phase

Each project receives 20 points from common criteria and 10 points from its phase-specific criteria. The same evidence may support several criteria, but the same accomplishment is not counted twice within one criterion.

Common criteria: 20 points

Criterion	Points	Full-credit evidence
Required web behavior and correctness	0-6	The common acceptance core works through the running application; persistence and invalid transitions behave correctly; known defects and missing evidence are reported.
Reproducible setup and operation	0-4	An unfamiliar group can follow the supported setup, run the application and checks, obtain demonstration data, and understand the declared environment and operational limits.
Scalability evidence and reasoning	0-5	Workloads, boundaries, conditions, distributions, limits, signals, formal relationships where required, trade-offs, and unsupported claims are handled explicitly.
Architecture and change capacity	0-3	Authority, request and data paths, contracts, ownership, consequential decisions, and change or recovery obligations are coherent and reviewable.
Responsible development-workflow evidence	0-2	Material AI assistance, failed attempts, human correction, acceptance, resource use where measurable, and known limits are reported compactly and honestly.

Use the following general interpretation within each point range:

- **Full credit:** complete, internally coherent, bound to the submitted revision, and supported by relevant evidence.
- **Most credit:** the central outcome is present, but one important condition, check, explanation, or limitation is weak.
- **Some credit:** partial evidence exists, but the result is difficult to reproduce, compare, or defend.
- **No credit:** absent, unusable, contradicted by the artifact, or presented without the evidence needed to assess it.

A visually polished or feature-rich application does not compensate for failure of the common behavioral core. Conversely, a modest implementation can earn full credit when it satisfies the contract and supports its decisions with strong evidence.

Establish-specific criteria: 10 points

Criterion	Points	Full-credit evidence
Local single-agent constraint and project harness	0-4	The declared local-model workflow is followed; instructions, feature and architecture material, commands, and checks make accepted work reviewable; manual work is disclosed.

Criterion	Points	Full-credit evidence
Baseline and bounded scalability claims	0-4	The two initial claims, shared workload, primary pressure, observation baseline, and evidence limits provide a repeatable starting point for later phases.
Ownership and local-workflow reflection	0-2	The ownership card and reflection distinguish model, harness, hardware or environment, project, and human contributions without turning impressions into causal claims.

Challenge-specific criteria: 10 points

Criterion	Points	Full-credit evidence
Fresh and comparable foundation-model implementation	0-3	Implementation B starts cleanly, respects the single-agent and transfer constraints, completes the common core, and records scope or configuration differences.
Architecture, friction, and pressure analysis	0-5	The architecture hypothesis, ownership card, standardized audit, primary and secondary pressures, documented-case transfer, recovery reasoning, and Little's Law check form one coherent analysis.
A-B interpretation	0-2	Observations, plausible explanations, practice effects, stack or hardware differences, and unresolved alternatives are separated clearly.

Compare-specific criteria: 10 points

Criterion	Points	Full-credit evidence
Fleet design, integration, correction, and trace	0-3	Independent agent work, task and authority boundaries, integration, one correction loop, transformed trace, and debugging findings are inspectable and bounded.
Applied scaling and deployment	0-5	Application- and workflow-level models, one implemented before-and-after response, its new failure or recovery obligation, realistic deployment, rerun workload, and target design are supported by evidence.
Final engineering judgment	0-2	The evidence matrix and decision narrative compare A, B, and C, label observation, inference, and proposal, reject alternatives, and state the next boundary and measurement.

Participation and learning diary: 3 points per lecture

Evidence	Points
Participates in the live lecture or completes the announced equivalent remote activity	1
Records one concrete idea, model, failure, or trade-off from the lecture in the learning diary	1
Connects that idea to project evidence, a prior assumption, or a decision the student would revisit	1

The diary entry may be written, audio, or video. Production polish is not graded; specificity and reflection are.

Self- and peer-assessment: 12 points per phase

Criterion	Points	Full-credit evidence
Evidence-based self-assessment	0-3	Identifies concrete contributions, decisions, missed commitments, learning, and next improvement without equating activity with value.
Useful peer feedback	0-3	Gives each teammate specific, respectful feedback connected to observable collaboration, review, integration, or project evidence.
Response to feedback and group reflection	0-3	Summarizes agreements and differences, records one adjustment, and identifies a groupworking condition that affected the result.

Criterion	Points	Full-credit evidence
Reliable contribution and shared ownership	0-3	Instructor judgment based on corroborated project history, submitted evidence, self-assessment, and peer reports shows that the student contributed and supported shared acceptance of the work.

Peer ratings are evidence, not automatic votes on another student's grade. The instructor resolves unsupported, retaliatory, mutually inflated, or contradictory reports against the project record and may ask for clarification. Private circumstances should be handled through the course's ordinary support process rather than disclosed to teammates in detail.

Passing and final grade

The maximum is 153 points:

- projects: `3 × 30 = 90`;
- participation and learning diaries: `9 × 3 = 27`;
- self- and peer-assessment: `3 × 12 = 36`.

To pass, a student must:

1. receive at least 10 points from each of the three projects; and
2. receive at least 50% of the available total points.

Let `p = 100 × earned_points / available_points` after any formally approved absence adjustment. The final grade is:

Percentage	Grade
Below 50%	Fail
50% to below 60%	1
60% to below 70%	2
70% to below 80%	3
80% to below 90%	4
90% or above	5

Keep the original evidence and scoring record when a factual correction, approved accommodation, or formal reassessment changes a result.